SKALA

KDT Backend

# 2. REST API와 Springboot

2026.01

본 문서는 SK(주)C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C



# 목차


**3일차 (오후)**

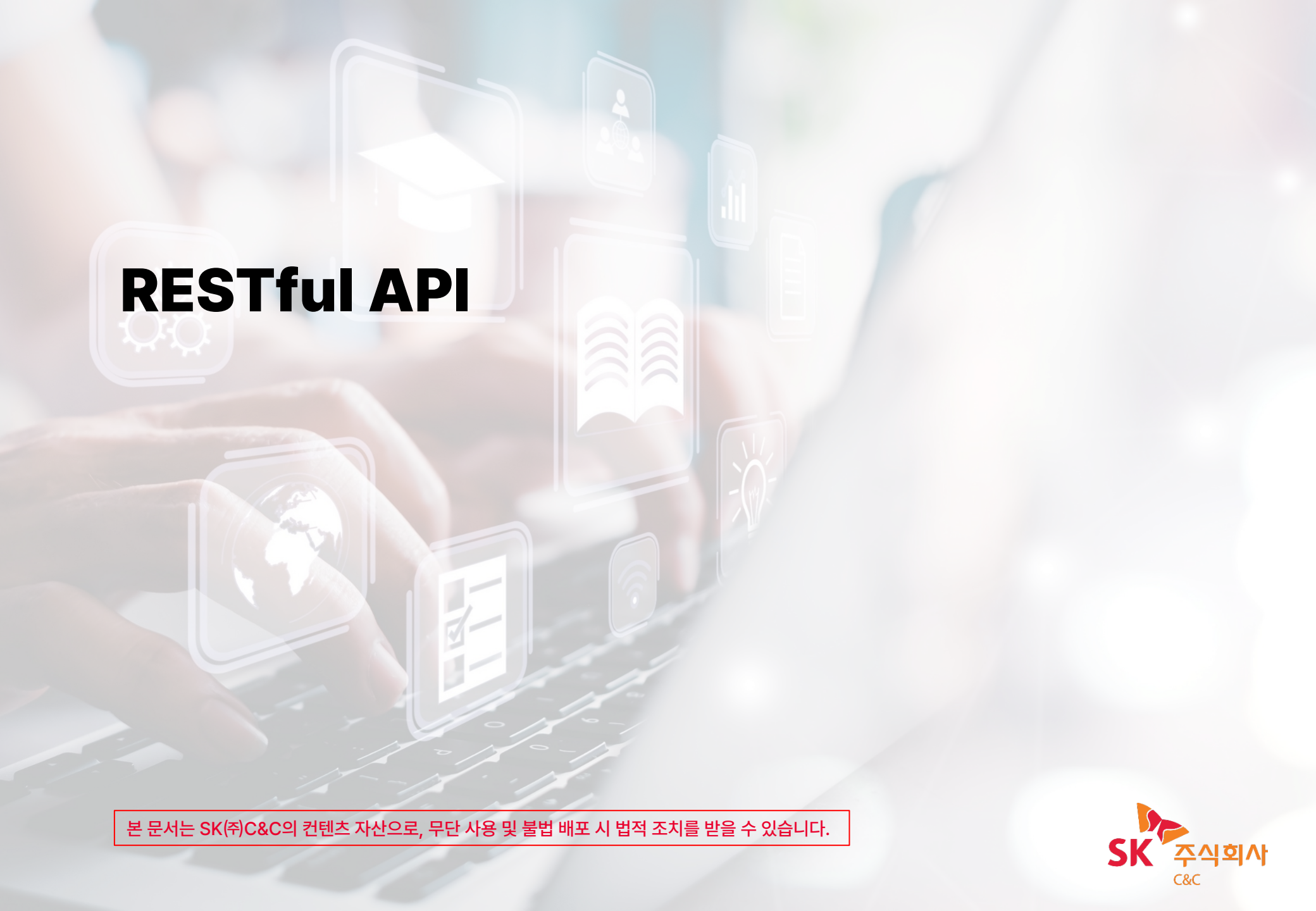
- Rest API
- spring 개요
- Spring boot 만들기
- 컴포넌트 스캔
- Spring MVC
- Controller
- Service & Repository
- Spring MVC F/W 이해 (실습)

**4일차**

- Lombok
- properties 관리
- IoC 와 DI
- IoC Container
- IoC & DI 실습
- Proxy Pattern
- AOP
- 입력값 검증
- Async
- 코드 검토하기

**5일차**

- JPA
- JPA Repository
- 관계 매핑
- Entity Manager
- Transactional
- Rest API 문서화
- Actuator
- Quiz
- 코드 검토하기



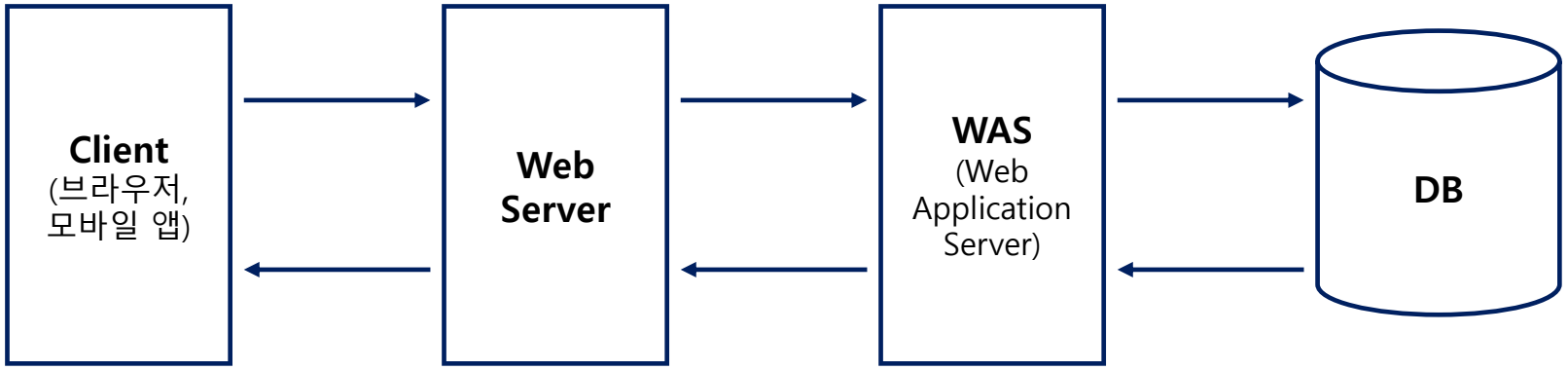
# RESTful API

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# 웹 애플리케이션

- 클라이언트 (사용자)와 서버 사이의 통신을 HTTP 프로토콜 등으로 데이터를 주고 받으며 동작하는 프로그램
- 웹 어플리케이션은 웹 브라우저를 통해 접속할 수 있음
- 클라이언트: HTML, CSS, JavaScript 와 같은 웹 기술로 구현 (Angular, React, Vue.js 등의 프레임워크 사용)
- 서버 : Java, Node.js, Python 등의 프로그래밍 언어로 구현 (Spring Boot, Express, Django 등의 프레임워크)

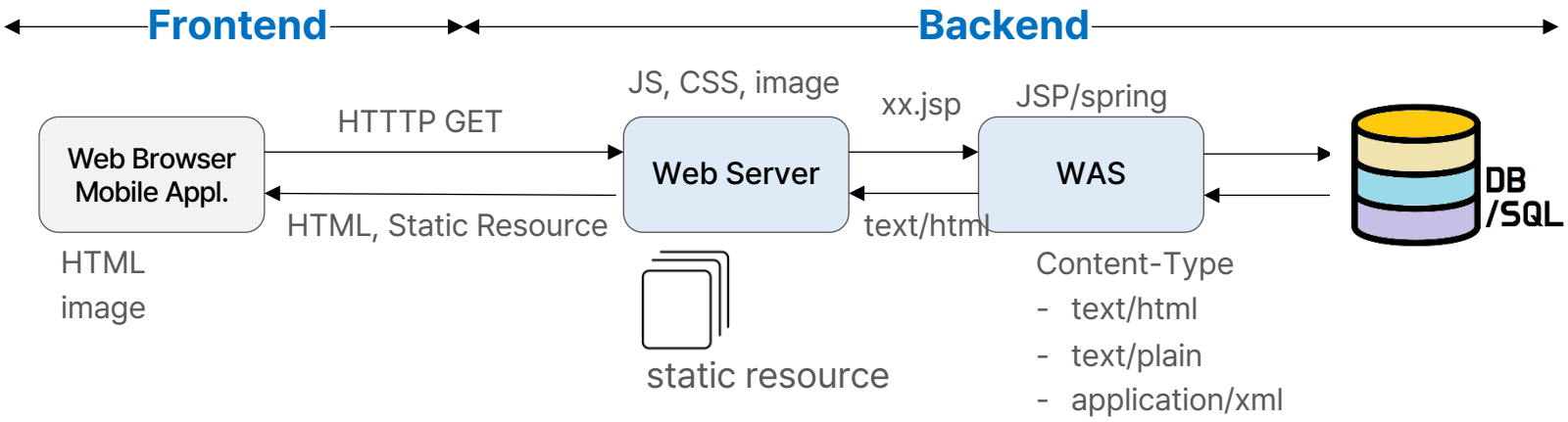


# [참고] Web Server와 WAS

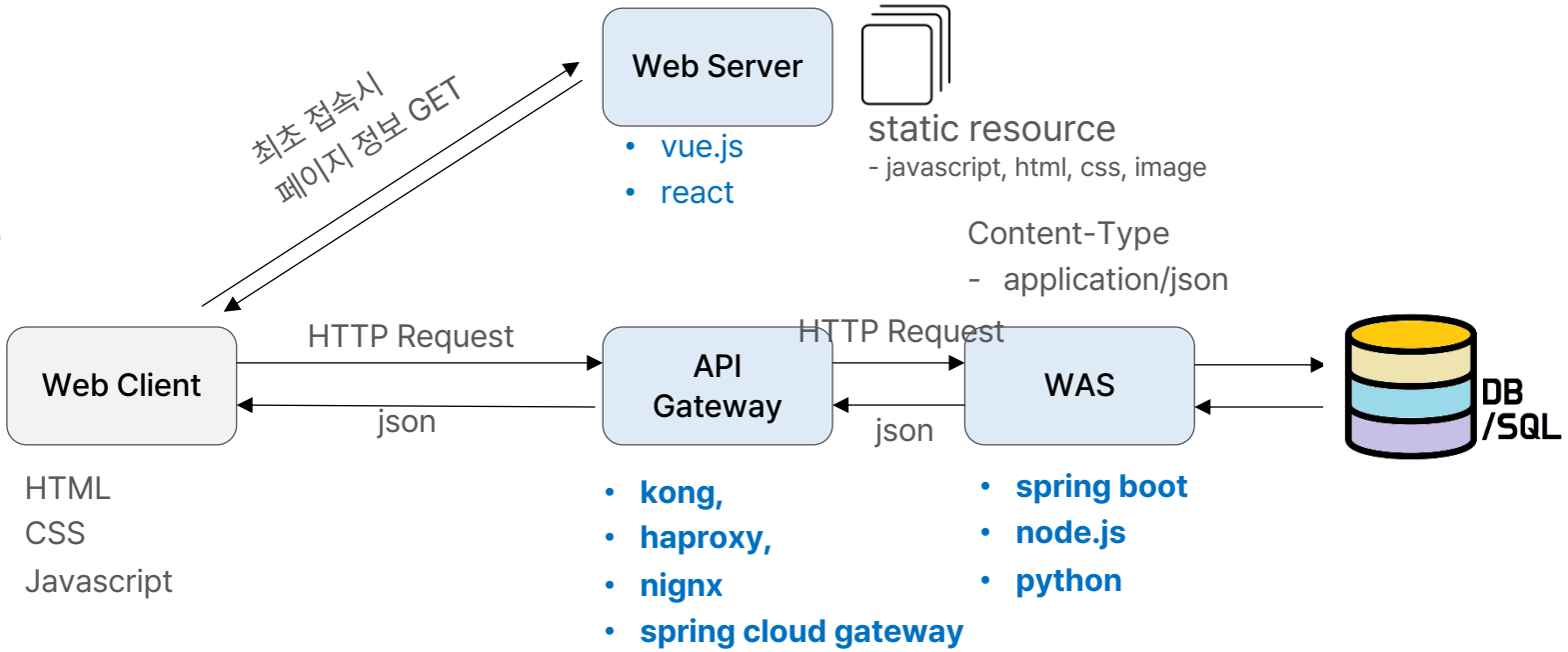
| 구분 | Web Server | WAS(Web Application Server) |
|---|---|---|
| 주요 기능 | 정적 컨텐츠 제공 | 동적 컨텐츠 제공 |
| 역할 | HTML, CSS, JS, 이미지 등 정적 파일 반환 | 비즈니스 로직 처리, 동적 페이지 생성 |
| 예시 | Apache, Nginx | Tomcat, JBoss, WebLogic |
| 처리 방식 | 클라이언트 요청 → 파일 반환 | 클라이언트 요청 → 로직 처리 → 결과 반환 |
| DB 연동 | 불가 | 가능 |
| 속도 | 빠름 | 상대적으로 느림 |
| 실행 환경 | OS 레벨 | JVM 등 어플리케이션 레벨 |

# 웹 애플리케이션 구조

## Monolith




## Rest API 기반 Micro Service



# REST (Representational State Transfer) 란?

웹을 거대한 공유 자원 저장소로 바라보고,
그 공유 자원을 전달하기 위한  다양한 표현 (XML, JSON, FILE etc)을 전달하는 방식

**Resource**

- 웹 상에 존재하는 이름을 붙일 수 있는 모든 종류의 정보나 객체 (**명사**: User, Domain 등)
- 사용자 조회에서의 **사용자**
- 주문 생성에서 **주문**

**REST 핵심 3대 요소**

- **리소스를 나타내는 주소 (URI)**: 무엇을 제어할 것인가?
- **행위를 나타내는 매소드 (HTTP Method)**: 어떻게 제어할 것인가 (GET, POST, PUT, DELETE etc)
- **표현 (Representation)**: 어떤 형태(JSON, XML etc)로 주고 받을 것인가?

로이 필딩(Roy Fielding) 이 2000년 박사 학위 논문에서 REST 개념을 처음 정립

# REST 핵심 구성요소: Resource

## 개념

- 서버가 제공하고 클라이언트가 조작할 수 있는 모든 대상을 자원으로 정의
- **REST에서 가장 핵심 개념**
- **데이터 그 자체가 아니라 데이터가 의미하는 개념이 자원**
- 자원은 **명사**로 정의

## 예시

```
User
Product
Article
Order
Review
Image
SensorData
```

# REST 핵심 구성요소: Resource Identifier (자원의 식별자)

### 개념

- 모든 자원은 고유하게 식별될 수 있어야 하며, REST에서는 이를 URI로 표시
- 자원(Resource)을 식별하는 수단 = URI

**URI**는 "어떤 자원이냐"를 표현하고

**HTTP Method**는 "그 자원을 무엇을 할 것이냐"를 표현한다.

### 예시

#### URI 명사형으로 리소스 지정

```
GET /users/10       # 10번 사용자 조회
PUT /users/10       # 사용자 정보 전체 수정
PATCH /users/10     # 부분 수정
DELETE /users/10    # 삭제
GET /users/10/orders  # 특정 사용자의 주문 목록 조회
```

#### 잘못된 예시

/createUser, /updateUser

동작(create, update)은 URI에 표현하는 것이 아니라 HTTP Method에 표현하기 때문.

# [참고] URI vs. URL

- **URI는 누구인가? 무엇인가, URL은 " 어디에 있는가?, 어떻게 가는가?"**
- **URI = 식별자 (개념 중심)**
- **URL = 위치 + 접근 (네트워크 중심)**

| 구분 | URI (Uniform Resource Identifier) | URL (Uniform Resource Locator) |
|---|---|---|
| 정의 | 리소스(자원)를 식별하는 모든 방법의 총칭 | 리소스의 위치(주소)를 명시하는 식별자 |
| 포함 관계 | URL은 URI의 하위 개념 | URI의 한 종류 |
| 형식 | scheme:[//authority]path[?query][#fragment] | scheme://host[:port]/path[?query][#fragment] |
| 예시 | mailto:someone@example.com<br>https://example.com/index.html | https://example.com/index.html<br>ftp://ftp.example.com/file.txt |
| 기능 | 리소스의 식별(이름, 위치, 둘 다 가능) | 리소스의 위치(주소) 지정, 접근 방법 명시 |
| 설명 | 이름(URN), 위치(URL) 모두 포함 | 리소스가 어디 있는지, 어떻게 접근하는지 명확히 나타냄 |

❖ **URN (Uniform Resource Name)**: 리소스의 "이름"에 초점을 맞춘 URI의 하위 개념

# REST 핵심 구성요소: 자원의 행위 (HTTP Method)

### 개념

자원을 조회, 생성, 수정, 삭제(CRUD)하는 행위는 URI가 아니라 HTTP 메서드를 통해 서버에 전달

| 행위 | HTTP Method | URI 예시 | 의미 |
|---|---|---|---|
| **생성 (Create)** | POST | /users | 새로운 사용자를 생성한다. |
| **조회 (Read)** | GET | /users/1 | 1번 사용자 정보를 가져온다. |
| **수정 (Update)** | PUT 또는 PATCH | /users/1 | 1번 사용자 정보를 전체 수정(PUT) 또는 일부 수정(PATCH)한다. |
| **삭제 (Delete)** | DELETE | /users/1 | 1번 사용자를 삭제한다. |

# REST 핵심 구성요소: Representation (자원의 표현)

### 개념

클라이언트는 자원 자체를 받는 것이 아니라 자원을 표현한 데이터 형태(Representation)를 받음

예를 들어

서버에 User라는 자원이 존재하지만 클라이언트는 User 객체 자체를 받는 것이 아니라 User를 JSON으로 표현한 것을 받음

### 대표적인 Representation 형태

- JSON (REST 기본)
- XML
- YAML
- HTML
- Text
- Image (jpg, png)
- Binary (file)
- PDF 등

# REST (Representational State Transfer) 설계 원칙

- REST는 로이 필딩(Roy Fielding)이 2000년 박사 논문에서 제안한 웹 아키텍처를 설계하기 위한 제약조건(Constraints) 기반의 아키텍처 스타일
- REST는 단순한 API 스타일이 아니라, 웹이 확장성과 단순성을 유지하도록 만드는 설계 원칙의 집합이다.

REST는 웹 아키텍처의 확장성과 단순성을 유지하기 위해
6가지 원칙을 만족할 때 비로소 RESTful한 시스템으로 정의

| # | REST 설계 원칙 | 필수/선택 |
|---|---|---|
| 1 | 클라이언트-서버 분리 | 필수 |
| 2 | 무상태성(Stateless) | 필수 |
| 3 | 캐시 가능(Cacheable) | 필수 |
| 4 | 일관된 인터페이스(Uniform Interface) | 필수 |
| 5 | 계층 구조(Layered System) | 필수 |
| 6 | 코드 온 디맨드(Code on Demand) | 선택 |

# REST 설계 원칙: 클라이언트-서버 분리

## 개념

- 클라이언트(UI)와 서버(비즈니스 로직, 데이터 저장) 역할 분리
- 클라이언트는 화면 구성과 사용자 경험에 집중하고, 서버는 데이터 저장/처리/비즈니스 규칙에 집중
- 서로 API라는 계약만 공유하며, 구현은 독립적

## 효과

- 프론트(Vue/React)와 백엔드(Spring Boot) 독립 개발 가능
- 서버 교체나 확장이 쉬움
- 유지보수성과 개발 속도 증가

## 예시

### 클라이언트 요청

```
GET /users/10
```

### 서버 JSON 응답

```
{ "id": 10, "name": "홍길동" }
```

- 클라이언트는 이 데이터로 화면을 표현할 뿐이며
- 서버가 DB를 MySQL로 바꾸든 PostgreSQL로 바꾸든 클라이언트는 전혀 상관 없음

# REST 설계 원칙: 무상태성

## 개념

- 서버는 클라이언트의 세션 상태를 보관하지 않음
- 요청 간 상태를 기억하지 않으므로, 각 요청은 완전한 독립성 보장
- 요청에는 인증/권한/요청 처리에 필요한 정보를 항상 포함 (Header, JWT,  HTTP-Only Cookie)

## 효과

- 서버 확장(Scale-out)이 매우 쉬워짐
  → 상태를 안 들고 있으니 어떤 서버가 받아도 처리 가능
- 시스템 단순화, 오류 감소

## 예시

### Stateful 방식 (상태 저장 세션 기반)

```
로그인 요청 → 서버가 세션 생성 → 이후 요청마다 서버 세션을 기준으로 판단
```

### REST 방식 (JWT 사용)

```
GET /profile
Host: api.example.com
Authorization: Bearer eyJhbGciOi...
```

서버는 “이 사용자가 이전 요청에서 로그인했는지” 기억하지 않고, JWT를 확인만 진행

# REST 설계 원칙: 캐시기능

## 개념

- 응답은 명확히 캐시 가능 여부를 나타내야 하며
- 캐싱을 통해 네트워크 비용을 줄이고 응답 속도를 개선
- HTTP 캐시 기능(Cache-Control, ETag 등)을 활용하는 것이 REST의 원칙

## 효과

- 빠른 성능
- 서버 부하 감소

## 예시

### Cache-Control 사용

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=600
```

이 응답은 600초(10분) 동안 재사용(캐시) 가능 표시 (브라우저, CND, Proxy 모두 이헤더를 보고 캐싱 동작 수행)

600초 후 캐시 만료 인식 후 서버에 재 요청 발송

서버는 리소스의 캐시 가능 여부, 기간, 조건 등을 클라이언트에 지시

# REST 설계 원칙: 일관된 인터페이스

## 개념

- 모든 API가 동일한 규칙을 따라야 한다.

### 1) 자원의 식별 (URI 기반)

- 자원은 URI로 표현해야 한다.
- URI는 동사가 아니라 명사 기반이어야 한다.

```
#올바른 예
GET /users/10
GET /orders/2025/items

# 잘못된 예
/getUserInfo
/doOrderCreate
```

### 2) 표준 메서드 사용

- HTTP의 표준 메서드 사용
  (GET, POST, PUT, DELETE)

```
GET /products           # 모든 상품 조회
GET /products/1         # 특정 상품 조회
POST /products          # 새로운 상품 생성
PUT /products/1         # 상품 정보 전체 수정
PATCH /products/1       # 상품 정보 일부 수정
DELETE /products/1      # 상품 삭제
```

### 3) 표준 문서 형태로 데이터 공유

- 서버는 자원을 표준 문서 형태로 전달한다.
- (JSON, XML, YAML 등)

```
{
  "id": 10,
  "name": "홍길동"
}
```

### 4) 자기서술적 메시지(Self-descriptive message)

- 요청/응답 메시지는 스스로 해석 가능해야 한다.

**요청**

```
POST /users
Content-Type: application/json
```

**응답**

```
HTTP/1.1 201 Created
Location: /users/10
Content-Type: application/json
```

→ 메시지에 모든 의미가 담겨 있어 문서 없이도 해석 가능해야 함

# REST 설계 원칙: 계층 구조

### 개념

- 클라이언트는 요청이 어느 서버 노드로 가는지 알 필요 없음
- 로드밸런서, 게이트웨이, 인증 서버, 프록시 등이 중간 계층으로 존재할 수 있음
- 서버는 여러 계층으로 나뉠 수 있음

### 효과

- 보안 강화 (예: 인증 서버 분리)
- 로드밸런싱, 캐싱, API 게이트웨이 등 추가 기능 쉽게 도입
- 구조 확장성 증가

### 예시

**요청 흐름 예:**

```
Client → API Gateway → Auth Server → Backend → Database
```

**클라이언트는 "백엔드 서버가 여러 개인지?"를 알 필요가 없다.**

# REST 설계 원칙: Code-on-Demand

## 개념

- 서버가 클라이언트에게 실행 가능한 코드를 내려보내고,
- 클라이언트는 이를 동적으로 실행하여 기능을 확장하는 방식

## 효과

- 서버는 새로운 기능을 코드 형태로 내려 보내어 클라이언트가 즉시 새로운 기능을 수행하도록 만들 수 있음
- 클라이언트를 업데이트 없이도 서버가 클라이언트의 행동을 바꿀 수 있음

## 예시

**브라우저에 새로운 JavaScript 파일을 내려보내면:**

- UI 동작이 바뀜
- 새로운 Form 검증이 추가됨
- 특정 페이지에서만 실행되는 맞춤 로직 수행

# 일관된 인터페이스: REST API 설계 규칙

| 규칙 | 설명 및 예시 |
|---|---|
| 자원(Resource) 명사 사용 | URI는 동작이 아닌 자원(명사)로 작성<br>*예) /users, /products/123/comments* |
| 동사 대신 HTTP 메서드 사용 | URI에 동사를 쓰지 않고, GET/POST/PUT/DELETE 등 HTTP 메서드로 동작을 표현<br>*예) GET /users* |
| 계층적 구조 | 자원 간 포함 관계는 URI로 계층 구조 표현<br>*예) /users/123/orders/5* |
| 단수/복수 일관성 | 보통 전체 집합은 복수형으로 표현 (단수형 가능) → 일관성 유지<br>*예) /users, /products* |
| 쿼리 파라미터 | 필터, 정렬, 검색 등은 쿼리 파라미터 사용<br>*예) /users?age=20&sort=name* |
| 상태코드 활용 | 처리 결과에 따라 적절한 HTTP 상태코드 반환 (200 OK에 일관성 있는 메시지 형식도 가능)<br>*예) 200(성공), 201(생성됨), 404(없음), 400(잘못된 요청)* |
| 응답 포맷 | 보통 JSON 형식 사용<br>*예) { "id": 1, "name": "Alice" }* |
| Stateless | 각 요청은 독립적으로 처리, 서버는 클라이언트의 상태 저장 X |

# REST API 예시

- **사용자 관리**

| 기능 | HTTP 메서드 | URI | 설명 |
|---|---|---|---|
| 목록 조회 | GET | /users | 모든 사용자 목록 |
| 생성 | POST | /users | 새 사용자 생성 |
| 단일 조회 | GET | /users/3 | id=3인 사용자 조회 |
| 전체 수정 | PUT | /users/3 | id=3인 사용자 정보 전체 수정 |
| 일부 수정 | PATCH | /users/3 | id=3인 사용자 정보 일부 수정 |
| 삭제 | DELETE | /users/3 | id=3인 사용자 삭제 |

- **게시판 관리**

| 기능 | HTTP 메서드 | URI | 설명 |
|---|---|---|---|
| 게시글 목록 | GET | /posts | 게시글 전체 목록 |
| 게시글 생성 | POST | /posts | 게시글 등록 |
| 게시글 조회 | GET | /posts/10 | id=10 게시글 조회 |
| 댓글 목록 | GET | /posts/10/comments | 게시글 10번의 댓글들 |



# Spring과 Spring Boot

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# Spring 개요

- Java 기반의 오픈소스 애플리케이션 프레임워크
- 엔터프라이즈급 애플리케이션 개발을 위한 표준 프레임워크
- 복잡한 자바 개발을 단순화하는 컨테이너 역할

| 특징 | 설명 |
|---|---|
| 경량 프레임워크 | 필요 모듈만 사용, 메모리 부담 적음 |
| 제어의 역전(IoC) | 객체 생성/관리를 개발자가 아닌 프레임워크가 담당 |
| 의존성 주입(DI) | 객체 간 결합도 최소화, 유지보수성 향상 |
| 관점 지향 프로그래밍(AOP) | 로깅, 보안, 트랜잭션 등 공통 기능을 분리, 코드 중복 감소 |
| 모듈화 | Web, Data, Security 등 다양한 기능을 모듈화해서 관리 가능 |
| 확장성 | 오픈소스 커뮤니티 기반 다양한 확장 프로젝트와 연동 |
| 테스트 용이성 | DI를 통한 Mock 객체 활용 등 유닛 테스트 용이 |

# Spring의 역사

- Spring: '복잡한 Java EE의 대안' → 유연하고 강력한 자바 엔터프라이즈 개발 프레임워크
- Spring Boot: '쉽게, 빠르게 Spring을!' → 설정 최소화, 스타트업/대기업 모두 사랑하는 표준 플랫폼으로 발전

| 구분 | Spring | Spring Boot |
|---|---|---|
| 등장 | 2003년(1.0) | 2014년(1.0) |
| 목적 | 유연하고 강력한 엔터프라이즈 개발 | 빠르고 쉽게 Spring 개발, 복잡한 설정 최소화 |
| 주요 특징 | IoC/DI, AOP, 다양한 모듈 | 자동설정, 스타터, 내장 서버, 쉬운 실행 |

# [참고] 전자정부 프레임워크

- 대한민국 행정안전부가 공공기관 정보시스템의 개발 효율화와 품질 제고 및 유지보수의 표준화·효율화를 위해 개발한 국가 표준 **오픈소스(스프링) 기반** 소프트웨어 개발 프레임워크

| 구분 | 주요 내용 |
|---|---|
| 기반 프레임워크 | **Spring** Framework, MyBatis, Maven/Gradle, log4j/slf4j 등 |
| 공통 컴포넌트 | 사용자, 권한, 메뉴, 파일, 암호화, 메시지, 코드 관리, 로깅 등 |
| 표준 템플릿 | 표준 프로젝트 구조, 기본 JSP/Controller/Service/DAO 샘플 제공 |
| 개발 도구 | 이클립스 기반 개발 환경(eGovFrame Dev Tool), 프로젝트 생성 지원 |
| 문서/가이드 | 개발 가이드, API 문서, 표준 코드 샘플, 품질 체크리스트 등 |

사이트 : https://www.egovframe.go.kr/home/main.do

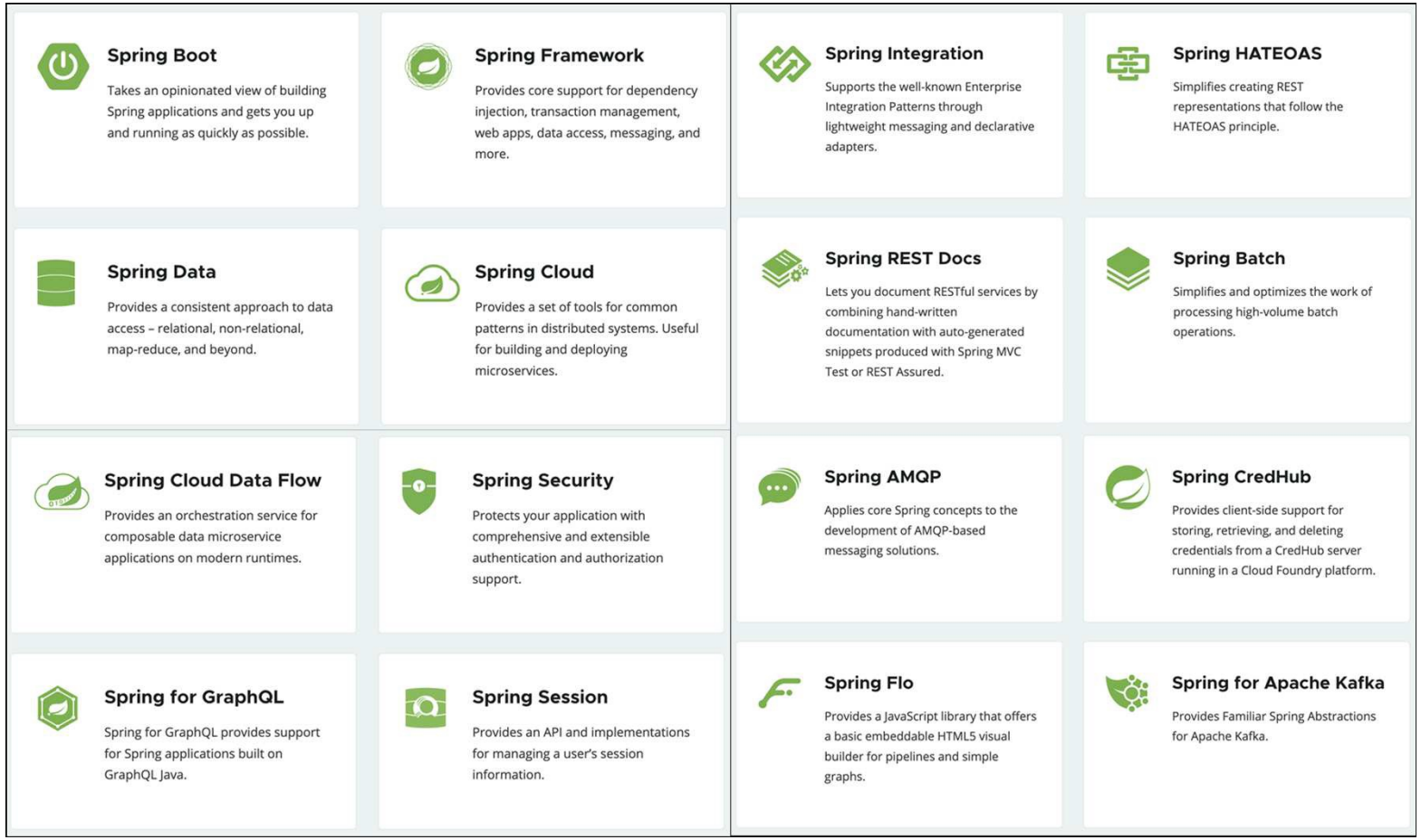
# Spring 생태계

| 프로젝트 | 주요 역할/설명 |
|---|---|
| Spring Boot | 복잡한 환경 설정을 자동화해주는 스타터(시작 도우미) |
| Spring Data JPA | DB 연동을 쉽고 표준화하여 개발 생산성/유지보수성 향상 |
| Spring Security | 인증/인가 등 보안 기능을 손쉽게 구현 |
| Spring Batch | 대용량 데이터 일괄 처리 |
| Spring Cloud | 마이크로서비스, 클라우드 기반 개발 지원 |
| Spring WebFlux | 비동기/논블로킹 방식 웹 애플리케이션 개발 |

❖ **실제 현업에서는 Spring Boot, Spring Data JPA 및 Spring Security 조합이 많이 쓰임**

# [참고] 스프링 프로젝트 사이트

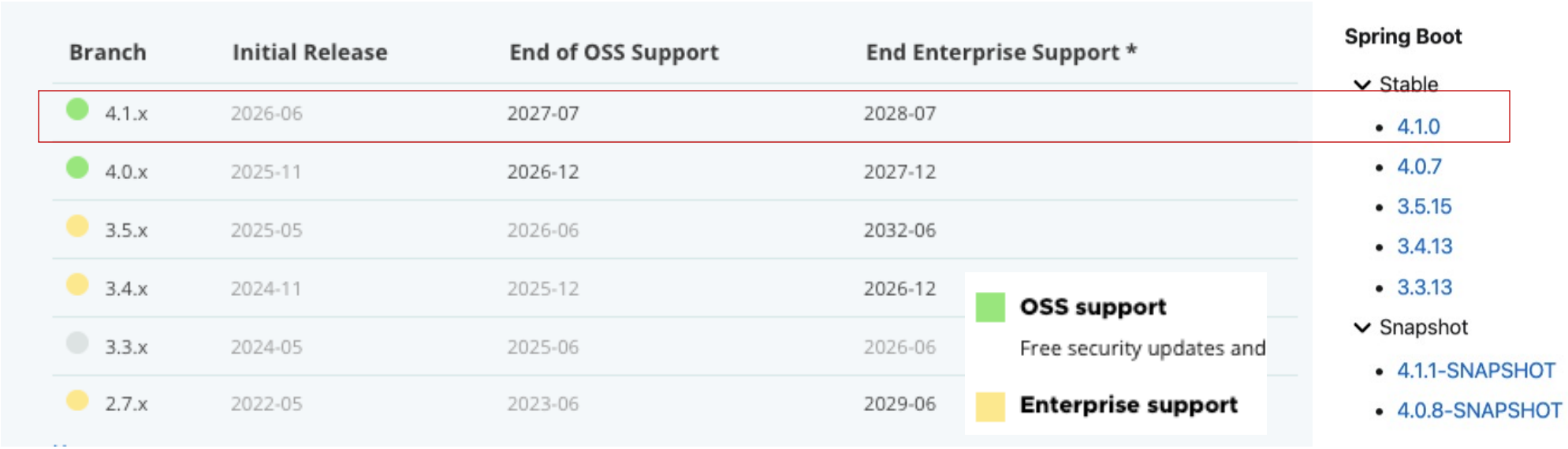
https://spring.io/projects



# Spring Boot의 등장 배경

- 전통적인 Spring 설정은 복잡한 XML/설정 파일, 환경 구축에 오랜 시간 소요
- 신규 프로젝트를 빠르게 만들고 배포하려면, 반복적/필수적인 설정을 자동화할 필요
- “Spring을 더 쉽고 빠르게!” → Spring Boot 등장

| | |
|---|---|
| **2014** | **Spring Boot 1.0 발표**- 자동설정(Auto Configuration), Starter 기반 의존성, 내장 서버(톰캣), JAR 실행 등 |
| **2016** | **Spring Boot 1.4~1.5**- 테스트 지원 강화, 보안, Actuator 등 부가 기능 확장 |
| **2018** | **Spring Boot 2.0**- Spring 5 기반, 리액티브 프로그래밍 정식 지원, 더 개선된 개발 경험 제공 |
| **2021~** | **Spring Boot 2.5~3.x**- Java 17, Jakarta EE, GraalVM 네이티브 이미지 등 최신 기술 적용 |



# Spring Boot의 주요 특징

| 특징 | 설명 |
| --- | --- |
| 자동 설정(Auto-configuration) | 환경/라이브러리 감지 후 적절한 설정 자동 적용 |
| 내장 서버 지원 | Tomcat, netty 등 별도 설치 없이 JAR로 바로 실행 가능 |
| Starter에 의한 의존성 관리 | 프로젝트에 맞는 기본 라이브러리 세트를 starter로 한 번에 추가 |
| 복잡한 XML 미사용 | 어노테이션 기반의 간단한 설정, 프로퍼티(YAML/Properties) 기반 |
| 운영/모니터링 용이 | 헬스체크, 메트릭스 등 운영 관리 기능 기본 탑재 |
| 쉬운 배포 | 빌드 후 JAR 실행, Docker 등 다양한 배포 환경과 연동 |

# [참고] 전통적인 Spring vs Spring Boot

| 비교 항목 | 전통적 Spring | Spring Boot |
|---|---|---|
| 서버 동작 방식 | 상시 기동중인 외부 WAS(WebSphere, WebLogic, Jeus) 에 WAR로 배포 | 별도 서버 없이 내장 서버(Tomcat, Jetty, Undertow, Netty) 포함 JAR로 실행 |
| 아키텍처 스타일 | MVC (Model View Controller) | Rest API Model 스타일 위주 |
| 패키징 | WAR 형식이 일반적 | JAR 기본 |
| 설정 방식 | XML 또는 Java Config 중심, 수동 설정 많음 | 자동 설정(Auto Configuration), Starter 의존성 제공 |
| 배포 방식 | WAS에 수동 배포<br>cp target/myapp.war $TOMCAT_HOME/webapps/ | java -jar로 바로 실행 가능 |
| 실행 환경 | WAS 설치 필요 | 내장 Embedded Engine 기반 바로 실행 |
| 서버 교체 | WAS 설정 변경 필요 | 의존성 변경만으로 교체 가능 |
| 시작 속도 | 상대적으로 느림 | 빠름 |
| 사용 편의성 | 초기 설정 복잡 | 간단, 빠른 개발 가능 |

Spring과 Spring Boot 모두 MVC 웹 애플리케이션과 REST API를 만들 수 있지만,
전통적으로 Spring은 외부 WAS + JSP 기반 MVC에 자주 쓰였고,
Spring Boot는 내장 서버를 활용한 REST API 서버 개발 위주로 사용



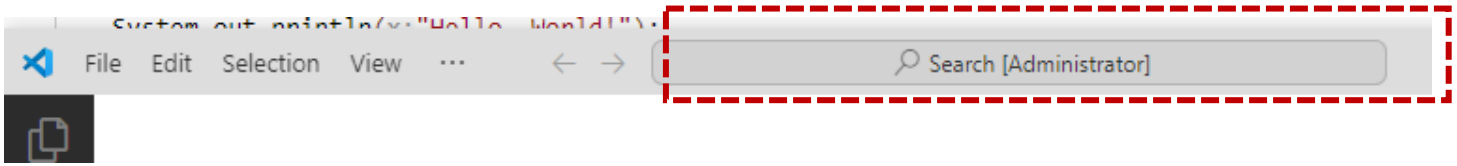
# spring boot 프로젝트 만들기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

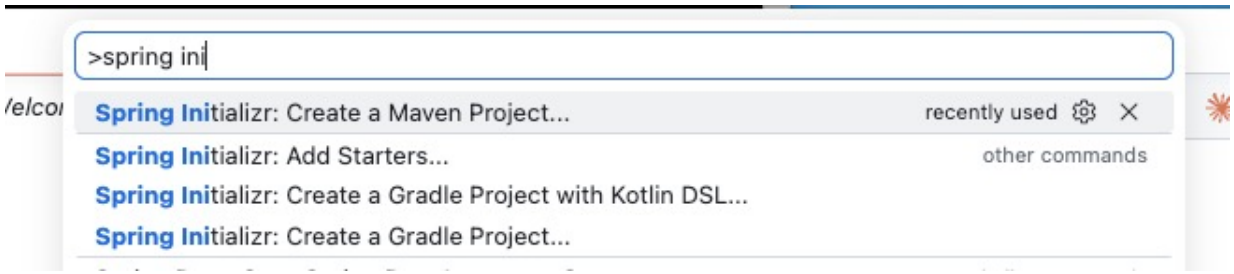
# Springboot 프로젝트 생성하기 (1/4)

- Search 선택

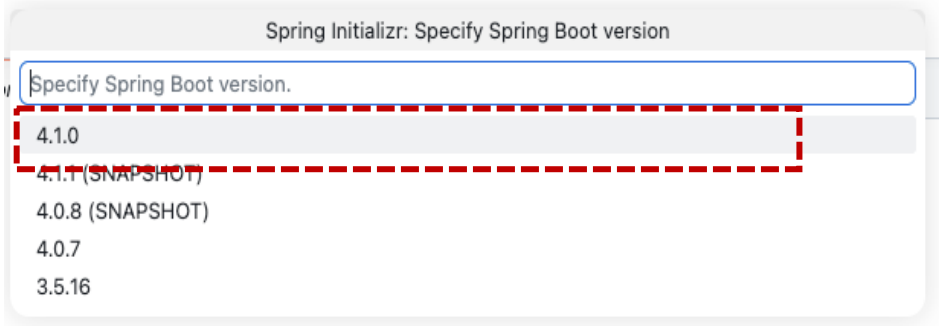



Command Palette 선택 단축키 : CMD + Shift + P

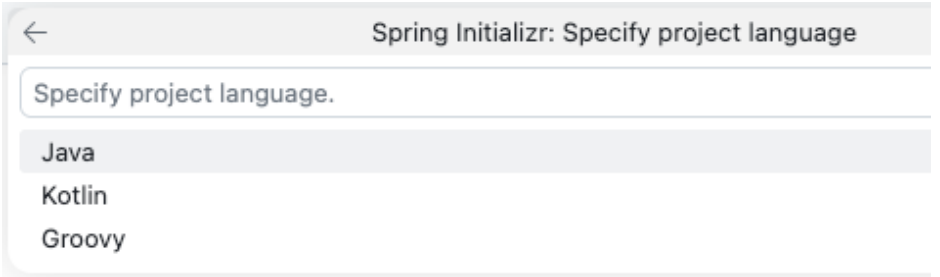
- Search에 “>spring Initial” 입력
- spring Initializr: Create a Maven Project 선택




- 4.1.0 선택

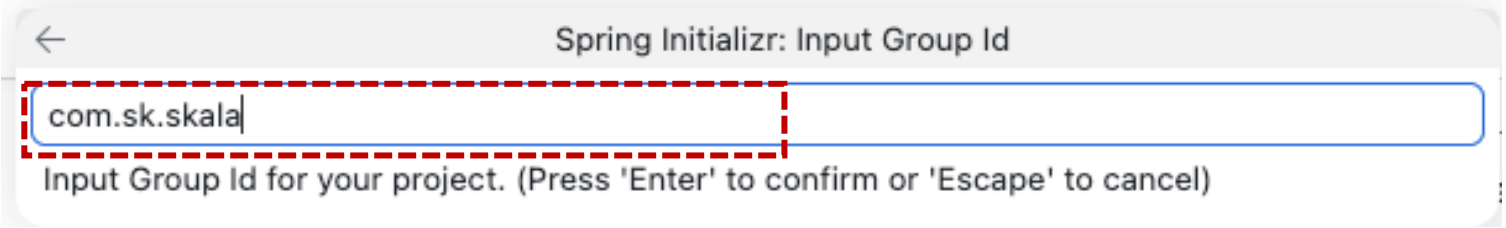



- Java 선택

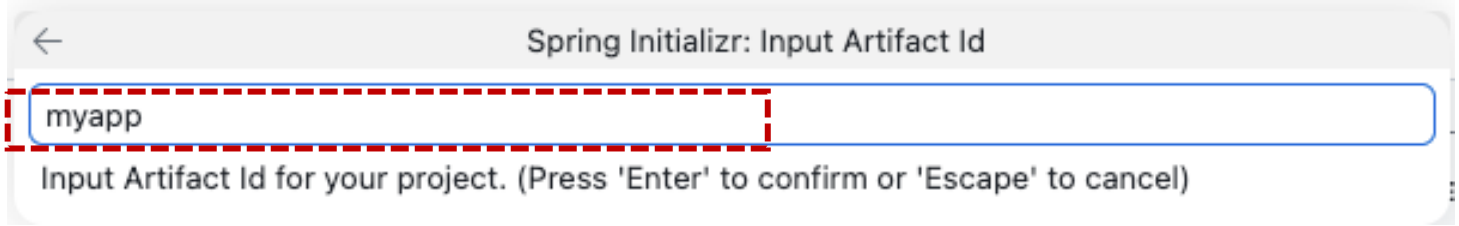


# Springboot 프로젝트 생성하기 (2/4)

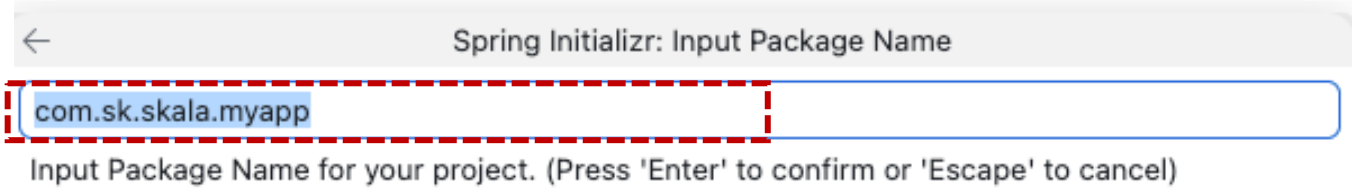
- 프로젝트 그룹 ID 입력: **com.sk.skala**



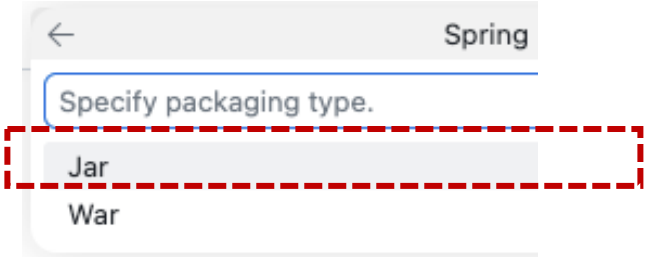

- Artifact Id 입력: myapp

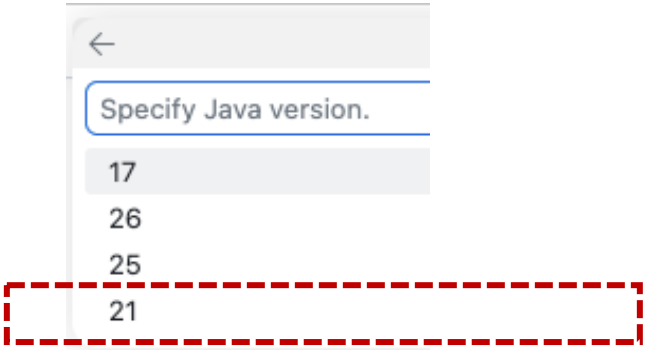



- **Full Package Name은 그대로 : 그냥 Enter**



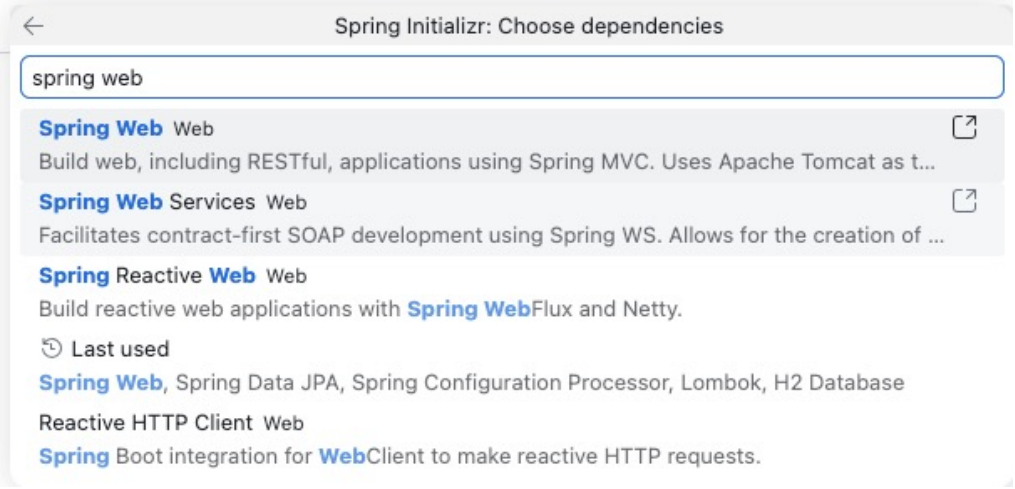

- 프로젝트 Artifact ID 입력 후 패키징 타입은 “Jar” > 버전 “21” 선택



# Springboot 프로젝트 생성하기 (3/4)

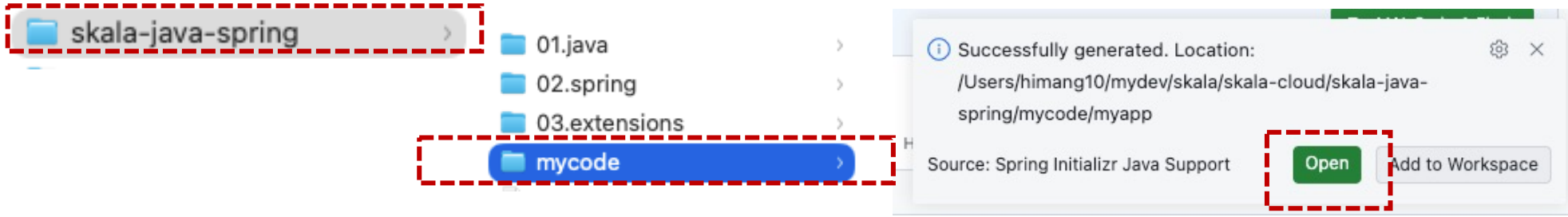
- **myapp 프로젝트에서 사용하기 위한 의존관계 라이브러리 선택**




반복것으로 검새 후 엔터

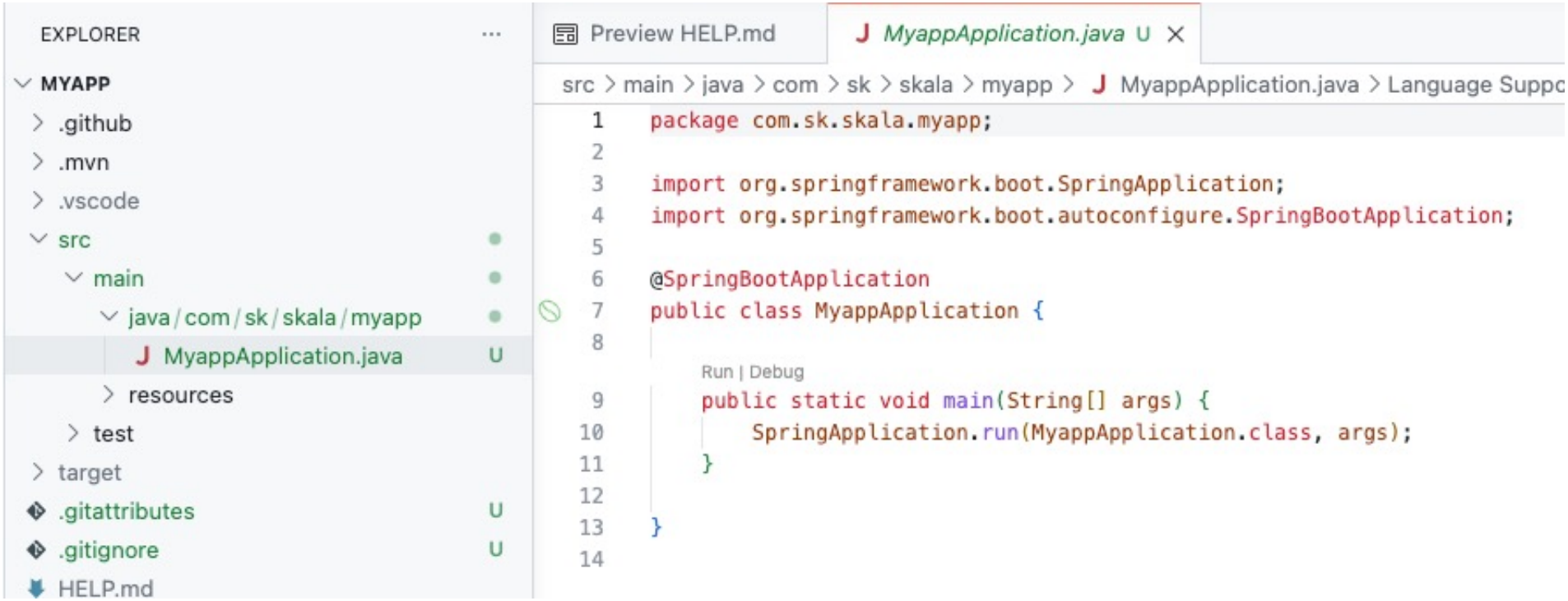
- 프로젝트 기본 dependencies 선택
  - Spring Web
  - Spring Data JPA
  - Spring Configuration Processor
  - Lombok
  - H2 Database

- 나의 코드를 구성하기 위한 디렉토리 선택 (skala-java-spring/mycode )



# Springboot 프로젝트 생성하기 (4/4)

- @SpringBootApplication 확인

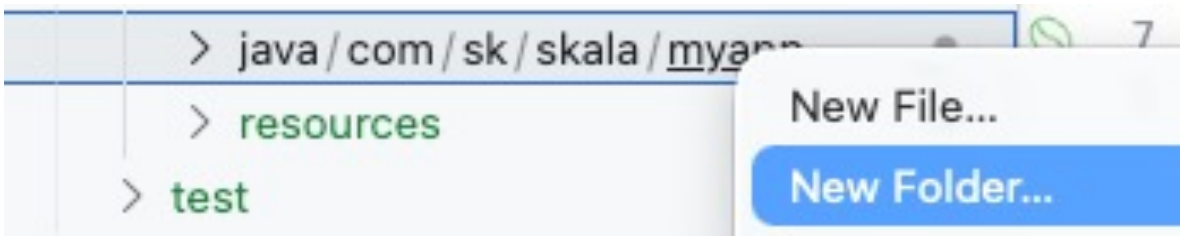



- **@SpringBootApplication** : 여러개의 스프링 부트 설정 어노테이션이 합쳐진 메타 어노테이션으로 **컴포넌트 스캔**을 진행
- **SpringApplication.run(MyappApplication.class, args)** : 스프링 부트 애플리케이션을 실행하는 명령어

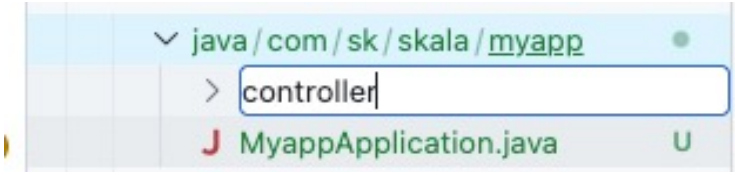
# [실습] Hello SKALA (1/4)

Browser 로 http://localhost:8080/api/hello를 요청을 했을때 Json 응답이 돌아오는 API 서버를 구성해보자

1. myapp package를 오른쪽 마우크 클릭 후 "New Folder" 선택




2. controller package (디렉토리)를 생성



# [실습] Hello SKALA (2/4)

- 생성된 controller package 아래에 "New File" 선택해서 HelloController.java 파일 생성
- HelloController.java에 아래 코드를 복사

**file Name : HelloController.java**

http://localhost:8080/**api/hello**

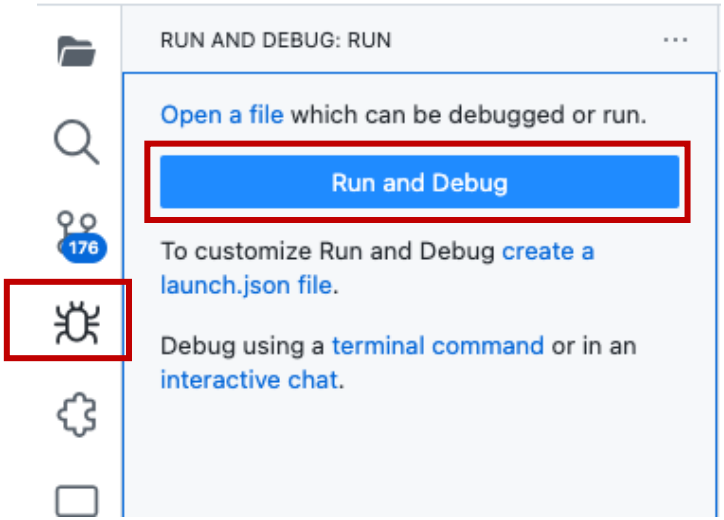
```
package com.sk.skala.myapp.controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("api")
public class HelloController {

    @GetMapping("/hello")
    public Map<String, String> helloWorld() {
        Map<String, String> response = new HashMap<>();
        response.put("message", "SKALA에 오신 것을 환영합니다");
        return response;
    }
}
```

# [실습] Hello World (3/4)

## 1. Run and Debug 실행

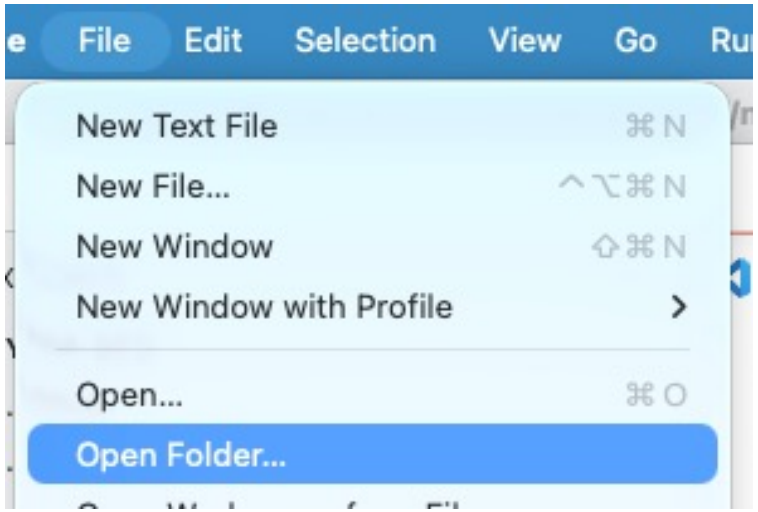





# [참고] Run and Debug 동작하지 않는 경우

pom.xml 또는 gradle.build 파일이 현재 VSCODE 프로젝트의 루트디렉토리인지 확인하고
그렇지 않는 경우  VSCODE를 새롭게 오픈해주세요

### 첫번째 방법: 메뉴로 직접 오픈하기



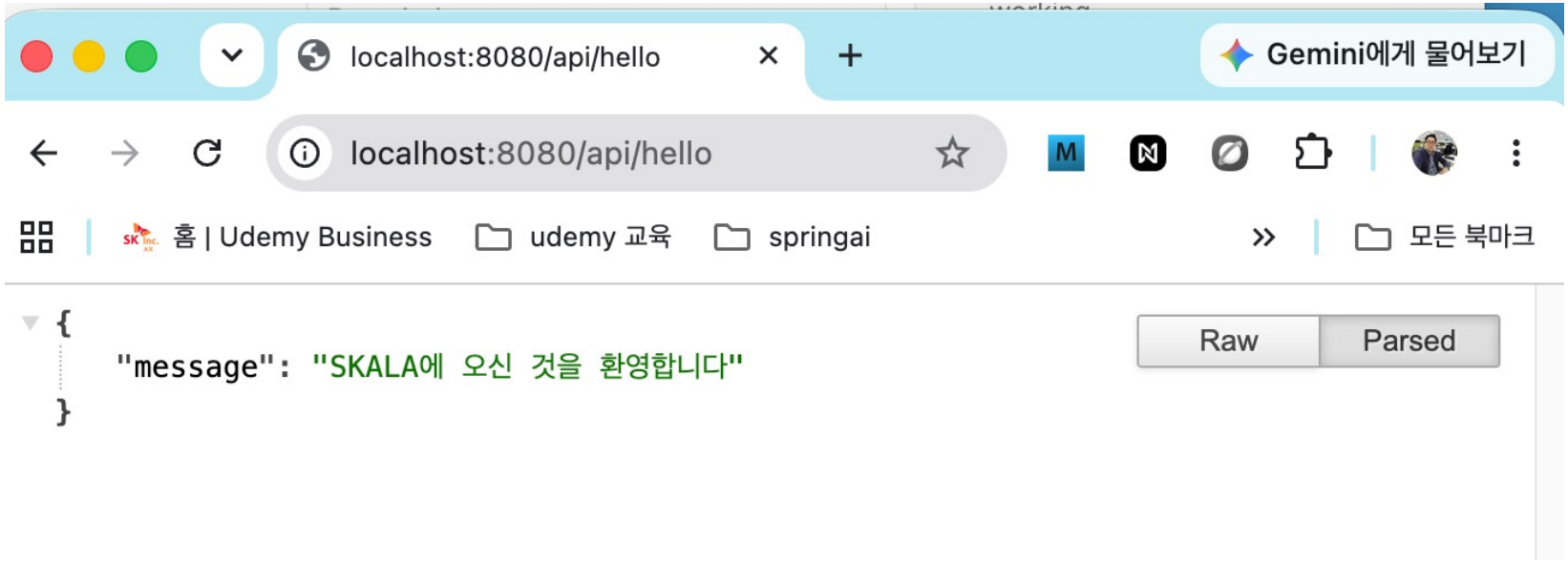

### 두번째 방법: 터미널에서 code  실행

```
> cd 01.training-code/01.start/myapp
> ls
pom.xml src target
> code .
```

# [실습] Hello World 접속하기 (4/4)

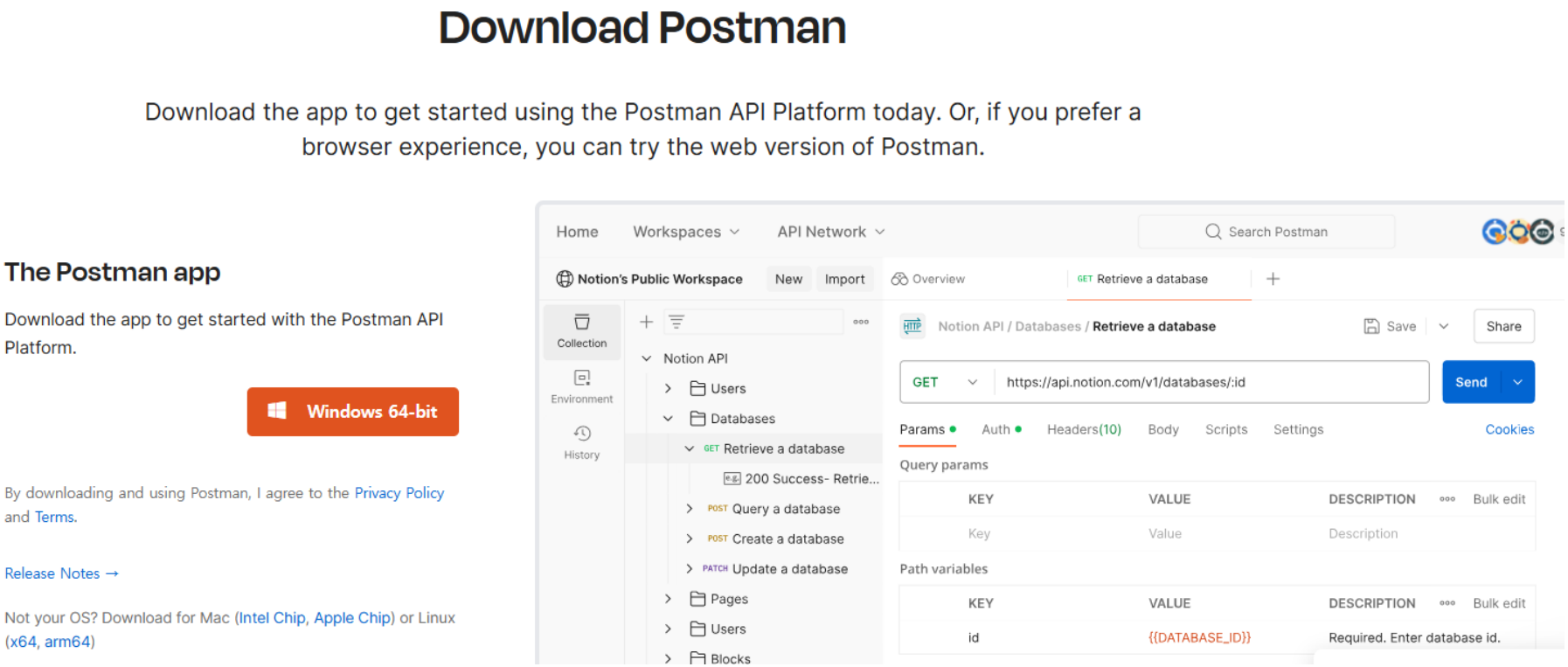
Browser 로 접속해보자

http://localhost:8080/api/hello



# Postman 설치

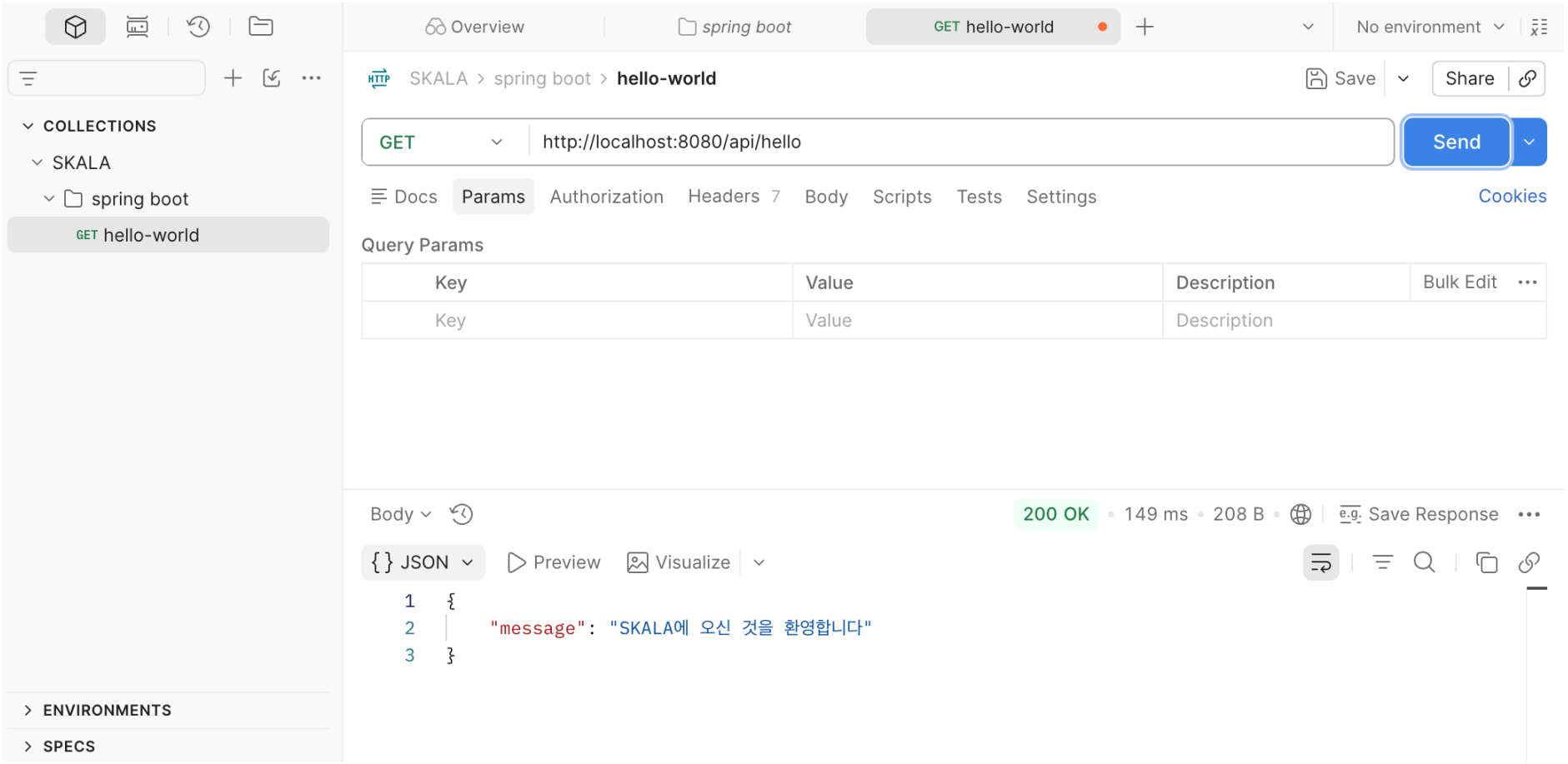
- 사이트: https://www.postman.com/downloads/
- OS 플랫폼에 맞게 설치



# [실습] Podman으로 hello world 접속하기

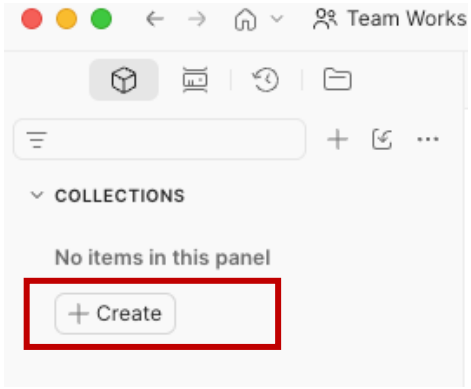
00.snippet/HelloController.java 복사

## Postman으로 접속해보기

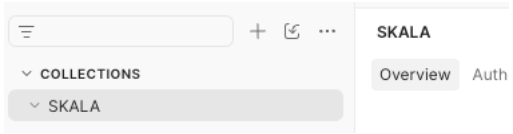


# [참고] podman 사용법

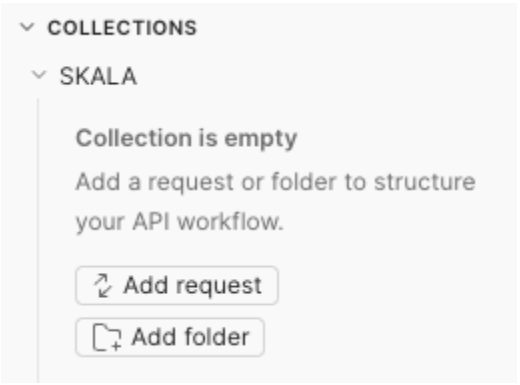
1. Collections 생성

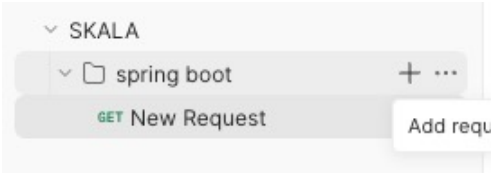



2. SKALA로 Collections 이름 설정




3. Add request 생성 (hello-world)




4. GET http://localhost:8080/api/hello 등록



# [참고] 포트 충돌 발생 시

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Web server failed to start. Port 8080 was already in use.

Action:

Identify and stop the process that's listening on port 8080 or configure this application to listen on another port.

(*|skala-2025:skala-practice)
~/mydev/skala/ywyi-skala-edu-code/backend/02.spring/ [master*] lsof -i:8080
COMMAND   PID     USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
java    10544 himang10   87u  IPv6 0x3d36b9dcdd08e795      0t0  TCP *:http-alt (LISTEN)
(*|skala-2025:skala-practice)
~/mydev/skala/ywyi-skala-edu-code/backend/02.spring/ [master*] kill -9 10544
(*|skala-2025:skala-practice)
~/mydev/skala/ywyi-skala-edu-code/backend/02.spring/ [master*]
```

port 충돌로 인한 실행 실패

```
> lsof -ti :8080 | xargs kill -9
```



# 컴포넌트 스캔

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# 컴포넌트 스캔(Component Scan)

- 컴포넌트 스캔은 스프링이 지정된 패키지(package) 이하를 탐색하여 특정 어노테이션이 붙은 클래스들을 찾아내고, 스프링 빈(Bean)으로 등록하는 자동 설정 기능
- 컴포넌트 스캔의 기본 범위는 @SpringBootApplication이 선언된 클래스의 하위 패키지 전체

**@SpringBootApplication 어노테이션의 조합 요소**

| 구분 | 역할 / 기능 |
|---|---|
| @Configuration | 스프링 컨테이너에게 자바 코드로 스프링 Bean 객체들을 생성하고 조립하게 설정 |
| @EnableAutoConfiguration | 외부 패키지에서 필요한 클래스를 찾아서 Bean으로 등록 |
| @ComponentScan | 개발자가 개발한 Bean을 자동 탐색해서 Bean으로 등록.<br>해당 패키지 및 하위 패키지의 @Component, @Service, @Repository 등의 Bean 자동 탐색 |

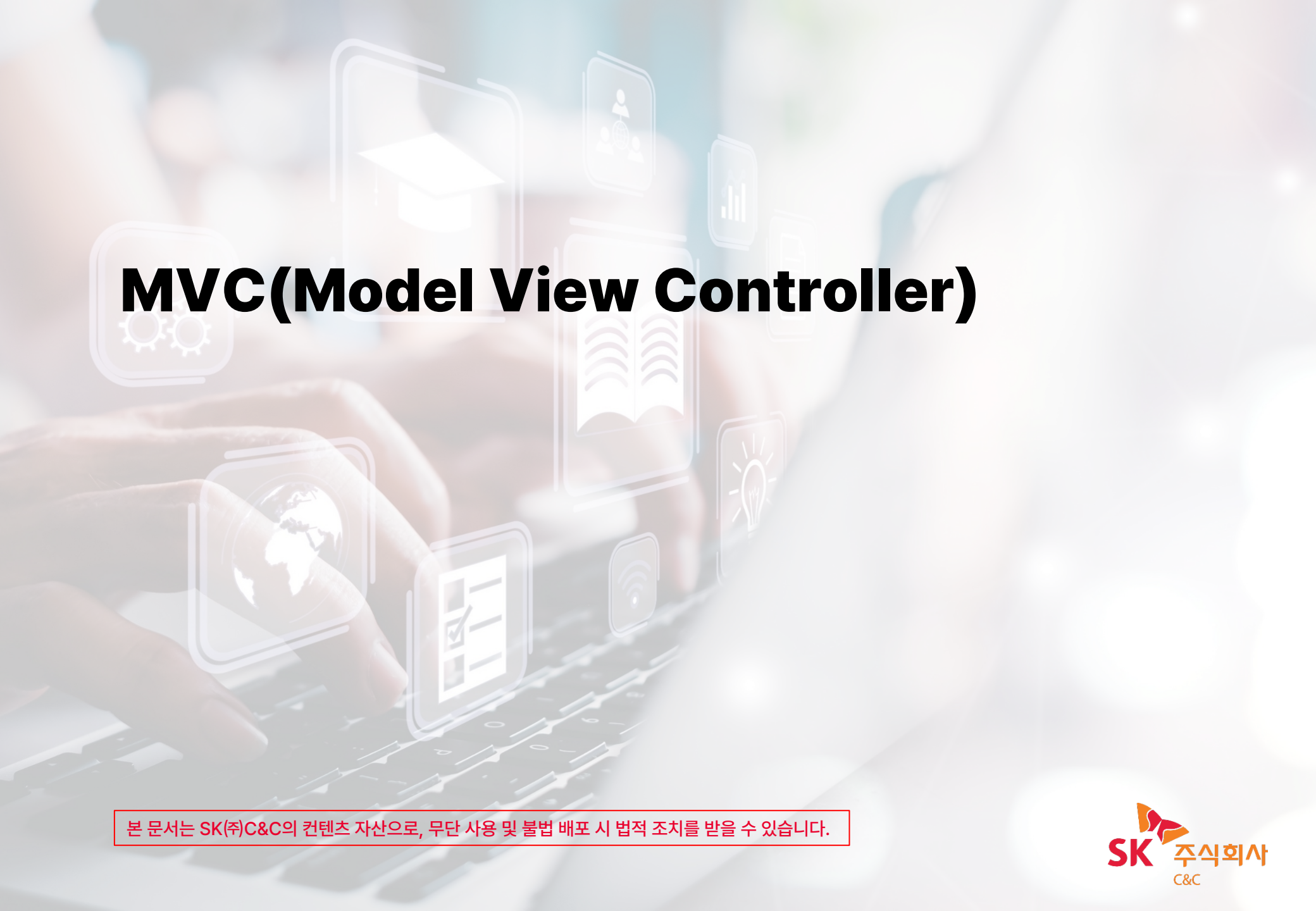
❖ **스캔 범위 제한 :** scanBasePackages 사용 (@ComponentScan에서는 basePackages)

```
@SpringBootApplication(scanBasePackages = {"com.skala.stock", "com.sk.common"})
public class StockApiApplication { ... }
```

# 컴포넌트 스캔 대상

- 스프링 부트의 **스테레오타입(Stereotype) 어노테이션**이 붙은 클래스들

| 어노테이션 | 설명 | 용도/예시 |
|---|---|---|
| @Component | 범용 컴포넌트 지정, 스프링이 빈으로 자동 등록 | 공통 모듈, 유틸리티 클래스 등 |
| @Controller | MVC 패턴의 컨트롤러로 지정, 웹 요청 처리 | REST API, 웹 페이지 처리용 컨트롤러 클래스 |
| @RestController | @Controller + @ResponseBody, REST API 개발에 특화 | RESTful 웹 서비스 컨트롤러 |
| @Service | 비즈니스 로직 담당 클래스 지정 | 서비스 계층 클래스(비즈니스 로직) |
| @Repository | 데이터 액세스 계층(DAO) 지정, 예외 변환 기능 제공 | JPA, MyBatis 등 DB 연동 클래스 |
| @Configuration | 자바 기반 설정 클래스 지정, @Bean 메서드 사용 가능 | 스프링 설정 및 빈 등록용 클래스 |
| @ConfigurationProperties | 외부 설정값(프로퍼티 등)을 객체에 바인딩 | YAML, properties 파일을 자바 객체에 매핑 |
| @EnableAutoConfiguration | 스프링 부트 자동 설정 활성화<br>(주로 @SpringBootApplication에 포함) | 스프링부트의 자동 설정 활성화 |



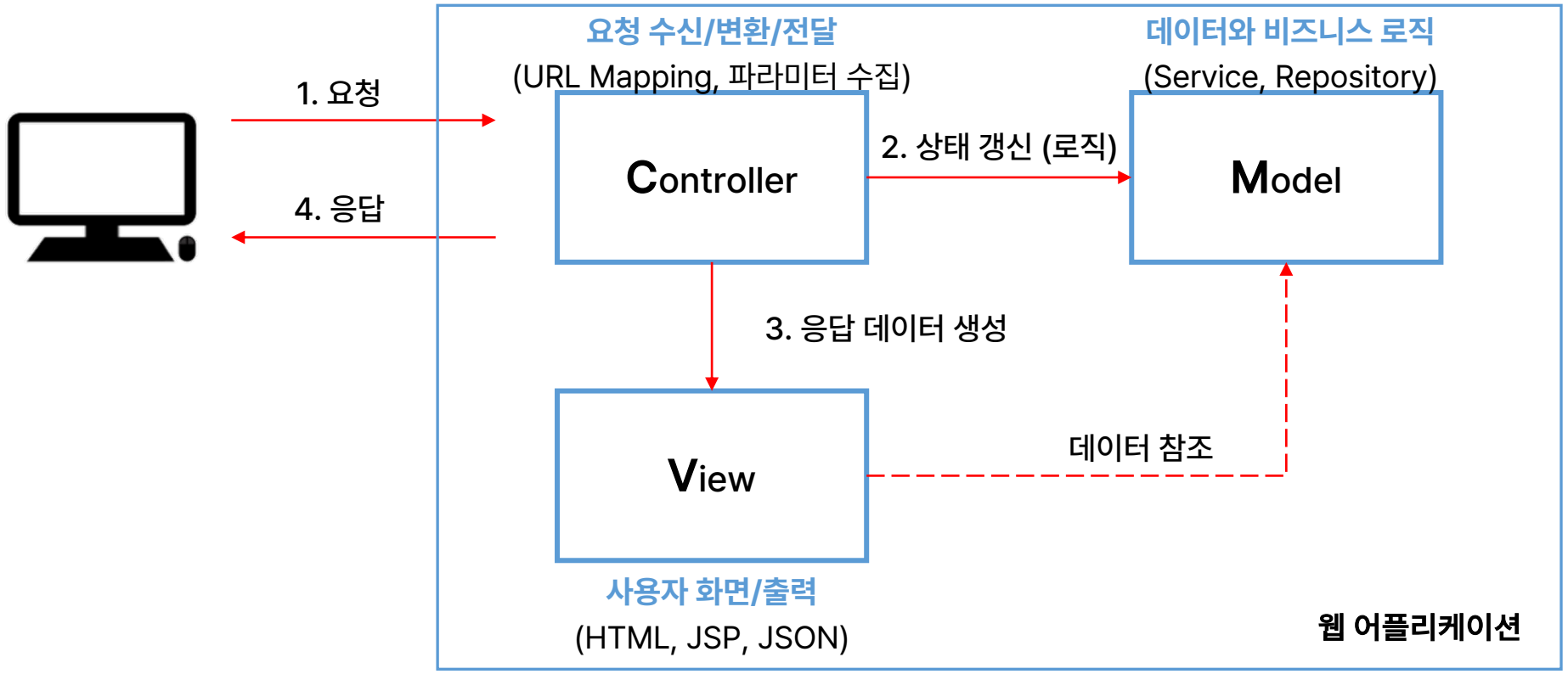
# MVC(Model View Controller)

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

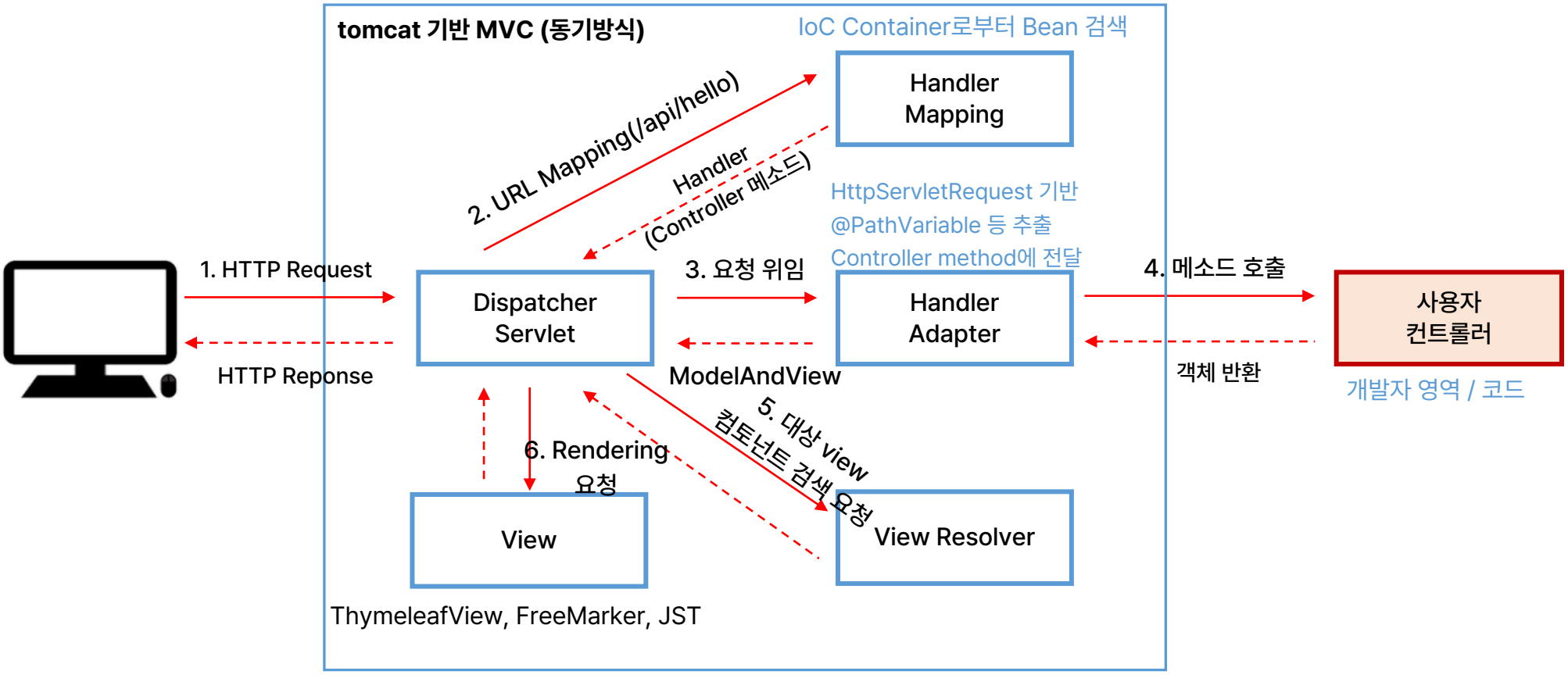
# MVC 모델

- **MVC 패턴: 1970 년대 전통적인 UI 아키텍처 패턴이며, 역할 분리 원칙을 따르는 구조**
- Spring은 이 일반적인 MVC 개념을 **웹 애플리케이션 개발에 맞게 프레임워크로 구현**



# Spring MVC

- HTTP Request를 받고 Controller를 찾아 사용자 업무 처리
- 처리된 결과를 VIEW 와 Model을 리턴하는 구조




참고) ModelAndView: Molde(데이터) + View (화면 정보) 으로  필요속성 정보와 Rendering을 위한 HTML, PDF 등을 그리기 위한 객체

# [참고] DispatcherServlet

## DispatcherServlet 의 역할

- Spring MVC의 **DispatcherServlet** 은 일종의 **Front Controller**
- 모든 요청은 서블릿 컨테이너(Tomcat)에서 **DispatcherServlet** 으로 전달
- DispatcherServlet 은 단순히 데이터를 건네받는 게 아니라, Spring IoC 컨테이너와 협력하여 **어떤 Controller가 이 요청을 처리할지**를 찾아내는 작업 진행

## 흐름 요약:

1. 톰캣이 요청 수신 → HttpServletRequest/HttpServletResponse 생성
2. DispatcherServlet 이 이 요청을 받음
3. **HandlerMapping** 을 이용해 어떤 Controller 메서드가 처리할지 검색
4. **HandlerAdapter** 가 해당 Controller 메서드를 실행할 수 있도록 호출
5. Controller → 비즈니스 로직(Service, Repository 등 IoC Container Bean) 호출
6. 결과(Model)를 ViewResolver에 전달 → View 렌더링
7. 응답을 HttpServletResponse 로 작성해서 톰캣이 클라이언트로 전송

# [참고] View Type

- View 컴포넌트는 HTML을 포함한 다양한 View를 생성

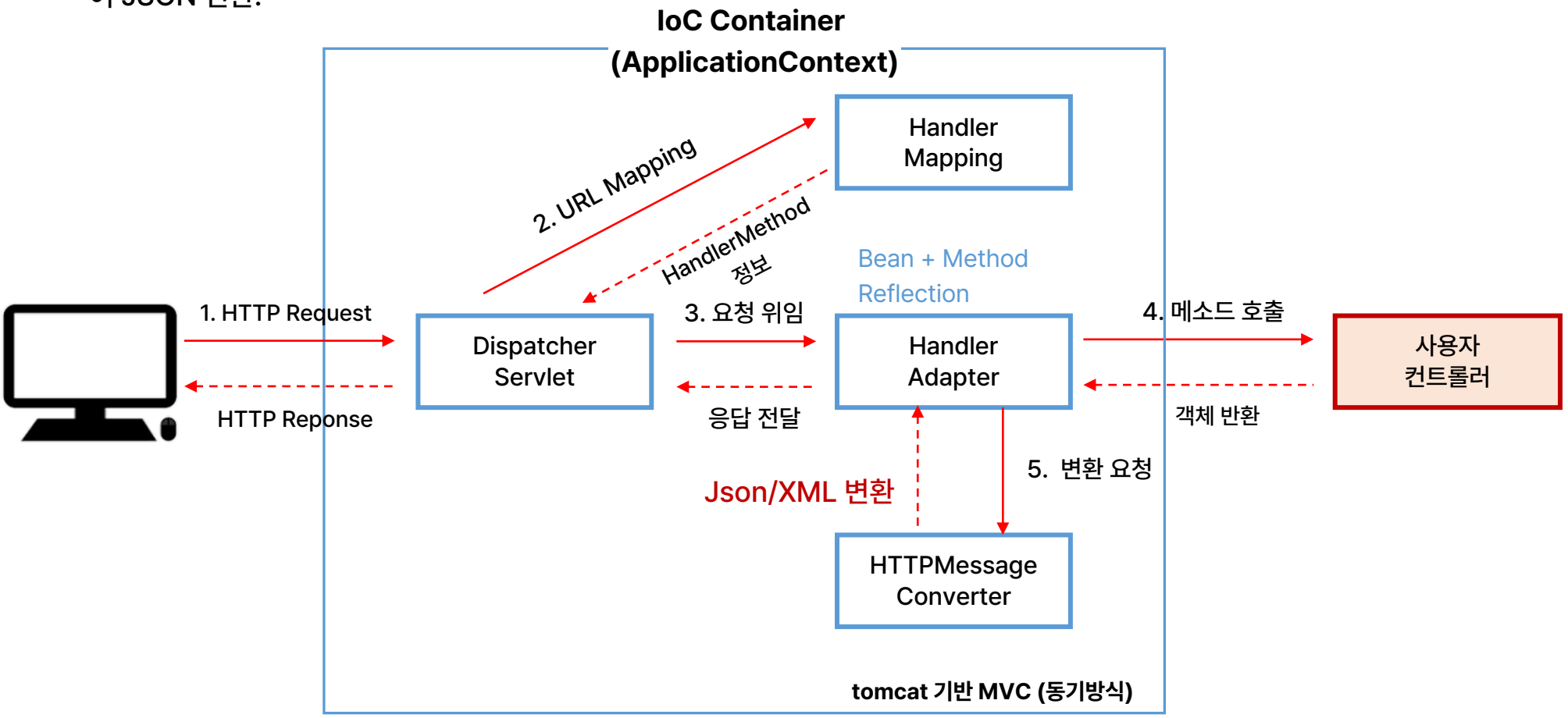
## View 컴포넌트의 의미

- DispatcherServlet이 ViewResolver를 통해 찾은 객체
- render(model, request, response) 메서드를 실행하여 모델 데이터를 이용해 최종 응답을 만듦
- HTML일 수도 있고, JSON, XML, PDF, 이미지 파일일 수도 있음

| View 구현체 | 설명 | 결과물 |
|---|---|---|
| ThymeleafView | Thymeleaf 템플릿 엔진 | HTML |
| FreeMarkerView | FreeMarker 템플릿 엔진 | HTML |
| JstlView (JSP) | JSP + JSTL | HTML |

# Rest API 기반 Spring boot MVC

- **스프링 MVC와 스프링 부트 REST API 처리의 차이**
  - 스프링 MVC View 반환: @Controller + return "뷰이름"; → 타임리프, JSP 등 ViewResolver가 동작.
  - REST API (JSON 반환): @RestController(@Controller + @ResponseBody) → HTTPMessageConverter가 동작하여 JSON 변환.

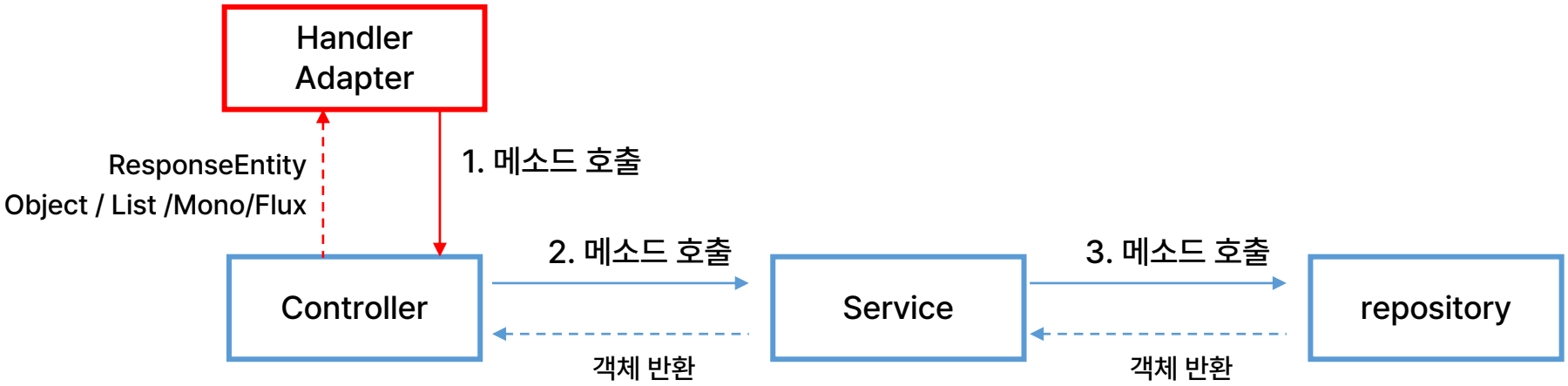


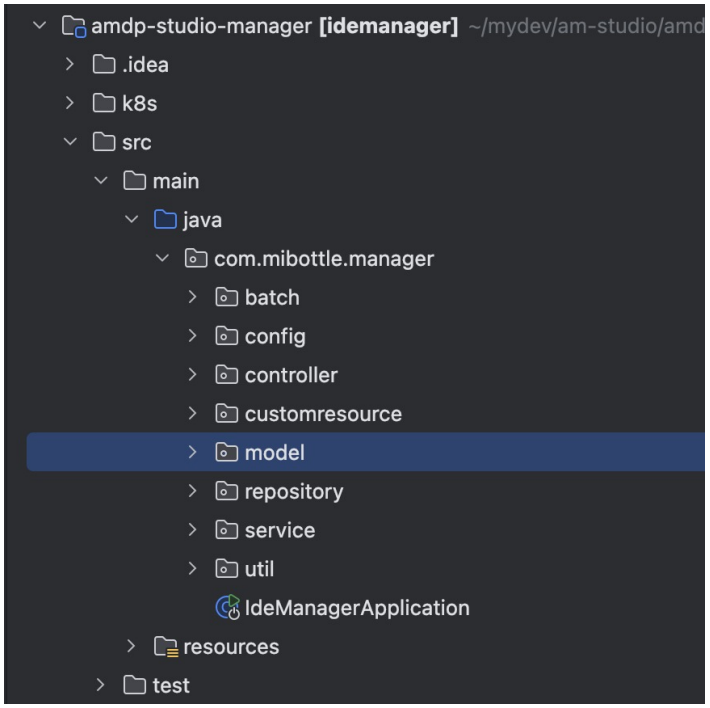
# [참고] Spring boot MVC Component

**spring 소스 위치** https://github.com/spring-projects/spring-framework

| 단계 | 클래스 | 설명 |
|---|---|---|
| Request 수신 | DispatcherServlet | 모든 HTTP 요청을 처리하는 프론트 컨트롤러<br>Servlet은 HTTP Message 수신 Listener |
| Handler Mapping | RequestMappingHandlerMapping | @RequestMapping(혹은 @GetMapping 등)과 매핑되는<br>핸들러 (Controller의 Method)를 찾음 |
| Handler Adapter | RequestMappingHandlerAdapter<br>HandlerMethodArgumentResolver | 컨트롤러 메서드를 실행하는 역할<br>Method에 대응되는 input parameter 자동 주입 연결 |
| Controller Bean | @RestController | 컨트롤러 역할을 하는 Spring Bean |
| Reflection 호출 | java.lang.reflect.Method#invoke | 실제로 컨트롤러의 메서드를 호출하는 Java Reflection API |
| Message Converter | MappingJackson2HttpMessageConverter | Java 객체를 JSON으로 변환 |
| Servlet Response | HttpServletResponse | 변환된 응답을 클라이언트에게 전달 |

# Rest API 기반 Spring boot MVC



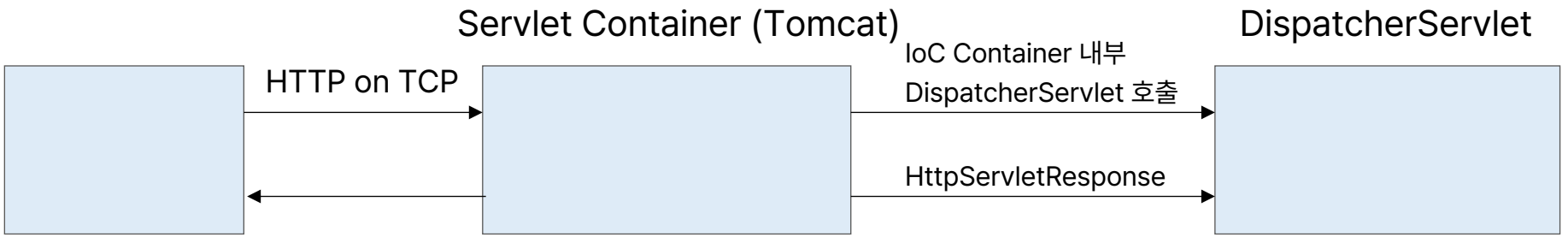
# [참고] Servlet 동작 구조

**(1) TCP 소켓 & HTTP Request 생성**

- 톰캣(Tomcat) 같은 서블릿 컨테이너가 TCP 포트(예: 8080) 를 열어 클라이언트 연결을 대기
- TCP 연결이 들어오면 HTTP 프로토콜을 해석해서 HttpServletRequest / HttpServletResponse 객체를 생성

**(2) DispatcherServlet 실행**

- 서블릿 컨테이너가 URL 매핑에 따라 DispatcherServlet.service() 호출
- service() → doGet() / doPost() 같은 메서드 실행
- 이때 이미 서블릿 컨테이너가 만들어 준 HttpServletRequest/HttpServletResponse를 인자로 받음




- TCP 소켓 관리
- HTTP Parsing
- HttpServletRequest / HttpServletResponse 생성
- IoC Container 내부 DispatcherServlet 호출

- Spring MVC 요청 처리
- Controller 호출
- HttpServletResponse 생성



# Controller와 HTTP 매개변수

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# Controller의 역할

클라이언트 요청을 가장 먼저 받아 해석하고, 전체적인 흐름을 제어하는 통제소 (Dispatcher)

### Controller의 역할

- **요청 접수 및 라우팅 (Request Mapping)**
  HTTP 요청 진입점

- **요청 데이터 검증 및 변환 (Data Validation & Binding)**
  Java 객체 변환 및 변환된 데이터 유효성 검증

- **비즈니스 로직 오케스트레이션 (Orchestration)**
  Service 계층 호출 및 데이처 처리 위임

- **응답 구조화 및 반환 (Response Generation)**
  JSON 데이터 변환 및 HTTP 상태코드 주입

# Controller 정의

스프링에서 컨트롤러를 지정해주기 위한 어노테이션으로는 @Controller 와 @RestController 가 존재

- Spring MVC 의 컨트롤러는 @Controller 를 활용
- Rest API 컨트롤러는 @RestController 를 활용 (JSON 데이터)

이 두 어노테이션의 큰 차이는 HTTP Response Body 가 생성되는 방식

# @Controller

## @Controller 는 View 를 반환하기 위해 사용함

Json Data 를 반환을 위해서는

- @ResponseBody 어노테이션 활용
- 반환되는 객체가 JSON 으로 serialized 되어 반환됨

```
@Controller
public class TestController {
    private final TestService testService;

    public TestController(TestService testService) {
        this.testService = testService;
    }

 // view 이름 반환
    @GetMapping(value = "/test/view")
    public String findView(Model model, @RequestParam String name) {
        Test result = testService.find(name);
        model.addAttribute("test", result);

        return "/test/view";      → resources/templates/test/view.html 을 전달
    }

    // JSON 데이터 반환
    @GetMapping(value = "/test")
    public @ResponseBody ResponseEntity<Test> find(@RequestParam String name) {
        return ResponseEntity.ok(testService.find(name));
    }

}
```

# @RestController

Rest API 형태 처리를 위한 Controller Annotation

@RestController는 @Controller + @ResponseBody 과 동일하게 동작

```
@RestController
public class TestController {
    private final TestService testService;

    public TestController(TestService testService) {
        this.testService = testService;
    }

    // JSON 데이터 반환
    @GetMapping(value = "/test")
    public ResponseEntity<Test> find(@RequestParam String name) {
        return ResponseEntity.ok(testService.find(name));
    }

    // view 이름 반환 (RestController에서도 가능하지만 JSON이 아닌 문자열 반환)
    @GetMapping(value = "/test/view")
    public String findView(@RequestParam String name) {
        Test result = testService.find(name);

        // 여기서는 예시로 단순 문자열 반환
        return "/test/view";
    }
}
```

# @RestController

- RESTful 웹 서비스를 만들 때 사용하는 핵심 어노테이션
- 클래스 내의 모든 메서드가 HTTP 요청을 받아서 JSON, XML 등의 형태로 응답을 자동으로 반환

| HTTP 메서드 | 어노테이션 | 설명 | 사용 예시 |
|---|---|---|---|
| GET | @GetMapping | 리소스 조회, 데이터 요청 | @GetMapping("/users") |
| POST | @PostMapping | 리소스 생성, 데이터 전송 및 등록 | @PostMapping("/users") |
| PUT | @PutMapping | 리소스 전체 수정 | @PutMapping("/users/{id}") |
| PATCH | @PatchMapping | 리소스 일부 수정 | @PatchMapping("/users/{id}") |
| DELETE | @DeleteMapping | 리소스 삭제 | @DeleteMapping("/users/{id}") |
| (모든 메서드) | @RequestMapping | 모든 HTTP 메서드에 사용 가능<br>- method로 지정 | @RequestMapping(value="/users",<br>method=RequestMethod.GET) |

# [참고] RequestMapping 의 활용

- Spring MVC에서 HTTP 요청 URL과 메서드/클래스를 매핑하기 위한 가장 기본적인 어노테이션

### @GetMapping, @PostMapping 등의 상위 개념

@GetMapping, @PostMapping 등은 @RequestMapping의 method 속성을 축약한 버전

```
@RequestMapping(value="/test", method=RequestMethod.GET)
```
=
```
@GetMapping("/test")
```

### 클래스 레벨에서 공통 URL Prefix 설정 가능

- 클래스에 붙이면 해당 컨트롤러의 모든 메서드 URL 앞에 공통 경로로 사용
- 예) @RequestMapping("/api")를 클래스에 선언 시, @GetMapping("/users")의 실제 경로는 /api/users

```
@RestController
@RequestMapping("/api") // 클래스 레벨에서 공통 prefix 설정
public class UserController {

    // GET /api/users
    @RequestMapping(value = "/users", method = RequestMethod.GET)
    public List<String> getUsers() {
        return List.of("홍길동", "이순신", "강감찬");
    }
```

# 파라미터 바인딩 어노테이션

- 클라이언트가 보낸 HTTP 요청의 다양한 정보(URL 경로 변수, 쿼리 파라미터, 요청 본문 등)를 컨트롤러 메서드의 매개변수(파라미터)에 손쉽게 연결(바인딩)해주는 역할

| 어노테이션 | 역할/용도 |
|---|---|
| @PathVariable | URL 경로의 값을 메서드 파라미터로 바인딩 |
| @RequestParam | 쿼리스트링, 폼 파라미터를 메서드 파라미터로 바인딩 |
| @RequestBody | HTTP 요청 Body(JSON, XML 등)를 자바 객체로 변환 |
| @ModelAttribute | 폼 데이터/쿼리 파라미터를 객체에 바인딩 |
| @RequestHeader | HTTP 요청 헤더 값을 파라미터로 바인딩 |
| @CookieValue | 쿠키 값을 파라미터로 바인딩 |

# 파라미터 바인딩 – @PathVariable

- URL 경로에 포함된 값을 메서드 파라미터로 바인딩

**Client Get Request:** http://localhost:8080/users/100

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable("id") Long id) {
    // /users/100  → id = 100
}
```

- URL의 **{id} 부분이 파라미터로 전달**됨
- 변수명이 같으면 **@PathVariable Long id**처럼 이름 생략 가능

# 파라미터 바인딩 – @RequestParam

- 쿼리 파라미터 또는 폼 데이터(POST form 등)를 바인딩 (?key=value 형태)

**Client Get Request:** http://localhost:8080/search?keyword=Spring

```
@GetMapping("/search")
public List<User> search(@RequestParam("keyword") String keyword) {
    // /search?keyword=Spring  → keyword = "Spring"
}
```

**Client Get Request:** http://localhost:8080/search?name=kim&age=20

```
@GetMapping("/search")
public String search(@RequestParam String name, @RequestParam int age) {...}
```

```
# 필수 여부, 기본값 지정도 가능

@RequestParam(required=false, defaultValue="all") String type
```

# [참고] 배열 형식의 파라미터 전달

- 같은 파라미터 이름을 반복해서 전달:

**Get Request:** http://localhost:8080/search?category=java&category=spring&category=boot

```
// 배열 형식으로 수신
@GetMapping("/search")
public String search(@RequestParam("category") String[] categories) {
    // categories: ["java", "spring", "boot"]
    return Arrays.toString(categories);
}

// List로 수신
@GetMapping("/search")
public String search(@RequestParam List<String> category) {
    // category: [java, spring, boot]
    return category.toString();
}
```

- 콤마(,)로 구분된 문자열 전달:

**Get Request:** http://localhost:8080/search?category=java&category=java,spring,boot

```
@GetMapping("/search")
public String search(@RequestParam("category") List<String> categories) {
    // categories: ["java", "spring", "boot"]
    return categories.toString();
}
```

❖ Spring은 기본적으로 콤마(,)로 구분된 값을 분리해서 리스트/배열로 변환

# 파라미터 바인딩 – @RequestBody

- HTTP 요청의 본문(Body)에 담긴 JSON, XML 등 데이터를 자바 객체로 자동 변환(Mapping)
- 주로 POST, PUT, PATCH에서 사용

```
curl -X POST http://localhost:8080/users \
     -H "Content-Type: application/json" \
     -d '{
           "name": "홍길동",
           "email": "hong@example.com",
           "age": 30
         }'
```

```
@PostMapping("/users")
public User createUser(@RequestBody User user) {
    // 요청 body의 JSON → User 객체 자동 매핑
}
```

# [참고] Javascript Fetch

- Javascript Fetch를 통해 Post 실행

```
fetch('http://localhost:8080/users', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: '홍길동’,
        age: 30
    })
})
.then(res => res.json())
.then(data => console.log('등록된 사용자:', data))
.catch(err => console.error(err));
```

# 파라미터 바인딩 – @RequestHeader

- HTTP 요청 헤더 값을 파라미터로 받음 (ex. 인증, 토큰, User-Agent 등)

```
curl -X GET "http://localhost:8080/api/headers" \
  -H "Authorization: Bearer eyJhbGciOi..." \
  -H "Content-Type: application/json" \
  -H "Origin: https://app.example.com" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: Authorization,Content-Type" \
  -H "Cookie: theme=dark; SESSION=abc123"
```

```
@GetMapping("/users/me")
public User getMe(
           @RequestHeader("Authorization") String token,
           @RequestHeader(value = "Content-Type", required = false) String contentType,
           @RequestHeader(value = "Origin", required = false) String origin,
           …
) {
    // Authorization: Bearer eyJhbGci...  → token 파라미터로 전달
}
```

# 파라미터 바인딩 – @CookieValue

- 요청 쿠키 값을 파라미터로 받음

```
curl -X GET http://localhost:8080/visit \
     -H "Cookie: visitTime=2025-08-15T13:30:00"
```

```
@GetMapping("/visit")
public String visit(@CookieValue("visitTime") String visitTime) {
    // 쿠키에 저장된 visitTime 값 사용
}
```

# [실습] myapp에 UserController 추가

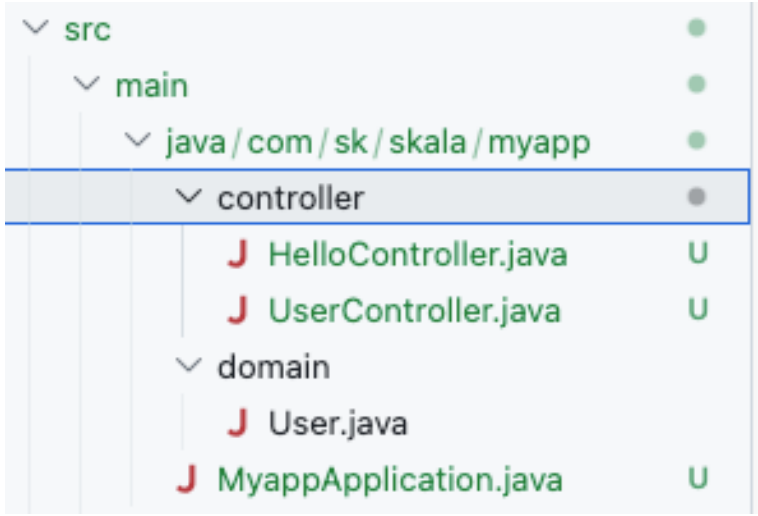
- User 정보를 검색, 변경, 삭제하는 UserContoller를 개발해보자

**1. controller/UserController와 domain/User Class를 추가한다**

**2. Controller에 다음 API를 지원하도록 코드를 개발하자**

- **GET** → /api/users, /api/users?name=xxx, /api/users/{id}
- **POST** → /api/users + JSON body
- **PUT** → /api/users/{id} + JSON body
- **DELETE** → /api/users/{id}

**package 구조**



# [실습] domain/User.java 코드 추가

```
public class User {
  private long id;
  private String name;
  private String email;

  // 생성자, getter 및 setter
  public User(long id, String name, String email) {}
  public long getId() {}
  public void setId(long id) {}
  public String getName() {}
  public void setName(String name) {}
  public String getEmail() {}

  public void setEmail(String email) {}
}
```

# [실습] controller/UserController 코드 추가 (1/2)

```
@RestController
@RequestMapping("/api")
public class UserController {
    private List<User> users = new ArrayList<>(List.of(
        new User(1, "alice", "alice@example.com"),
        new User(2, "bob", "bob@example.com"),
        new User(3, "charlie", "charlie@example.com")
    ));
    private long userIdCounter = 4;
```

userIdCounter 는 Post 시 추가하는 위치 지정

# [실습] controller/UserController 코드 추가 (2/2)

```
 // 모든 사용자 조회
@GetMapping("/users")
public List<User> getAllUsers() {…}

// GET: 특정 사용자 가져오기
@GetMapping("/users/{id}")
public User getUserById(@PathVariable long id) {…}

// GET : @RequestParam 으로 특정 사용자 가져오기
@GetMapping("/users")
public User getUserByEmail(@RequestParam long id) {…}

// POST: 사용자 추가
@PostMapping("/users")
public User createUser(@RequestBody User user) {…}

// DELETE: 사용자 삭제
@DeleteMapping("/users/{id}")
public void deleteUser(@PathVariable long id) {…}

// PUT: 사용자 정보 수정
@PutMapping("/users/{id}")
public User updateUser(@PathVariable Long id, @RequestBody User updatedUser) {…}
```

# [실습] 실행하기

### 1. GET (모든 사용자 조회)

```
# 모든 사용자 조회
curl -X GET http://localhost:8080/api/users

# 특정 이름으로 조회 (예: alice)
curl -X GET "http://localhost:8080/api/users?name=alice"

# 특정 ID 사용자 조회 (예: id=2)
curl -X GET http://localhost:8080/api/users/2
```

### 2. POST (새 사용자 추가)

```
curl -X POST http://localhost:8080/api/users \
  -H "Content-Type: application/json" \
  -d ‘{
        "id": 4,
       "name": "skala",
       "email": ”skala@example.com"
     }'
```

### 3. PUT (기존 사용자 수정)

```
curl -X PUT http://localhost:8080/api/users/4 \
  -H "Content-Type: application/json" \
  -d '{
        "name": "bobby",
        "email": "bobby@example.com"
      }'
```

### 4. DELETE (사용자 삭제)

```
curl -X DELETE http://localhost:8080/api/users/1
```

# [참고] 참고 코드

아래 코드를 참고할 수 있음

```
01.training-code/02.layer/01.myapp-controller
```



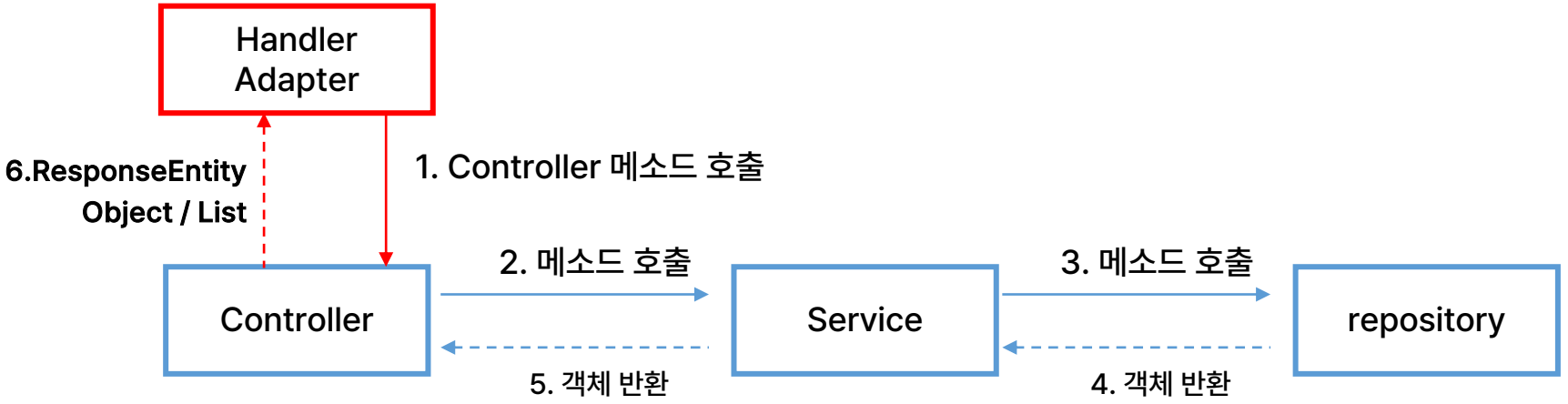
# Service & Repository

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# 기본 API Service 구성

객체의 생성과 의존성 관리를 개발자가 직접 하는 것이 아니라, **IoC 컨테이너 (Spring F/W 에서) 가 대신 수행**하는 개념




클라이언트로부터 들어오는 HTTP Request의 객체을 받고, 응답 객체 또는 ReponseEntity 를 반환하는 계층

비즈니스 로직(구현 하고 자하는 업무 로직)을 담당하는 계층

데이터베이스 (H2, Mariadb, postgre ...)

# Repository의 역할

비즈니스 도메인 객체 (Entity)를 영구적으로 저장하고 관리하는 데이터베이스 연결 역할

## Repository의 역할

- **데이터의 영속성 제공 (Persistence)**
  - 메모리 저장 객체를 데이터베이스에 영구 저장

- **데이터베이스 기술 및 쿼리 숨김 (Abstraction / Encapsulation)**
  - 복잡한 SQL문이나 ORM(JPA, Hibernate, MyBatis 등)의 구체적인 사용법을 Repository 내부에 숨김
  - 객체 지향적인 메서드 호출만으로 데이터를 획득/처리

- **영속성 컨텍스트 관리 및 데이터 매핑 (Data Mapping)**
  - Java 객체와 데이터 테이블간 변환 및 변경 사항 감지/동기화

- **인프라 및 기술 환경과의 격리 (Decoupling)**
  - 특정 DB 기술 (SQL, JPA 등)에 직접 종속되지 않음
  - 외부 환경의 변화에 코드가 수정되지 않도록 구성

# Repository 구현

데이터베이스와 직접 통신하는 계층 (CRUD 담당)

**특징:**

- JpaRepository를 상속받은 UserRepository를 선언 시 자동 구현체 생성
- JPA, MyBatis, JDBC 등을 활용
- Spring Data JPA의 경우 메서드 네이밍만으로 SQL 자동 생성 가능

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByName(String name);   // 추가 생성한 메소드 선언
}
```

**자동 구현되는 Method**

- findAll() → 모든 엔티티 조회
- findById(ID id) → 특정 ID로 조회 (Optional 반환)
- **save(S entity) → 엔티티 저장 (insert or update)**
- deleteById(ID id) → ID로 삭제
- delete(T entity) → 엔티티 삭제

# User Entity 만들기

Repository를 통해 Java Object와 DB Table 간 매핑하고 동기화하기 위해
User 객체를 Domain Entity로 선언 필요 (JPA, ORM)

```
@Entity
@Table(name = "users")
public class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private long id;
  private String name;
  private String email;

  // 기본 생성자 (JPA 필수)
  public User() {}

..
  기존 Constructure, Getter, Setter 유지
```

# Service의 역할

소프트웨어 본질인 핵심 비즈니스 규칙을 정의하고 트랜잭션의 단위를 관리하는 역할 수행

## Service의 역할

- **핵심 비즈니스 로직 수행 (Core Business Logic)**

- **트랜잭션 경계 설정 및 관리 (Transaction Management)**
  - 데이터베이스 작업의 원자성 (Atomicity)을 보장하는 트랜잭션 단위
  - @Transactional annotation 선언 위치

- **Repository 오케스트레이션 (Data Orchestration)**
  - 여러 Repository (데이터 접근 계층)를 조합하고 제어

- **인프라 및 기술 환경과의 격리 (Decoupling)**
  - 웹 기술 (HTTP, JSON, Controller)이나 특정 DB 기술 (SQL, JPA 등)에 직접 종속되지 않음
  - 외부 환경의 변화에 코드가 수정되지 않도록 구성

# Service 구현

실제 구현하고자 하는 비즈니스 로직을 구현하는 계층

**특징**:

- Controller와 Repository 사이의 중간다리
- 트랜잭션 관리, 여러 Repository 조합 처리
- 가급적 DB 접근 로직은 넣지 않고, 순수한 로직을 구현 (역할을 명확하게 구분)

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(FileUserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // 이름 필터처럼 내부적 로직 처리는 가능하면 Service에서 처리
    public List<User> findAll(Optional<String> name) {
        Collection<User> all = userRepository.findAll();
        if (name.isPresent()) {
            String search = name.get();
            return all.stream()
                    .filter(u -> u.getName() != null && u.getName().equalsIgnoreCase(search))
                    .collect(Collectors.toList());
        }
        return new ArrayList<>(all);
    }

    public Optional<User> findById(long id) {
        return userRepository.findById(id);
    }
```

# 의존관계 자동 주입 (DI)

Controller, Service, Repository 의 호출을 위해 @Autowire annotation 또는 생성자 함수를 통해 자동 의존성 주입

```
@RestController
@RequestMapping("/api")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/users")
    public List<User> getAllUsers(@RequestParam Optional<String> name) {
        return userService.findAll(name);
    }

    @GetMapping("/users/{id}")
    public ResponseEntity<User> getUserById(@PathVariable long id) {
        log.info("getUserById called with id: {}", id);
        return userService.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
```

# [실습] Controller-Service-Repository 만들기

**기존 myapp 프로젝트를 Controller → Service → Repository 계층을 구성해보자**

① repository와 service package를 생성하자
② UserRepository를 생성해서 H2 DB에 CRUD를 할 수 있도록 구성하자
③ 기존 순수 User Java를 User Entity로 변경해보자
④ UserService.java를 생성하고, UserController에서 호출할 수 있는 method를 만들자.
⑤ UserController는 UserService의 메소드를 호출하는 방식으로 전환하자



# [실습] Repository 만들기

## 1. 기존 User Class를 User Entity 로 변경하기

```
@Entity
@Table(name = "users")
public class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private long id;
  private String name;
  private String email;

…
```

## 2. UserRepository 생성하기

```
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
}
```

# [실습] H2 환경 구성 및 초기 데이터 주입하기

## 3. application.properies 설정

```
spring.application.name=myapp

# H2 설정
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# H2 콘솔 활성화
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA 설정
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# data.sql을 JPA 테이블 생성 후 실행
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always
```

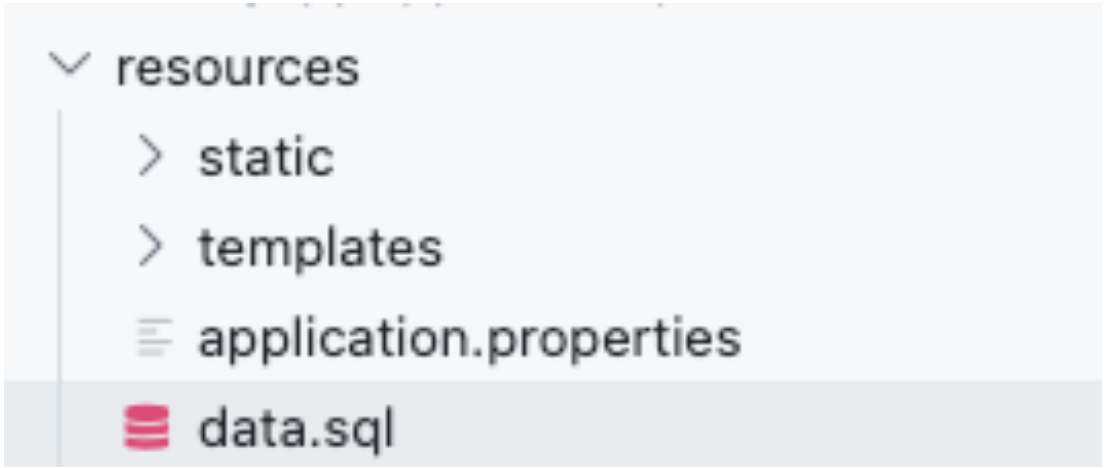
# [실습] 초기 User Data 주입

### 4. 아래의 INSERT Query를 가진 data.sql 파일을 resources 디렉토리에 생성

data.sql

```
INSERT INTO users (name, email) VALUES ('홍길동', 'hong@sk.com');
INSERT INTO users (name, email) VALUES ('김철수', 'kim@sk.com');
INSERT INTO users (name, email) VALUES ('이영희', 'lee@sk.com');
```

resoources/data.sql



# [실습] Service 만들기 (1/2)

## 5. UserService 생성하기

```
@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // 모든 사용자 조회
    public List<User> getAllUsers() {…}

    // 특정 사용자 조회
    public Optional<User> getUserById(long id) {…}

    // 사용자 추가
    public User createUser(User user) {…}

    // 사용자 삭제
    public void deleteUser(long id) {…}

    // 사용자 정보 수정
    public Optional<User> updateUser(long id, User updatedUser) {…}
}
```

# [실습] Service 만들기 (2/2)

**Service의 각 메소드는 아래의 userRepository 메소드를 호출해서 완성하면 됩니다.**

| 유형 | method | 설명 |
|---|---|---|
| Query | userRepository.findAll() | 테이블의 모든 사용자 데이터를 조회<br>반환 타입은 일반적으로 List<User> |
| Query | userRepository.findById(id) | 전달받은 ID에 해당하는 사용자 한 명을 조회<br>데이터가 없을 수 있으므로 Optional<User> 반환 |
| Insert / Update | userRepository.save(user) | 새로운 사용자이면 INSERT,<br>이미 존재하는 사용자이면 UPDATE를 수행<br>저장된 User를 반환 |
| Delete | userRepository.deleteById(id) | 전달받은 ID에 해당하는 사용자를 삭제<br>반환값은 void |

# [실습] Controller 수정하기

## 6. UserController에서 UserService 기반 메소드를 호출하는 방식으로 method 수정하기

```
@RestController
@RequestMapping("/api")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/users")
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

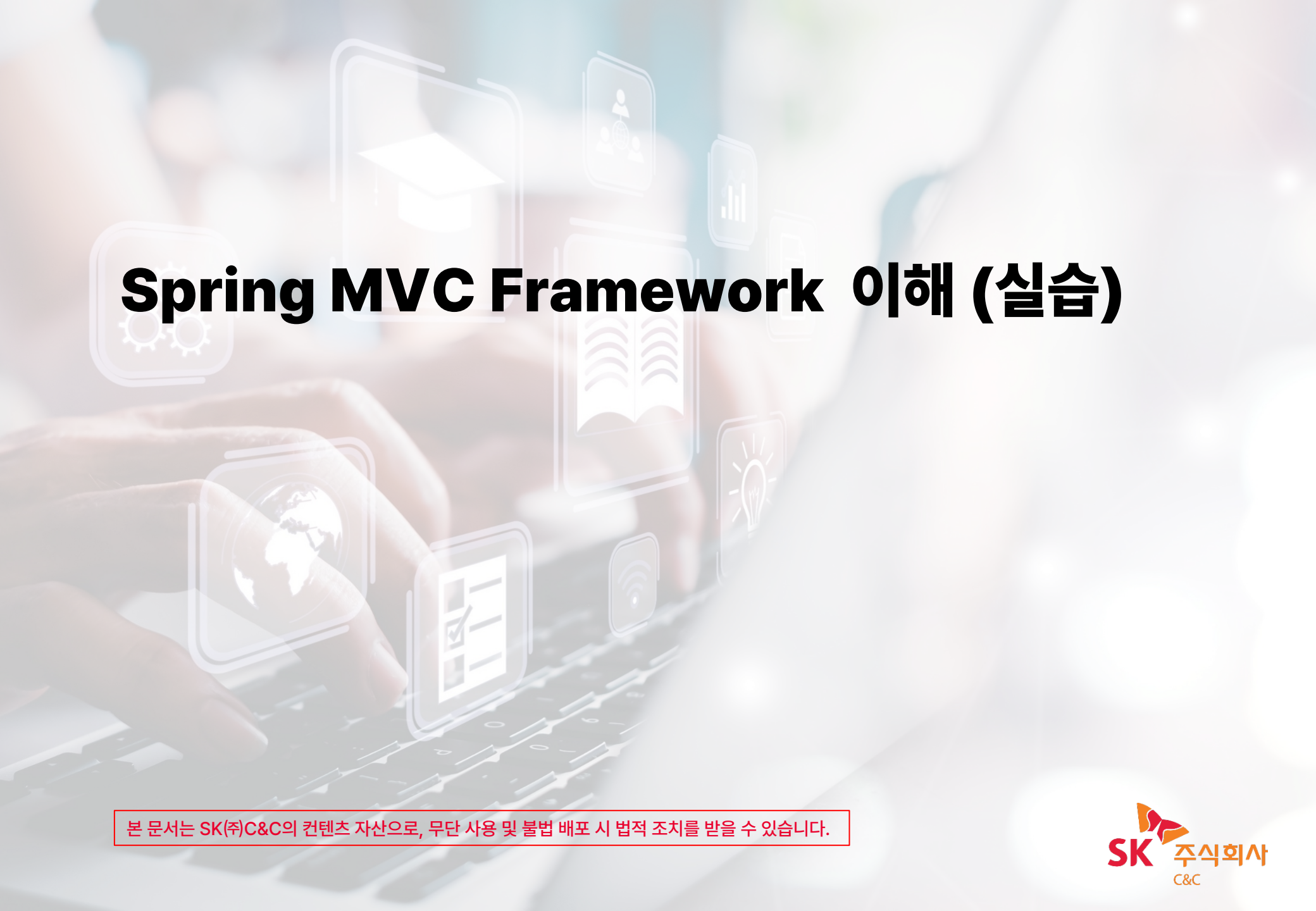
    // GET: 특정 사용자 가져오기
    @GetMapping("/users/{id}")
    public User getUserById(@PathVariable long id) {
        return userService.getUserById(id).orElse(null);
    }
    …

}
```

# [참고] 참고 코드

아래 코드를 참고할 수 있음

```
01.training-code/02.layer/02.myapp-svc-repo
```



# Spring MVC Framework 이해 (실습)

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# [실습] 샘플로 Spring 구조 이해하기

01.training-code/17.http-api-server

**Java Annotation, Reflection, Socket 기능을 이용해서 간단한 HTTP Server 를 구성해본다.**

- @Controller, @PathVariable, @PostMapping, @GetMapping 의 동작 이해
- Controller component를 로딩해서 등록 방식 이해
- Browser/Client 에서 HTTP Request 요청 시 Socket을 통해 어떻게 수신하고 Controller를 호출하는지 이해

**01.training-code/17.http-api-server 아래의**

- GUIDE.md 파일을 읽고,
- 코드를 검토해보자

# [실습] 샘플로 Spring 구조 이해하기

## 1) 서버 실행 (02.spring/01.training-code/17.http-api-server 로 이동)

```
> javac -d out $(find sk -name "*.java") && echo BUILD_OK

> java -cp out sk.skala.com.httpserver.Main
BUILD_OK
[컨트롤러 등록] UserController
  GET  /users → getUsers()
  GET  /users/{id} → getUser(경로 변수)
  POST  /users → createUser()

HttpWebServer started on port 8080
등록된 라우트 수: 2
========================================
```

## 2) Client 실행

```
> java HttpWebClientjava
```

## 3) Chrome Browser에서 다음 URL 호출 하기

```
http://localhost:8080/users

http://localhost:8080/users/1

http://localhost:8080/users/2

http://localhost:8080/notfound
```



# Lombok

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# Lombok 이란

Lombok은 인도네시아 서누사텡가라 지방에 있는 섬...

Java 는 인도네시아 섬 자바 그리고 인도네시아 섬 Lombok..




## Lombok은 ?

개발자의 boilerplate code를 줄이고 가독성을 높일 수 있도록 지원하는 Java 라이브러리

## Boilerplate Code(보일러플레이트 코드)는 ?

- 반복적이고 본질적인 로직과 무관하지만 반드시 작성해야 하는 코드
- Java에서는 객체의 필드를 다룰 때
  getter(), setter(), equals(), hashCode(), toString() 메서드 등이 대표적 사례

# 주요 Lombok 어노테이션

| 어노테이션 | 역할/설명 | 코드 사용 위치 |
|---|---|---|
| @Getter | 모든 필드에 대해 getter 메서드 자동 생성 | 클래스 |
| @Setter | 모든 필드에 대해 setter 메서드 자동 생성 | 클래스 |
| @ToString | toString() 메서드 자동 생성 | 클래스 |
| @EqualsAndHashCode | equals()와 hashCode() 메서드 자동 생성 | 클래스 |
| @NonNull | Null을 넣으면 즉시 NullPointException 발생 시킴 | 필드 |
| **@NoArgsConstructor** | 기본 생성자(파라미터 없는 생성자) 자동 생성 | 클래스 |
| **@AllArgsConstructor** | 모든 필드를 파라미터로 받는 생성자 자동 생성 | 클래스 |
| **@RequiredArgsConstructor** | final, @NonNull 필드만을 모아서 생성자 함수를 자동 생성 | 클래스 |
| **@Data** | @Getter, @Setter, @ToString, @EqualsAndHashCode, @RequiredArgsConstructor를 한 번에 적용 | 클래스 |
| **@Builder** | 빌더 패턴 자동 생성 (객체 생성 유연화) | 클래스 |
| **@Slf4j** | 로그 변수(log) 자동 생성 (Log 객체 주입) | 클래스 |

# Lombok 적용 예시

## 적용 전

```
public class User {
    private Long id;
    private String name;

    // 기본 생성자
    public User() {}

    // 모든 필드 초기화 생성자
    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getter & Setter
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    // toString() : 객체 정보를 문자열로 반환
    @Override
    public String toString() {
        return "User{id=" + id + ", name='" + name + "'}";
    }

    // equals() : 두 객체가 같은지 비교 (주로 id와 name이 모두 같을 때)
    // hashCode() : 객체를 해시 기반 컬렉션(Map, Set)에 넣을 때 사용
}
```

## Lombok 적용 후

```
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import lombok.Builder;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    private Long id;
    private String name;
}
```

# [참고] record를 이용한 Immutable 객체 만들기

- record는 java 16부터 정식 지원되는 불변(Immutable) 데이터 객체를 간결하게 정의하기 위한 문법
- constructor, getter, toString, equals, hashCode가 자동 생성 (setter 기능은 제외)
- DTO 만들기 위한 방식으로 사용

```
import lombok.Value;

public record Stock(String symbol, String name,int price) { }


//객체 생성 후 필드 값을 변경 불가
Stock stock = new Stock("005930", "삼성전자", 80000);
System.out.println(stock.name());    // "삼성전자"

// stock.price(90000);                // 컴파일 오류 (setter 없음, 값 변경 불가)
```

# Lombok - @Slf4j

- 로그(Logger) 객체를 자동으로 주입 - 클래스에 붙이면 private static final org.slf4j.Logger log 필드가 생성
- 로그 레벨별로 출력: log.info(), log.warn(), log.error(), log.debug(), log.trace()
- System.out.println 대신 사용

```
import lombok.extern.slf4j.Slf4j;

@Slf4j
public class StockService {

    public void printStock(Stock stock) {
        log.info("Stock 정보: {}", stock);
        log.debug("디버깅 로그입니다");
        log.error("에러 상황입니다");
    }
}

// 에러 출력 시에는 마지막 인자에 Exception 전달
log.error("에러 발생: {}", userId, e); // SLF4J 권장 방식
```

- SLF4J 로그 메시지 포맷은 {}만 지원.
- %s, %d 등은 지원하지 않으므로 필요하면 String.format() 사용 가능 (비권장 - 문자열 결합 및 객체 생성 연산이 발생)

# [참고] 로그 생성 기준

- Spring boot에서 로그 수준(Log Level)을 지정하려면 application.yaml 또는 application.properties에서 logging.level 속성을 설정 필요

```
logging:
  level:
    root: INFO                  # 전체 기본 로그 수준
    com.example.demo: DEBUG     # 특정 패키지 로그 수준
    org.springframework.web: WARN  # 스프링 웹 로그만 경고 이상 출력
```

**로그 수준 옵션**

Spring Boot에서 지정할 수 있는 로그 레벨은 다음과 같습니다.

default: INFO

- **TRACE** : 가장 상세한 로그 (디버깅용 세부 정보)
- **DEBUG** : 디버깅에 유용한 정보
- **INFO** : 일반적인 실행 정보
- **WARN** : 경고 메시지
- **ERROR** : 오류 메시지
- **OFF** : 로그 출력 안 함

# [실습] User에 @Data 적용하기

- @Data, @NoArgsConstructor, @AllArgsConstructor를 적용하고 필드를 제외한 모든 method를 제거해자

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private long id;
  private String name;
  private String email;
}
```

# [실습] "@Slf4j" 를 이용한 로깅 설정

- Controller와 Service 클래스에 @Slf4j 어노테이션을 설정하고 info() 및 debug() 이력 생성

## 1. resources 디렉토리아래에 application.yaml 생성 및 아래와 같이 설정

```
spring:
  application:
    name: myapp

logging:
  level:
    '[com.sk.skala.myapp.controller]': DEBUG
    '[com.sk.skala.myapp.service]': DEBUG
```

## 2. Controller method에 info debug 설정 (다른 곳에도 설정하기)

```
@Slf4j
@RestController
@RequestMapping("/api")
public class UserController {
```

```
 @GetMapping("/users")
    public List<User> getAllUsers(@RequestParam Optional<String> name) {
        log.info("getAllUsers called");
        log.debug("getAllUsers called with name filter: {}", name.orElse("none"));
        return userService.findAll(name);
    }
```

# [실습] 실행 해보기

## 3. postman 으로 호출해보기

```
http://localhost:8080/api/users
```

```
http://localhost:8080/api/users?name=1
```



# Properties 관리

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# 애플리케이션 설정 정보 관리

- application.properties / application.yml 역할
  - **환경설정 관리**: 데이터베이스 접속 정보, 서버 포트, 로깅 레벨, 외부 API 키, 커스텀 프로퍼티 등
  - **프로젝트의 런타임 환경별 설정 관리**: 개발, 테스트, 운영 환경에 따라 각기 다른 설정값을 분리하고 관리

```
# application.properties

server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/stockdb
spring.datasource.username=stockuser
spring.datasource.password=stockpw

logging.level.org.springframework=INFO

# 사용자 정의 값
stock.api.base-url=https://api.stock.com
```

```
# application.yml

server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/stockdb
    username: stockuser
    password: stockpw

logging:
  level:
    org.springframework: INFO

stock:
  api:
    base-url: https://api.stock.com
```

# 애플리케이션 설정 정보 관리

comma 로 구분하여 작성

사용하는 곳에서는 List 로 받도록 설정

1. 
```
key:
    value:
      - a
      - b
      - c
      - d
```

2. 
```
key:
    value: [a, b, c, d]
```

3. 
```
key:
    value: a, b, c, d
```

# 설정 정보 관리를 위한 의존성 주입

## 의존성 추가

```
<!-- Configuration Properties 자동 완성 및 metadata 생성용 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-configuration-processor</artifactId>
    <optional>true</optional> <!-- 컴파일 시에만 필요 -->
</dependency>
```

# 설정 정보 사용 – @Value

- 장점: 간단하게 한두 개 값 읽을 때 적합
- 단점: 값이 많아지면 관리가 어렵고, 계층구조 표현이 불편

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class StockApiClient {
    @Value("${stock.api.base-url}")
    private String baseUrl;

    // baseUrl을 사용하는 코드
    public void connect() {
        System.out.println("API URL: " + baseUrl);
    }
}
```

주입 시점: compoentScan 시 DI

# 설정 정보 사용 – @ConfigurationProperties

- 계층적 구조의 설정을 객체로 묶어 사용할 때 활용

```yaml
# application.yml

server:
  port: 8081

stock:
  api:
    base-url: https://api.stock.com
    timeout: 1000
  caching:
    enabled: true
```

```java
@Data
@Component
@ConfigurationProperties(prefix = "stock.api")
public class StockApiProperties {
    private String baseUrl;
    private int timeout;
}
```

- 사용방법

```java
@Service
public class StockApiClient {
    private final StockApiProperties stockApiProperties;

    public StockApiClient(StockApiProperties stockApiProperties) {
        this.stockApiProperties = stockApiProperties;
    }

    public void connect() {
        System.out.println("API URL: " + stockApiProperties.getBaseUrl());
    }
}
```

# 설정 정보 사용 – @ConfigurationProperties

- **계층적 구조의 설정**을 객체로 묶어 사용할 때 활용

```yaml
# application.yml

server:
  port: 8081

stock:
  api:
    base-url: https://api.stock.com
    timeout: 1000
  caching:
    enabled: true
```

```java
@Data
@Component
@ConfigurationProperties(prefix = "stock")
public class StockProperties {
    private Api api;
    private Caching caching;

    @Data
    public static class Api {
        private String baseUrl;
        private int timeout;
    }

    @Data
    public static class Caching {
        private boolean enabled;
    }
}
```

# 설정 정보 클래스 서비스 사용 예시

```
@Service
public class StockApiClient {

    private final StockProperties stockProperties;

    public StockApiClient(StockProperties stockProperties) {
        this.stockProperties = stockProperties;
    }

    public void connect() {
        String baseUrl = stockProperties.getApi().getBaseUrl();
        int timeout = stockProperties.getApi().getTimeout();
        boolean cachingEnabled = stockProperties.getCaching().isEnabled();

        System.out.println("API URL: " + baseUrl);
        System.out.println("Timeout: " + timeout);
        System.out.println("Caching Enabled: " + cachingEnabled);
    }
}
```

# 환경 별 애플리케이션 프로파일 분리

- 런타임 환경(개발, 테스트, 운영 등)에 따라 설정 파일명을 구분하여 관리
  - application-**dev**.yml (개발)
  - application-**prod**.yml (운영)
  - application-**test**.yml (테스트)

- **실행 명령어에서 환경(profile) 지정 방법**

**JVM 옵션으로 적용하는 방식**

```
java -Dspring.profiles.active=dev -jar skala-stock-api.jar
```

❖ JAVA 환경변수 처리(Spring Boot가 내부적으로 System.getProperty를 통해 참조)

**Spring Boot 옵션으로 적용하는 방식**

```
java -jar skala-stock-api.jar --spring.profiles.active=dev
```

❖ **Spring Boot 옵션(--spring.profiles.active=dev)**은 jar 뒤에 사용 → 애플리케이션 아규먼트로 전달

# [실습] 환경별 Properties 사용하기 (1/4)

local 과 prod 프로파일 별로 설정이 다르게 동작하도록 profile을 분리해보자

1. application.yaml을 기본으로 application-local.yaml과 application-prod.yaml로 분리
2. profile 별로 다른 설정값 구성

```
# application.yaml

spring:
  profiles:
    active: local
```

```
# application-local.yml

server:
  port: 8080

developer:
  owner:
    name: yona
    role: backend
    level: middle
  team:
    position: "4th floor"
    detail: ”class 4"
```

```
# application-prod.yml

server:
  port: 8080

developer:
  owner:
    name: yona
    role: kubernetes
    level: high
  team:
    position: ”5th floor"
    detail: ”class 5"
```

# [실습] 환경별 Properties 사용하기 (2/4)

config package 구성 및 DeveloperProperties.java 생성

### 1. config package folder 생성




### 2. DeveloperProperties.java 구성

```
@Data
@Component
@ConfigurationProperties(prefix = "developer")
public class DeveloperProperties {

    private Owner owner = new Owner();
    private Team team = new Team();

    @Data
    public static class Owner {
        private String name;
        private String role;
        private String level;
    }

    @Data
    public static class Team {
        private String position;
        private String detail;
    }
}
```

# [실습] 환경별 Properties 사용하기 (3/4)

GET /api/developer-info 을 위한 DeveloperInfoController.java 생성

1. DeveloperProperties 와 Environment Bean 객체를 주입
2. DTO DeveloperInfoResponse (String activeProfile, DeveloperProperties developer) {} record

```
@RestController
@RequestMapping("api")
public class DeveloperInfoController {

    private final DeveloperProperties developerProperties;
    private final Environment environment;

    public DeveloperInfoController(DeveloperProperties developerProperties, Environment
environment) {
        this.developerProperties = developerProperties;
        this.environment = environment;
    }

    @GetMapping("/developer-info")
    public DeveloperInfoResponse getDeveloperInfo() {
        String activeProfiles = String.join(", ", environment.getActiveProfiles());
        return new DeveloperInfoResponse(activeProfiles, developerProperties);
    }
}

record DeveloperInfoResponse(String activeProfile, DeveloperProperties developer) {}
```

# [실습] 환경별 Properties 사용하기 (4/4)

profile 별 (local, prod)로 동작시키고 /api/develop-info 접속을 통해 설정 정보 변경 여부를 확인해보자

**1. local profile 실행**

java -jar ./target/skalaspringboot-0.0.1-SNAPSHOT.jar --spring.profiles.active=local

**2. prod profile 실행**

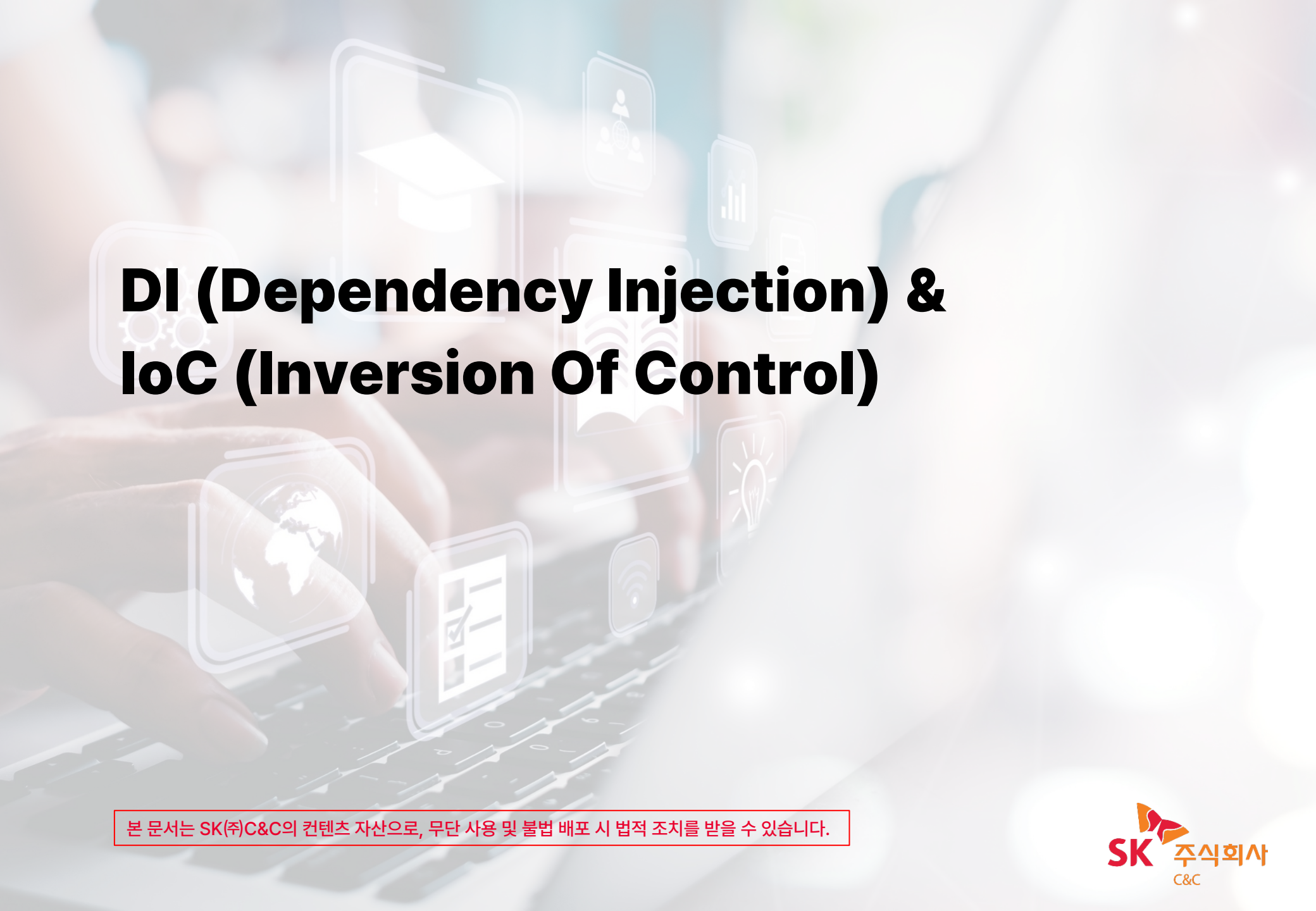
java -jar ./target/skalaspringboot-0.0.1-SNAPSHOT.jar --spring.profiles.active=prod

localhost:8080/api/developer-info 접속해서 확인하기

# [참고] VS Code 런타임 설정 실행

- .vscode/launch.json 파일에 환경변수나 아규먼트를 지정

```
{
    "configurations": [
        {
            "type": "java",
            "name": "Spring Boot-MyappApplication<myapp>",
            "request": "launch",
            "cwd": "${workspaceFolder}",
            "mainClass": "com.sk.skala.myapp.MyappApplication",
            "projectName": "myapp",
            "args": "--spring.profiles.active=prod",
            "envFile": "${workspaceFolder}/.env"
        }
    ]
}
```



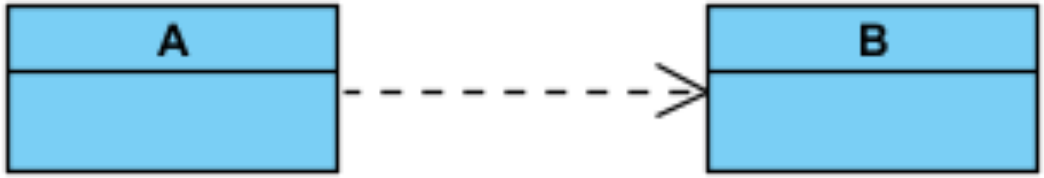
# DI (Dependency Injection) & IoC (Inversion Of Control)

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# 의존성 (Dependency)

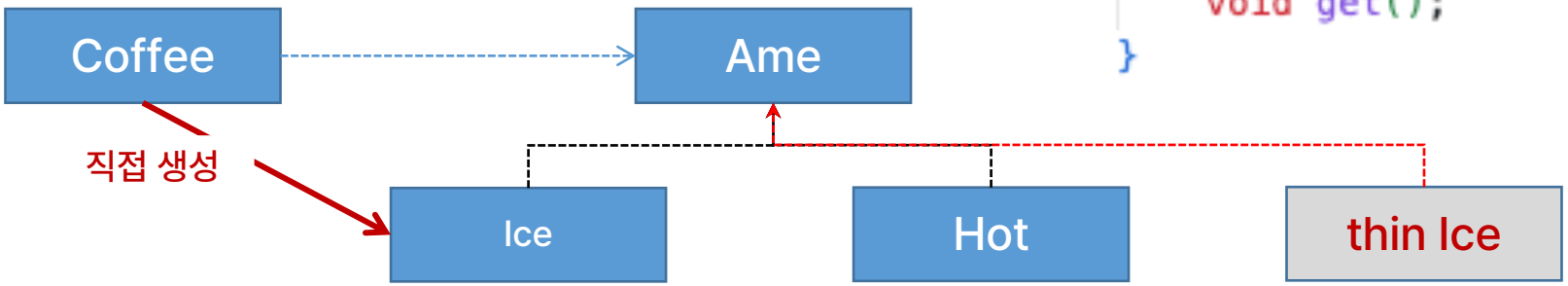
- 의존성은 하나의 객체가 자신의 기능을 수행하기 위해 반드시 필오료 하는 다른 객체를 의미
- 하나의 객체가 다른 객체의 메서드를 호출하거나 사용하는 관계를 의존성으로 정의




Class A는 Class B에 의존적이다(A가 B의 Method를 호출한다)

# 의존관계 생성

Coffee 객체는 Ame 객체를 사용한다 (Method를 호출한다)




```java
public interface Ame {
    void get();
}
```

- Coffee에서 사용하는 Ame 구현체는
- 직접 생성 (New)
- 새로운 ThinIce 객체를 사용하고자 하는 경우 코드 변경 필요

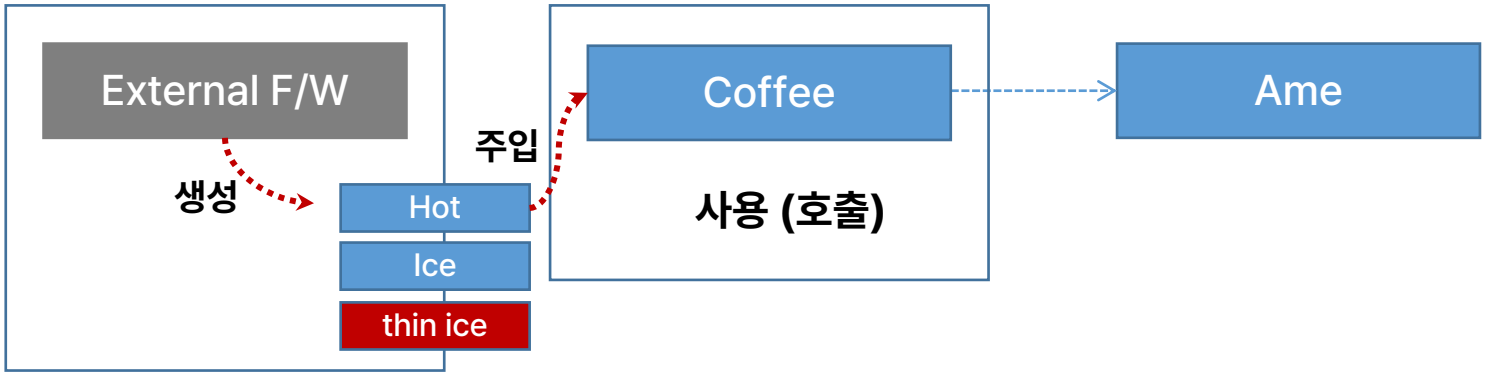
```java
public class Coffee {
    private String kind;
    private Ame ame;

    public Coffee(String kind) {
        this.kind = kind;
    }

    public void coffeeType() {
        if (kind.equals("hot")) {
            ame = new Hot();
        } else if (kind.equals("ice")) {
            ame = new Ice();
        } else if (kind.equals("thinIce")) {
            ame = new ThinIce();
        }
        ame.get();
    }
}
```

# 의존성 주입 (Dependency Injection)

Coffee 객체는 Ame 객체를 사용한다 (Method를 호출한다)




### 의존성 객체 생성은 외부에서

```
Coffee hot     = new Coffee(new Hot());
Coffee ice     = new Coffee(new Ice());
Coffee thinIce = new Coffee(new
ThinIce());
```

### 업무 로직에서는 주입된 객체를 사용만

```
public class Coffee {

    private Ame ame;

    // 외부에서 구현체를 주입받음 (생성자 주입)
    public Coffee(Ame ame) {
        this.ame = ame;
    }

    // 새로운 커피 추가 시에도 코드 업무 로직 변경 없음
    public void coffeeType() {
        ame.get();
    }
}
```

# [실습] DI 이해하기

01.traing-code/05.di/

01.trainng-code/05.di/before는 의존성 주입 전단계,
01.training-code/05.di/after는 의존성 주입 후 단계 코드이다.

1. before 의 코드를 검토해보고 실행해보자

```
> javac -d bin *.java
> java -cp bin Main
```

2. after의 코드를 검토해보고 실행해보자

```
> javac -d bin *.java
> java -cp bin Main
```

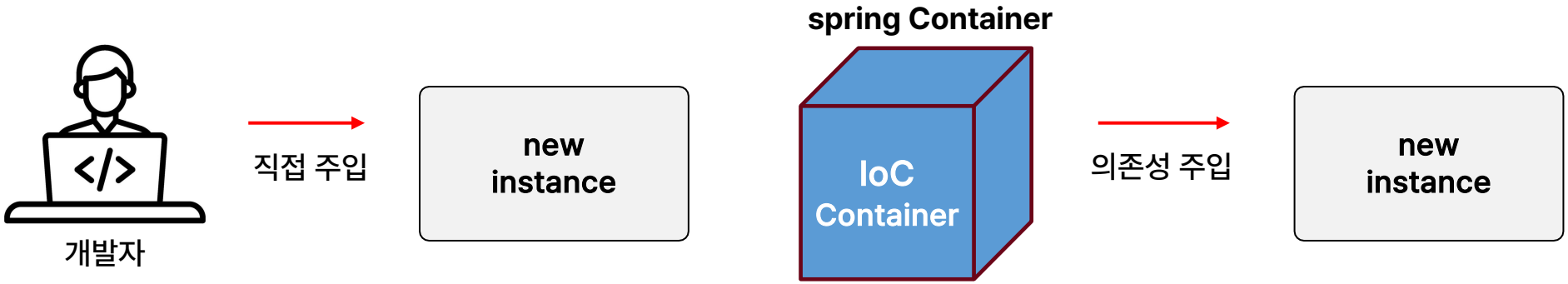
3. before와 after에 각각 ThinIce  Class를 추가해보고 어떤 차이가 있는지 이해해보자

```
// 작은 알갱이 얼음 아메리카노 구현체
public class ThinIce implements Ame {

    @Override
    public void get() {
        System.out.println("나는 연한 아이스 아메리카노");
    }
}
```

# IoC(Inversion Of Control)

- 의존성 주입을 받는 관점에서는 외부에서 의존성이 주입 (DI)로 정의
- 의존성을 컨트롤을 가지는 주체 즉 외부 (Framework: spring) 관점 IoC 로 정의




IoC Container가 @Controller, @Compoent, @Service에 포함된 Bean을 직접 주입 실행

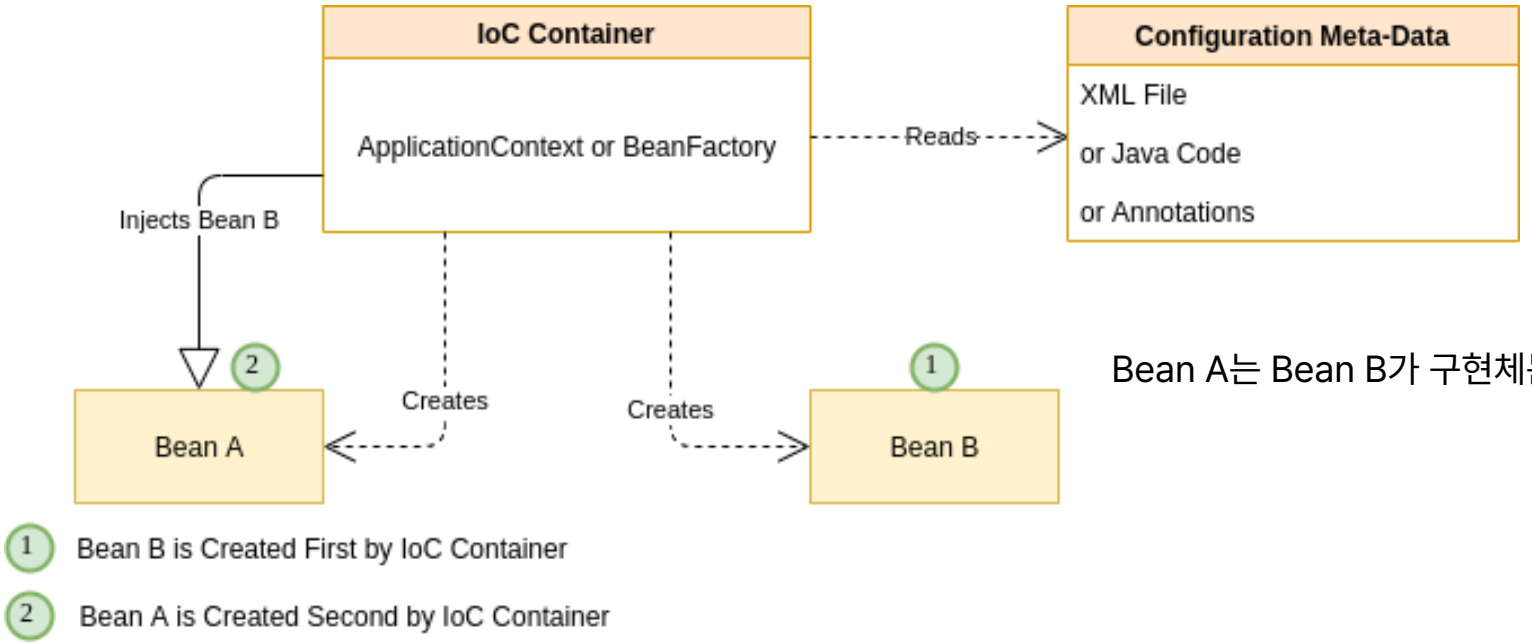
```java
@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserService userService;  생산자 의존성 주입 대상

    public UserController(UserService userService) {
        this.userService = userService;
    }
```

# IoC Container 기반 DI(Dependency Injection, 의존성 주입)

- 객체가 의존하는 다른 객체를 "직접 생성"하지 않고, "외부(컨테이너)"에서 주입 받는 방식
- 객체 사이의 결합도를 낮추고, 테스트나 변경이 용이해 짐

## Bean A는 Bean B를 사용하는 경우




Bean A는 Bean B가 구현체는 알지못하지만 사용

| DI 주입 방식 | 장점 | 단점 |
|---|---|---|
| 생성자 주입 **(권장)** | 불변 객체 설계, 테스트 용이, 필수 의존성 보장 | 코드 다소 길어질 수 있음 |
| 필드 주입 | 코드 간결, 간단 실습에 적합 | final 불가, 테스트 불편, 권장X |
| Setter 주입 | 선택적 의존성 주입에 적합 | setter 오남용 가능, 불변성 약함 |

# DI – 생성자 주입 (Constructor Injection)

- 주식(Stock) 정보를 조회하는 서비스(StockService)가 데이터 저장소(StockRepository)에 의존한다고 가정

```
@Service
public class StockService {
    private final StockRepository stockRepository;

    // 생성자 주입
    public StockService(StockRepository stockRepository) {
        this.stockRepository = stockRepository;
    }

    public String getStockInfo(String ticker) {
        return stockRepository.getStockInfo(ticker);
    }
}
```

### ❖ 생성자 주입에서 private final 선언의 필요성

- final로 선언하면 해당 필드는 초기화 이후 값이 변하지 않는 불변(immutable) 객체가 됨.
- 생성자를 통해 반드시 1번만 값이 할당되고, 이후 변경 불가 → 안정성과 신뢰성 확보
- Spring이 주입할 때도 반드시 생성자를 통해 값을 넣기 때문에, 필수 의존성이 반드시 주입 되었음을 컴파일 타임에 확인할 수 있음

# DI – 필드 주입 (Field Injection)

- 주식(Stock) 정보를 조회하는 서비스(StockService)가 데이터 저장소(StockRepository)에 의존한다고 가정

```
@Service
public class StockService {

    @Autowired
    private StockRepository stockRepository;

    public String getStockInfo(String ticker) {
        return stockRepository.getStockInfo(ticker);
    }
}
```

# DI – Setter 주입 (Setter Injection)

- 주식(Stock) 정보를 조회하는 서비스(StockService)가 데이터 저장소(StockRepository)에 의존한다고 가정

```
@Service
public class StockService {
    private StockRepository stockRepository;

    @Autowired
    public void setStockRepository(StockRepository stockRepository) {
        this.stockRepository = stockRepository;
    }

    public String getStockInfo(String ticker) {
        return stockRepository.getStockInfo(ticker);
    }
}
```

### ❖ Setter 주입에서도 @Autowired가 필요한 이유

- Setter 주입 방식에서는 객체 생성 후, Setter 메서드를 통해 Spring이 의존 객체를 "주입"
- 일반 자바 메서드는 Spring이 자동으로 호출해주지 않으므로,

  이 메서드가 DI(의존성 주입) 용도임을 Spring에게 명확히 알려야 함 → @Autowired로 표시

# [참고] 생성자 주입이 필드 주입보다 권장되는 이유

- 생성자 주입이 필드 주입보다 더 안전하고, 견고한 코드를 작성할 수 있게 해주기 때문에 생성자 주입을 권장

| 구분 | 생성자 주입 | Non-생성자 주입 |
|---|---|---|
| **불변성(Immutable) 보장** | final 키워드를 사용해서, 객체 생성 후 의존성 변경 불가<br>→ 안정성 증가 | Autowired, setter는 final 선언 불가<br>이로인한 의존성 변경 가능성 존재 |
| 의존성 주입 시점 | Bean 생성 단계 | BeanPostProcessor 단계 |
| 필수 의존성 보장 | 필수 의존성을 생성자 파라미터로 강제하여 누락 시 컴파일/런타임 에러<br>→ 오류 조기 발견 | 선택적 |
| 테스트 용이성 | 단위 테스트 시 생성자를 통해 mock/fake 객체 직접 주입 가능<br>→ 테스트 코드 작성 편리 | Autowired는 테스트 데이터 주입 용이하지 않음 |
| **순환 참조** 조기 감지 | 순환 참조 발생 시 애플리케이션 구동 시점에 즉시 에러 발생<br>→ 빠른 버그 확인 | Autowired, Setter는 늦게 감지 |
| 의존성 명확성 및 가독성 | 생성자 시그니처만 봐도 어떤 의존성이 필요한지 한눈에 파악<br>→ 코드 가독성 및 유지보수 용이 | 가독성 낮음 |

# [참고] 순환 참조 오류

- 생성자 주입은 객체를 생성하면서 필수로 모든 의존성을 동시에 주입해야 하므로, 서로 상대방을 필요로 하면 무한루프에 빠져 객체를 만들 수 없는 상태가 됨.
- 스프링은 이런 구조를 바로 감지해 시작 즉시 에러를 발생

```
***************************
APPLICATION FAILED TO START
***************************

Description:

The dependencies of some of the beans in the application context form a cycle:

   aService (field private final BService AService.bService)
      ↓
   bService (field private final AService BService.aService)
      ↓
   aService (field private final BService AService.bService)

Action:

There is a circular dependency between two beans.
Consider revising your code.
```

### ❖ 해결 방법

- 순환 참조는 “비즈니스 설계” 자체를 다시 점검해야 한다는 신호
- 의존 구조를 리팩토링 하거나, 중간 계층(인터페이스, 이벤트 등)으로 분리해서 해결



# Spring 컨테이너

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# Spring Container

자바 객체 (Bean)의 생성부터 소멸까지의 생명 주기를 관리하고,  객체 간 의존 관계를 설정

싱글톤 객체 생성, 목록화, 의존관계 연결 (생성자 함수에 넣어주기)

## 핵심 역할

- **객체 관리:**
  개발자가 new 연산자로 객체를 직접 생성하지 않고, 컨테이너가 객체를 대신 생성/소멸
- **의존성 주입 (DI):**
  객체 A가 객체 B를 필요로 할때, 컨테이너가 런타임에 B를 A에게 주입하여 조립
- **싱글톤 관리:**
  객체를 기본적으로 단 하나만 생성 (Singleton Pattern)하여 재사용

# IoC (Inversion Of Control) : 제어의 역전

- 객체의 생성, 생명주기 관리, 의존성 주입 등을 개발자가 직접 하지 않고, 컨테이너가 대신 관리해주는 원리
- 전통적 방식(개발자 직접 제어)와 반대 개념 : 제어권이 개발자에서 프레임워크로 넘어감
- 코드 결합도를 낮추고, 유연한 아키텍처와 테스트가 가능한 구조로 만들 수 있음

### 전통 방식

```
OrderService orderService = new OrderService(new OrderRepository());
```

### IoC 적용 방식

```
@Component
public class OrderService {
    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }
}
```

IoC Container는 @Compoment로 표시된 객체 생성 단계 (Bean) 에서 OrderRepository 객체를 주입하여
의존성 주입을 IoC Container가 실행

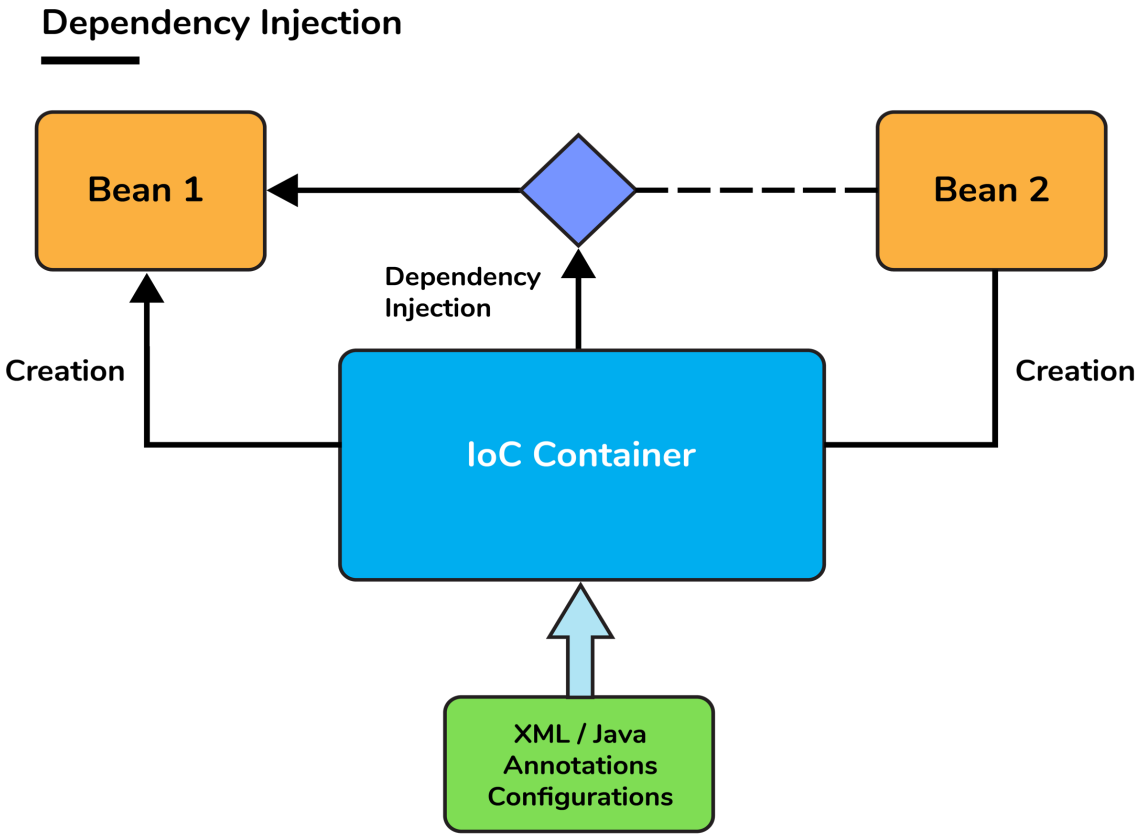
# IoC Container

## spring Container는 IoC 컨테이너 패턴을 적용한 컨테이너

- 애플리케이션에서 사용할 객체(빈, Bean)를 생성하고, 필요한 의존성을 주입하며, 객체의 생명주기를 관리하는 컨테이너
- 여기서 IoC는 프로그램의 흐름 제어 권한을 개발자가 아닌 프레임워크에게 넘긴다는 의미

### 역할

- 애플리케이션 컴포넌트(Bean)의 중앙 저장소
- Bean 소스부터 Bean을 생성 및 관리
- 의존성 주입을 통한 객체 주입 실행



# IoC 컨테이너 구현체

- 스프링 빈의 생성, 설정, 관리, 소멸 등 생명주기(Lifecycle) 전반을 담당하는 핵심 엔진
- IoC Container의 구현체가 ApplicationContext (이것은 BeanFactory 상속)

| 구분 | BeanFactory | ApplicationContext |
|---|---|---|
| 설명 | 가장 기본적인 컨테이너 | BeanFactory 확장판 |
| 주요 기능 | Bean의 생성/조회/관리 | BeanFactory의 모든 기능 + **부가기능** 지원 |
| Bean 로딩 방식 | 지연 로딩 (Lazy Loading) | 즉시 로딩 (Eager Loading) |
| Bean 저장 | ConcurrentHashMap<String, BeanDefition><br>ComponentScan 대상을 생성/등록 | BeanFactory 상속 |
| 부가 기능 | 미 지원 | 국제화, 이벤트 발행, AOP, Proxy Pattern, 메시지 자원, 환경설정 등 |
| 사용 예 | Spring 2.x 이전 사용 | Spring 2.x 이후부터 사용 |

# 빈(Bean)

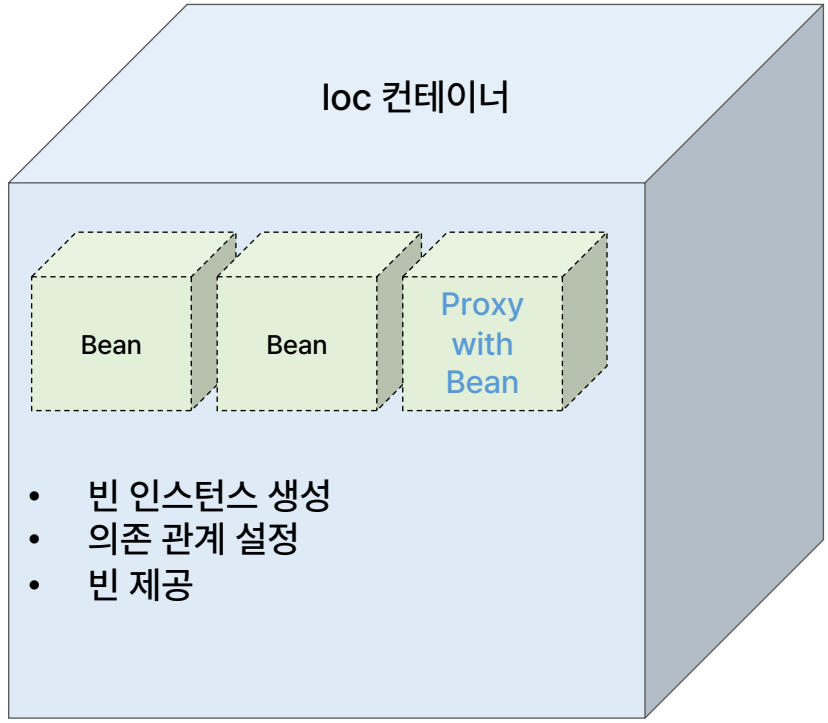
## ▪ 빈(Bean)이란?

- 스프링이 관리하는 객체(인스턴스)를 의미
- IoC Container가 Bean을 생성하고, 생명주기를 관리
- @Component, @Controller, @Service 등의 annotation을 통해  실행 시점에 Bean 자동 생성

## ▪ Bean의 특징

- **싱글턴(Singleton)** 방식으로 관리 (기본)
- 객체 생성, 의존성 주입, 소멸 등 생명주기를 컨테이너가 책임짐

IoC Container는 내부적으로 Map과 유사한 Collection을 가지고 있으며
여기에 Bean 을 등록하고 Bean 호출 시
생성이 아닌 기존 Bean 을 Input Argument로 전달



# 싱글턴(Singleton) 방식

- **싱글턴(Singleton) 패턴이란 프로그램 실행 내내 단 하나의 객체만 생성해서 공유하는 디자인 패턴**
- **Spring 컨테이너는 기본적으로 모든 Bean을 싱글턴으로 관리**
- **싱글턴 특징**
  - 메모리 절약: 객체를 한 번만 생성해서 여러 곳에서 같이 쓰면 메모리를 아낄 수 있음.
  - 일관된 데이터 관리: 모든 사용자가 같은 객체(=동일 인스턴스)를 사용하므로 데이터 일관성이 높아짐.
  - 객체 생성 비용 감소: 반복적인 객체 생성 작업을 줄일 수 있음.

## ❖ 스프링에서의 싱글턴 Bean

- @Component, @Service, @Repository, @Controller 형태로 **어노테이션** 으로 등록된 Bean을 컨테이너당 한 개만 생성 관리.

# [참고] POJO 클래스

**Spring bean은 기본적으로 POJO 기반으로 구성되어, 특정 프레임워크에 종속되지 않도록 POJO 기반 구성 지향**

**Spring이 POJO를 관리하는 방식**은 단순한 POJO 클래스와는 약간의 차이가 존재.
Spring은 POJO 객체를 **IoC 컨테이너(WebApplicationContext)에 등록하여 관리**하고,
이를 통해 **의존성 주입(DI) 및 AOP 기능을 추가**

### 일반적인 POJO 클래스

```
public class UserService {
    public void getUserInfo() {
        System.out.println("Fetching user information...");
    }
}
```

### @Component를 이용한 자동 Bean 등록

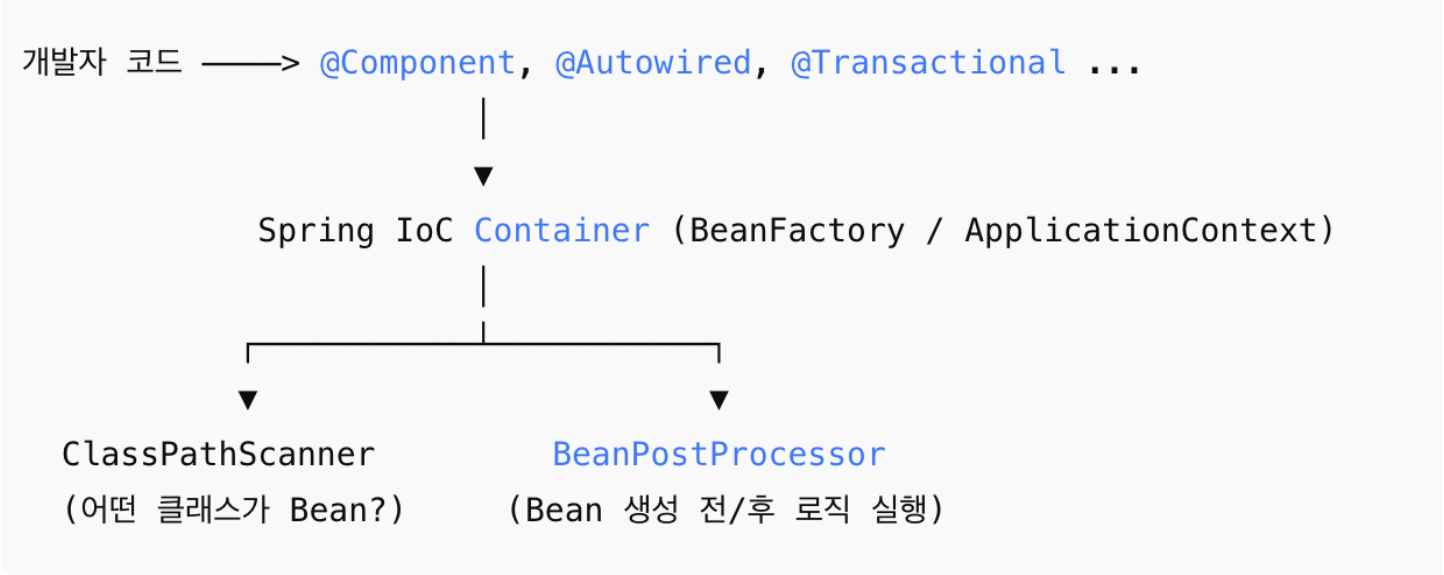
```
import org.springframework.stereotype.Component;

@Component
public class UserService {
    public void getUserInfo() {
        System.out.println("Fetching user information...");
    }
}
```

# 어노테이션(Annotation)을 이용한 DI 대상 식별

## @Component, @Controller, @Service, ... 적용을 통한 의존성 주입

- **설정 및 의존성 주입의 간소화:**
  복잡한 설정 없이 객체(Bean)를 자동으로 등록하거나 관리하고, 의존성 주입을 간편하게 처리




- 패키지 내부 탐색, Bean 대상 검색
- 스캔된 Bean들이 실제 객체로 생성되는 과정에서 생성 후 특별한 로직을 실행 (의존성 주입, Proxy 객체 바꿔치기 )

# [참고] Bean 생성 클래스 정의 Annotation

| 어노테이션 | 용도/설명 |
|---|---|
| @Component | 스프링이 자동으로 Bean으로 등록해줌 |
| @Service | 비즈니스 로직을 담은 Service 클래스에 사용 |
| @Repository | 데이터 접근 계층(DAO)에 사용, 데이터 예외 변환 처리 |
| @Controller | MVC 패턴에서 Controller 역할, 웹 요청 처리 |
| @RestController | RESTful API Controller (JSON 반환) |
| @Autowired | 필요한 의존성을 자동 주입 (DI) |
| @Bean | 메서드의 반환 객체를 직접 Bean으로 등록 |



# IoC Container & DI 이해 실습

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# [실습] IoC container 구조를 이해해보자

05.simple-ioc-container

**Spring의 IoC(Inversion of Control) 컨테이너가 내부적으로 어떻게 동작하는지를 순수 Java 리플렉션만으로 재현하여 핵심 개념을 학습한다.**

- 개발자가 직접 new로 객체를 생성하지 않고 컨테이너가 대신 생성·주입한다는 제어의 역전(IoC) 원리를 이해
- 생성자 기반 의존성 주입(Constructor Injection)이 어떻게 이루어지는지 코드 수준에서 파악
- @Service 같은 커스텀 어노테이션을 런타임에 읽는 방법(@Retention(RUNTIME) + 리플렉션)을 이해

```
06.simple-ioc-container/
├── README.md
└── src/
    └── com/
        └── sk/
            └── skala/
                └── ioc/
                    ├── Main.java                          ← 진입점 (컨테이너 사용 예시)
                    ├── annotation/
                    │   └── Service.java                   ← 커스텀 @Service 어노테이션
                    ├── container/
                    │   └── SimpleIoCContainer.java        ← IoC 컨테이너 핵심 구현
                    └── service/
                        ├── DatabaseService.java           ← 최하위 서비스 (의존성 없음)
                        ├── UserService.java               ← DatabaseService 에 의존
                        └── OrderService.java              ← UserService 에 의존 (최상위)
```

# [실습] IoC container를 실행하기

05.simple-ioc-container

Spring의 IoC(Inversion of Control) 컨테이너가 내부적으로 어떻게 동작하는지를
순수 Java 리플렉션만으로 재현하여 핵심 개념을 학습한다.

## 1. 컴파일하기

```
# 프로젝트 루트로 이동
cd 06.simple-ioc-container

# bin 디렉토리에 컴파일
javac -d bin $(find src -name "*.java")
```

## 2. 실행해보기

```
java -cp bin com.sk.skala.ioc.Main
```

# [코드 검토] IoC container 코드 검토



개별적으로 먼저 확인후에 샘플 코드를 함께 리뷰해보자



# [실습] TestService를 @Service로 등록해보기

@Service annotation을 가지는 TestService.java를 추가하고 실행해보자

## 1. service/TestService.java

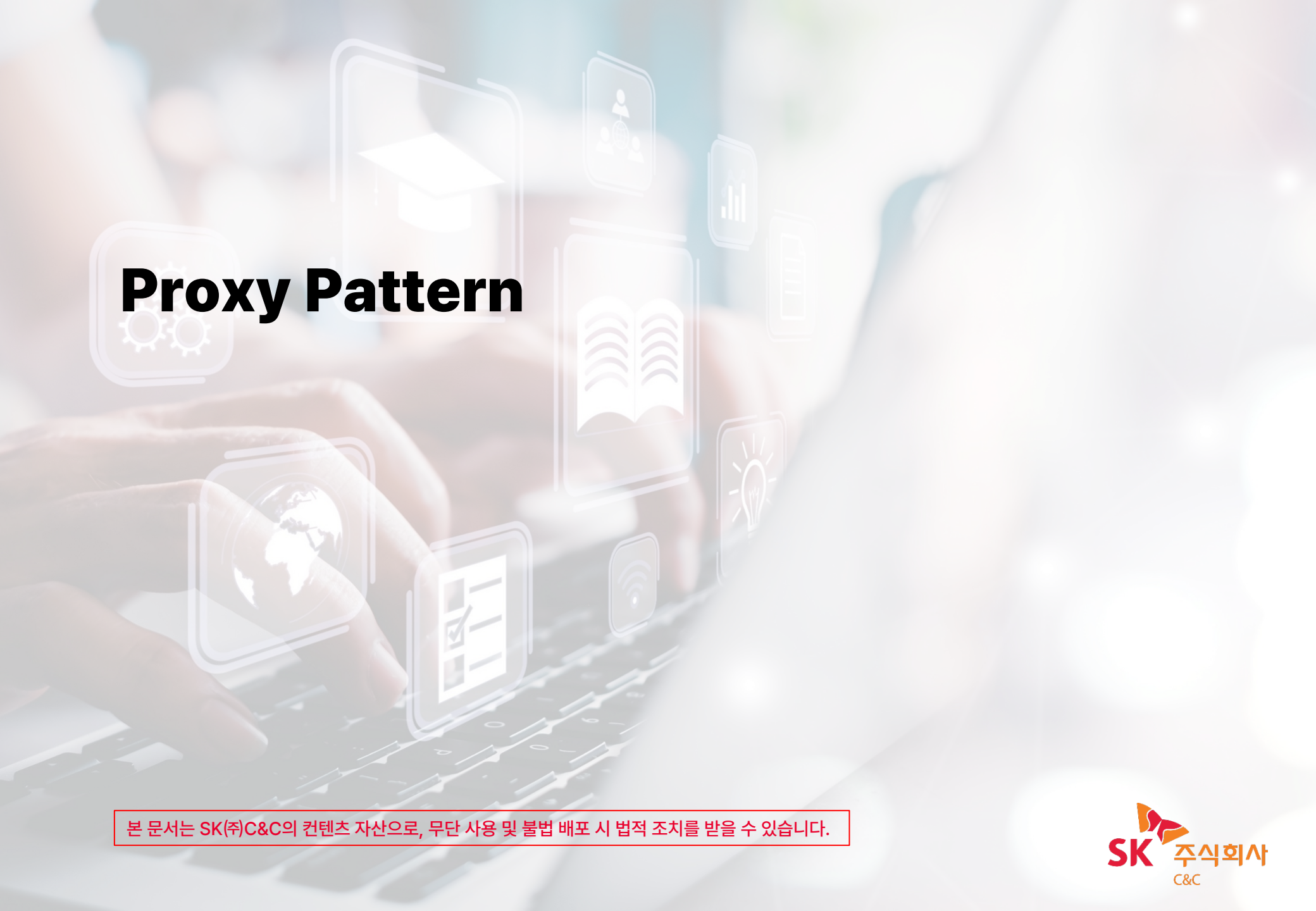
```
@Service
public class TestService {
        private final UserService userService;

        public TestService(UserService userService) {
                this.userService = userService;
        }

        public String runTest(String name) {
                return userService.getUser(name);
        }
}
```

## 2. Main.java 에 추가

```
 TestService testService = container.getBean(TestService.class);
        String testResult = testService.runTest("홍길동");
        System.out.println("  " + testResult);
```



# Proxy Pattern

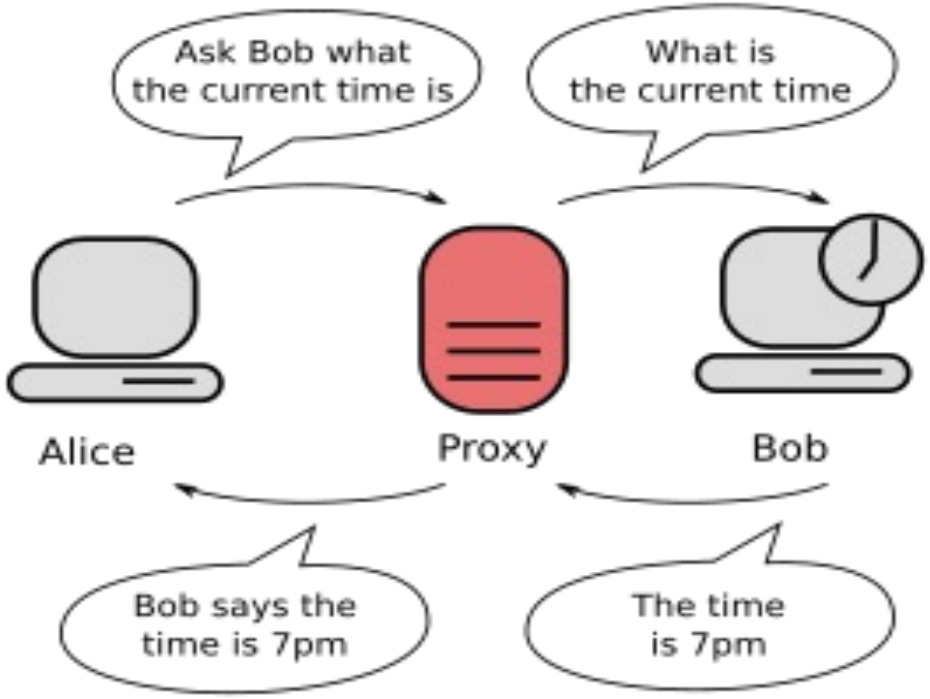
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

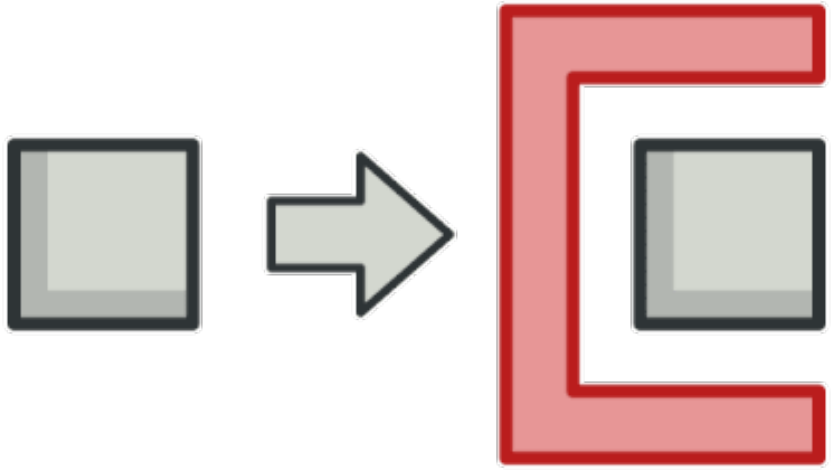
SK 주식회사 C&C

# Proxy 패턴 정의

Proxy라는 용어는 원래 법률 및 정치 분야에서 **대리인**이라는 개념에서 유래

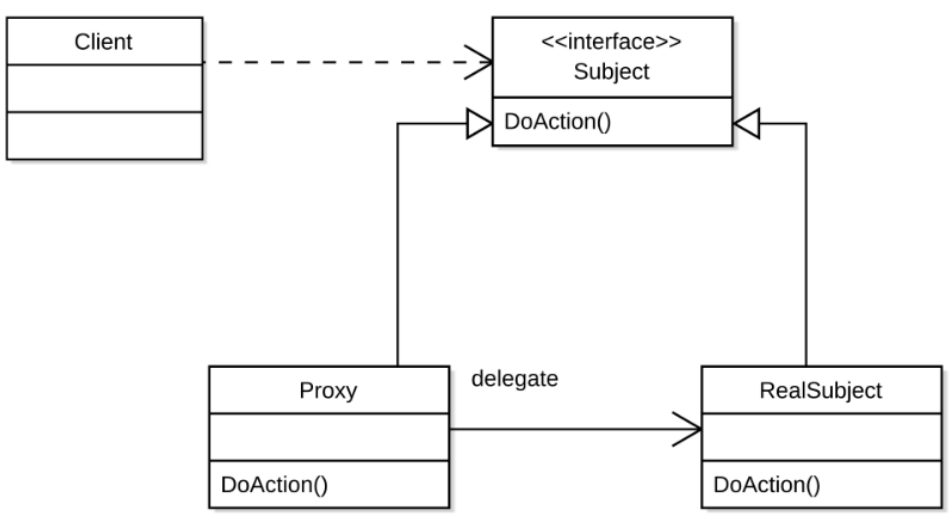
- Proxy Vote(대리 투표): 어떤 사람이 직접 투표할 수 없을 때, 다른 사람이 대신 투표하는 것
- Proxy Server(프록시 서버): 사용자의 요청을 대신해서 처리하는 서버

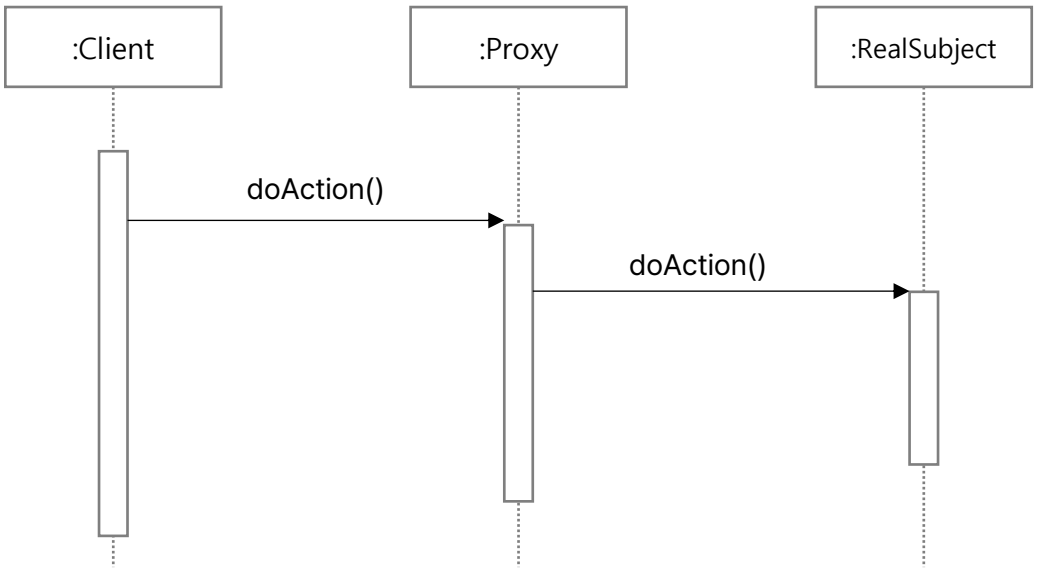


# Proxy 패턴

Proxy Pattern은 동일한 인터페이스를 가직 구현 클래스는 Client에서 사용할때 Overriding으로 동일한 이터페이스에 다른 기능을 정의하여 사용



# Proxy 패턴 예시코드

Proxy Class는 RealSubject를 Wrapping 해서 내부에 감싸는 방식으로 지정하고 Proxy Class의 operation()은 realSubject.operation() 호출 전/후에 추가적인 작업을 수행할 수 있도록 지원하는 구조

```
// 클라이언트 코드
public class ProxyPatternExample {
    public static void main(String[] args) {
        Subject proxy = new Proxy();  // Proxy가 RealSubject를 호출
        String name= ”honggildong”;
        proxy.operation(name);
    }
}
```

```
// Subject 인터페이스
interface Subject {
    String operation(String name);
}
```

```
// Proxy: RealSubject에 대한 접근을 제어하는 클래스
class Proxy implements Subject {
    private RealSubject realSubject;

    @Override
    public void operation(String name) {
        if (realSubject == null) {
            realSubject = new RealSubject();
        }
        //pre-processing 처리
        String fullName = “pre:” + name;
        fullName = realSubject.operation(fullName);
        //post-processing 처리
        fullName = fullName + “:post”;
    }
}
```

delegate

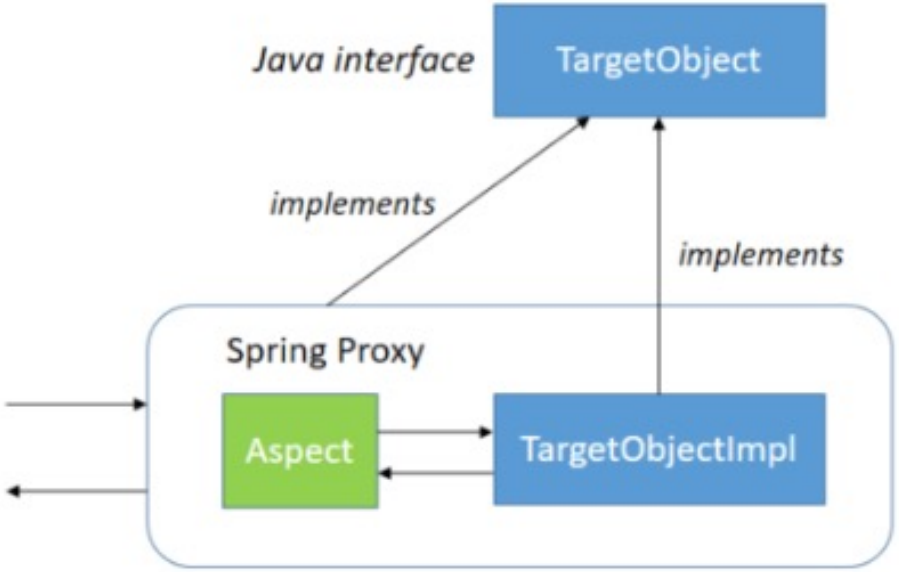
```
// RealSubject: 실제 기능을 수행하는 클래스
class RealSubject implements Subject {
    @Override
    public String operation() {
        System.out.println("RealSubject: 작업 수행");
    }
}
```

# Spring Proxy 구현 방식

동적 프록시 (Dynamic Proxy)를 통해 개발자가 작성한 코드를 직접 수정하지 않고 기능 확장

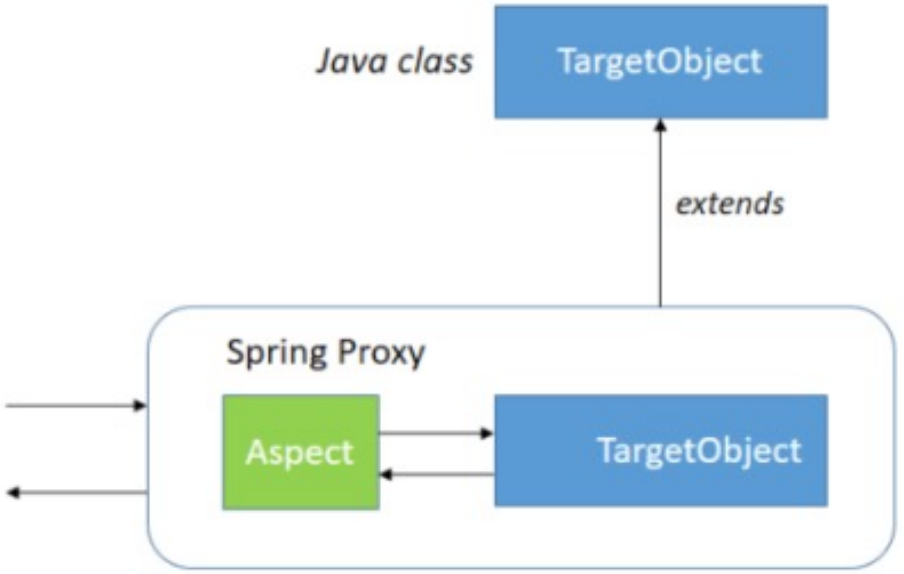
## JDK Proxy (Interface based)

- 인터페이스 기반으로 동작
- java.lang.reflect.Proxy 클래스를 사용
- 인터페이스가 반드시 필요
- Reflection을 활용




## CGLIB(Code Gen Lib) Proxy (class based)

- 바이트코드를 조작하여 프록시 생성
- 인터페이스가 없어도 사용 가능
- **서브클래스(Proxy Class)를 생성**하여 원래 클래스의 메서드를 오버라이드
- 직접 코드 생성 (Reflection 사용하지 않음)




성능이 JDK Proxy 대비 뛰어남 (Non-Reflection)

# CGLIB Proxy 메커니즘을 사용하여 동작하는 기술들

### Custom AOP (Aspect Oriented Programming)

@Aspect

### 내장 선언적 기능

### @Transactional (스프링 트랜잭션 관리)

### @Cacheable / @CacheEvict (스프링 캐시 추상화)

```
@Cacheable(value = "products", key = "#productId")
public ProductDto getProductById(Long productId) {
    return productRepository.findById(productId).orElseThrow();
}
```

### @Async (비동기 메서드 실행)

### @Validated (메서드 레벨 검증)

# 수동으로 Proxy Bean 생성 방법: @Configuration 과 @Bean

- Configuration라고 정의된 클래스 내에서 Bean을 수동으로 생성 가능
- 내부적으로  IoC Container에 Bean 등록을 요청 (위임)

MyService는 @Service로 Annotation을 포함하지 않았지만 수동으로 MyService를 Bean으로 생성 가능

```
@Service ——// 제거해서 자동 Bean 생성하지 않기
public class UserService {
    private final UserRepository userRepository;

    public MyService UserService(UserRepository userRespository) {    // 의존성 주입
        return new UserService(userRepository);
    }
}
```

```
@Configuration
public class AppConfig {

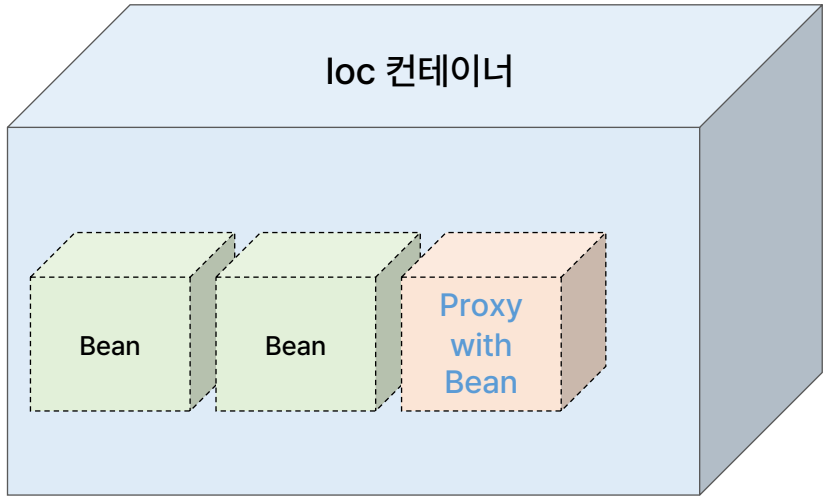
    @Bean
    public MyService UserService(UserRepository userRespository) {    // 의존성 주입
        return new UserService(userRepository);
    }
}
```

# Proxy with Bean 만들기 (3/3)

- @Configuration을 이용한 Bean 생성
  UserService를 UserServiceProxy를 wrapping 후 Bean으로 등록

```
@Configuration
public class AppConfig {

    @Bean
    public UserService userService(UserRepository userRepository) {
        // 실제 대상 생성
        UserService target = new UserServiceImpl(userRepository);
        // 프록시로 감싸서 반환
        return new UserServiceProxy(target);
    }
}
```



# [실습] Configuration 기반 Proxy 적용하기

myapp spring boot 코드에서 UserService의 모든 method를 호출 시 마다 아래와 같이 호출 전후에 시간을 프린트하고 호출되는 총 소요시간을 화면에 프린트하도록 UserService를 위한 Proxy를 구현해보자
출력 모습은 다음과 같다.

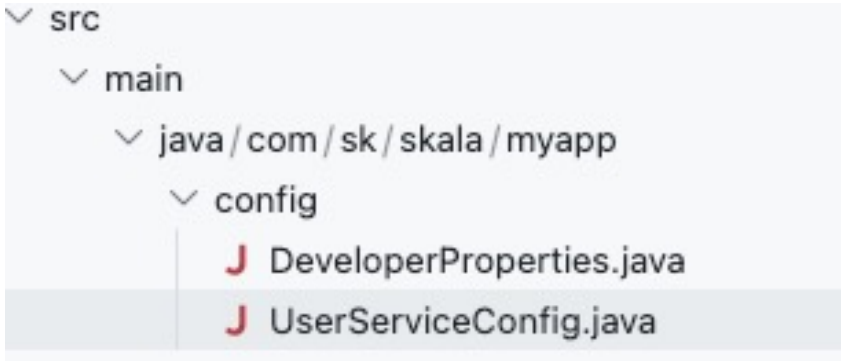
**출력 예시**
[Proxy] getAllUsers 메소드 시작: 14:23:05.123
## 메소드 호출
[Proxy] getAllUsers 메소드 종료: 14:23:05.135 | 총 소요 시간: 12 ms

## 1. service/UserService.java에서 @Service를 comment out 한다.

```
//@Service
public class UserService {
…
```

# [실습] Configuration 기반 Proxy 적용하기

**2. 01.training-code/07.proxy 내 config/UserServiceConfig.java 코드를 자신의 config/ 아래에 복사**




**3. Run and Debug 실행한다.**

**4. podman 또는 browser 로 접속하기**

http://localhost:8080/api/users

**4. console log를 확인해보자**

```
[Proxy] getAllUsers 메소드 시작: 17:51:11.974

[Proxy] getAllUsers 메소드 종료: 17:51:12.134 | 총 소요 시간: 160 ms
```



# Aspect Oriented Programming

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# Java의 AOP(Aspect-Oriented Programming)

- **AOP(관점 지향 프로그래밍)는 OOP(객체지향 프로그래밍)의 한계를 보완하는 프로그래밍 패러다임**
  - OOP는 주로 '기능' 단위로 클래스를 구현
  - AOP는 '공통 관심사(Aspect)'를 모듈화 하여 코드 중복을 줄이고 유지보수가 쉽도록 구현
- **대표적인 공통 관심사(Aspect) 예시**
  - 로깅(Logging)
  - 트랜잭션 처리(Transaction)
  - 보안(Security)
  - 성능 측정(Profiling)




*https://codegym.cc/groups/posts/543-what-is-aop-principles-of-aspect-oriented-programming*

# Spring Boot의 AOP

- **별도 설정 없이 spring-boot-starter-aop 의존성만 추가하면 바로 사용**
- **선언적 AOP 지원**: @Aspect 어노테이션 기반으로 Aspect 클래스를 작성
- pom.xml에 의존관계 주입은 다음과 같이

```
<!-- Spring Boot Starter AOP -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

# AOP 핵심 구성요소

AOP 적용을 위한 AOP 클래스는 다음과 같이 구성

```
@Aspect
@Component
public class LoggingAspect {

    // 포인트컷 정의
    @Pointcut("execution(* com.example.service.*.*(..))")
    public void serviceMethods() {}

    // Before 어드바이스
    @Before("serviceMethods()")
    public void beforeAdvice(JoinPoint joinPoint) {
        System.out.println("[Before] " + joinPoint.getSignature());
    }

    // After 어드바이스 - 동일한 포인트컷 재사용
    @After("serviceMethods()")
    public void afterAdvice(JoinPoint joinPoint) {
        System.out.println("[After] " + joinPoint.getSignature());
    }

    // Around 어드바이스 - 동일한 포인트컷 재사용
    @Around("serviceMethods()")
    public Object aroundAdvice(ProceedingJoinPoint joinPoint){
        System.out.println("[Around] 메소드 실행 전");
        Object result = joinPoint.proceed();
        System.out.println("[Around] 메소드 실행 후");
        return result;
    }
}
```

**Aspect**

AOP 적용 클래스라고 명시

Bean 객체 생성 시 AOP Proxy 생성 및 wrapping

**Bean Component**

AOP 설정 클래스는 반드시 Bean 객체로 정의 되어야 함

**Pointcut**

어떤 Method를 대상으로 하는지 정의

com.example.service 패키지 내 모든 서비스(*),

모든 method(*) 에 적용

**Advice**

메소드 호출 시 횡단 관심 기능을 어느 지점에서 실행할 지를 결정 . @Before는 Method 호출전 먼저 실행

**JoinPoint**

Method 정보를 담고 있는 객체들의 역할

**Target**

Advice가 적용되는 실제 비즈니스 객체 (원본 빈)

Proxy가 감싸고 있는 원본 인스턴스를 의미

# [참고] JoinPoint Method

**1) joinPoint.getSignature() ➜ 메서드 시그니처 정보**

현재 어드바이스가 걸린 "메서드"의 서명 정보(이름, 반환타입, 파라미터 타입 등)

**2) joinPoint.getArgs() ➜ 호출 시 전달된 인자 값 배열**

- 실제 호출 시 넘어온 값들(런타임 값).
- 예시: updateMember(7, "bob") 호출 → joinPoint.getArgs() 는 [7, "bob"]

**3) joinPoint.getTarget() ➜ 실제 타겟(원본) 객체**

- 프록시가 감싸고 있는 원본 Bean Instance
- 예시 : joinPoint.getTarget().getClass().getName() → com.example.service.MemberService

**4) joinPoint.getThis() ➜ 프록시 객체**

- 현재 호출을 처리 중인 프록시 인스턴스(가장 바깥쪽 프록시).
- 예시 : CGLIB 프록시: com.example.service.MemberService$$SpringCGLIB$$a1b2c3

  JDK 동적 프록시: jdk.proxy2.$Proxy38

# Pointcut 표현식: execution

메서드 실행 (join point)을 메서드 시그니처 패턴으로 매칭하는 포인트 컷

```
                반환값                                  클래스(타입)          파라미터
execution(* com.example.domain.*Service.find*(..))
지시자          패키지                                  메서드
```

- **execution(* com.example.user.*.*(..))**
  user 패키지 아래의 모드  클래스의 전체 메소드를 대상 이며, 모든 반환값 유형 (void, String, int ...)
- **execution(* com.example.user.UserService.*(..))**
  UserService  클래스의 전체 메소드를 대상
- **execution(* com.example.user.UserService.find*(..))**
  UserService  클래스에서 find 로 시작하는 메소드를 대상
- **execution(String  com.example.user.UserService.*(..))**
  UserService  클래스에서 반환값이 String  인 메소드를 대상

- **와일드 카드 (*, ..) 사용 가능**

✓ * :  하나의 패키지 또는 클래스 이름 매칭

✓ .. : 하위 패키지 포함

# Pointcut 표현식: within

특정 타입(클래스 또는 패키지) **내부의 모든 메서드를 포인트컷**으로 지정할 때 사용합

```
within(com.example.domain..*)
```

- 지시자
- 기준 패키지
- 하위 패키지 포함
- 모든 클래스

- **와일드 카드 (*, ..) 사용 가능**

✓ * : 하나의 패키지 또는 클래스 이름 매칭

✓ .. : 하위 패키지 포함

# Pointcut 표현식: annotation

특정 Annotation이 붙어 있는 메서드를 포인트컷으로 지정할 때 사용

```
@Pointcut("@annotation(com.sk.skala.myapp.aop.Metrics)")
public void metricsAnnotation() {}
```

- com.sk.skala.myapp.aop.Metrics → annotation 타입

**@Aspect 적용 방식**

```
@Around("metricsAnnotation()")
public Object measureExecutionTime(ProceedingJoinPoint joinPoint) {…}
```

# Pointcut 표현식

스프링 IoC 컨테이너에 등록된 빈(Bean)의 이름(ID)을 직접 지정하여 포인트컷을 정의하는 지시자

```
@Pointcut("bean(orderService)")
public void targetOrderService() {}
```

- orderService → 스프링 빈 이름 (Bean name)

```
@Pointcut("bean(*Service)")
public void targetAllService() {}
```

- *Service → 빈 이름 패턴 (접미사 Service)

**@Aspect 적용 방식**

```
@Around("targetAllServices()")
public Object logExecution(ProceedingJoinPoint joinPoint) {…}
```

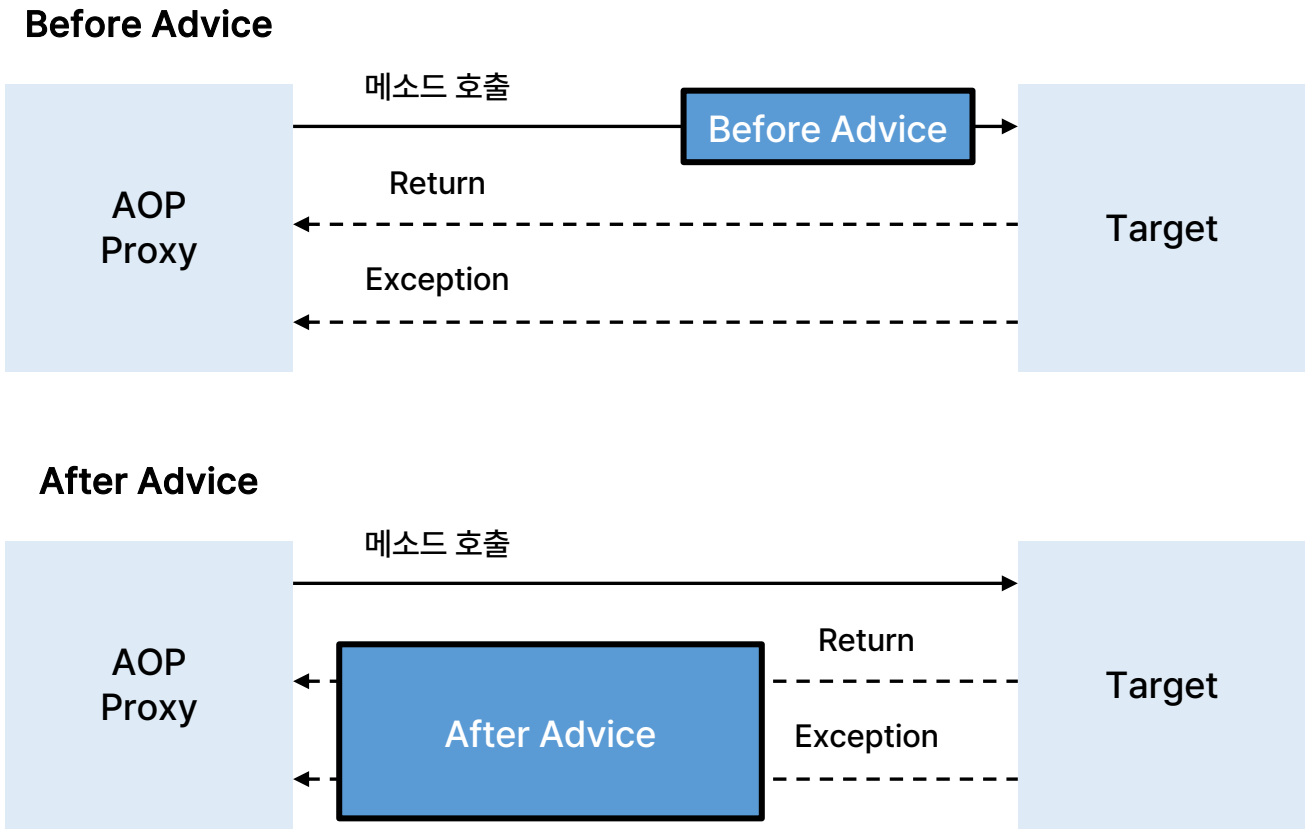
# Spring Boot의 Advice Type

- **선언적 AOP 지원**: @Aspect 어노테이션 기반으로 Aspect 작성, Pointcut, Advice(@Before, @After, @Around 등) 선언

| 어노테이션 | 역할 |
|---|---|
| @Before | 메서드 실행 전에 Advice 실행 |
| @After | 메서드 실행 후에 Advice 실행 (정상/예외 상관없이 항상 실행) |
| @AfterReturning | 메서드 정상 실행 후 Advice 실행 |
| @AfterThrowing | 예외 발생 후 Advice 실행 |
| @Around | 메서드를 감싸서 실행 전후 모두 Advice 실행되도록 설정 |

# Advice 유형별 동작 방식

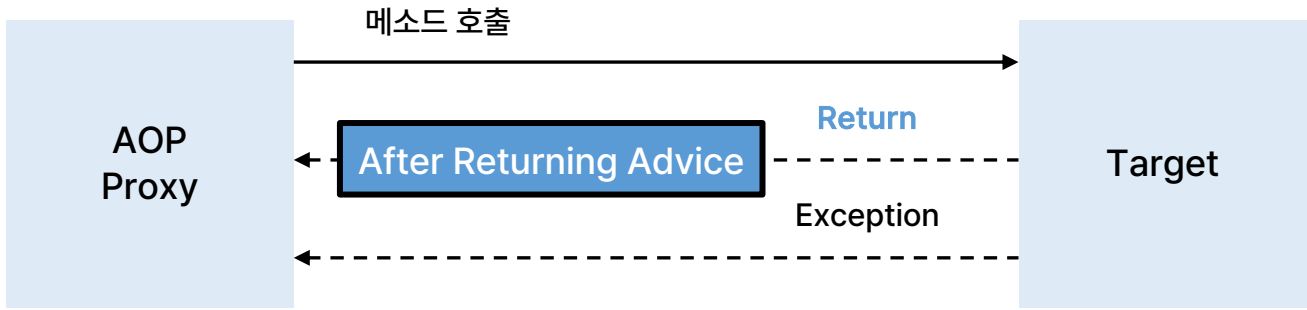
스프링에서는 총 5가지의 Advice 유형이 존재함



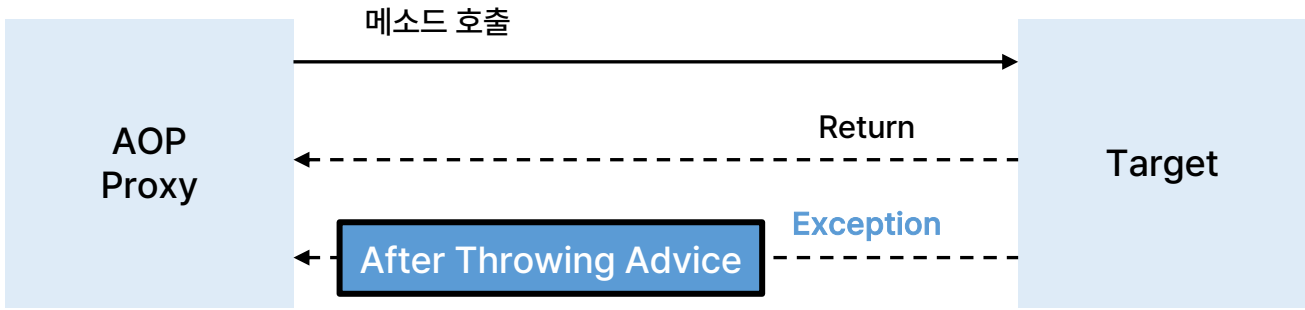
# Advice 유형별 동작 방식

스프링에서는 총 5가지의 Advice 유형이 존재함

### After Returning Advice

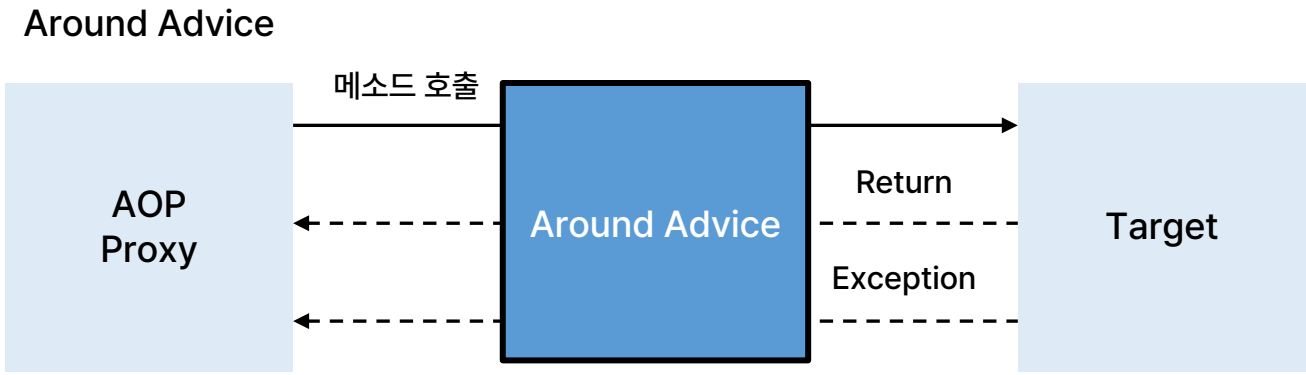



### After Throwing Advice



# Advice 유형별 동작 방식

스프링에서는 총 5가지의 Advice 유형이 존재함



# [참고] @Around 구현 예시

- 메서드 실행 전/후, 예외 발생 여부 등 "전체 실행 흐름"을 모두 제어

```
@Aspect
@Component
public class LoggingAspect {
    @Around("execution(* com.example.service.*.*(..))")
    public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
        System.out.println("메서드 실행 전");
        Object result = joinPoint.proceed(); // 실제 메서드 실행
        System.out.println("메서드 실행 후");
        return result;
    }
}
```

- Target Method의 Input parameter 를 변경하고 싶은 경우

```
Object[] args = jointPoint.getArgs();

Long userId = (Long) args[0];
String item = (String) args[1];
Integer qty = (Integer) args[2]; // int는 오토박싱되어 들어옴

// 인자 수정: 공백 제거 + 수량 상한
args[1] = item.trim();
args[2] = Math.min(qty, 99);

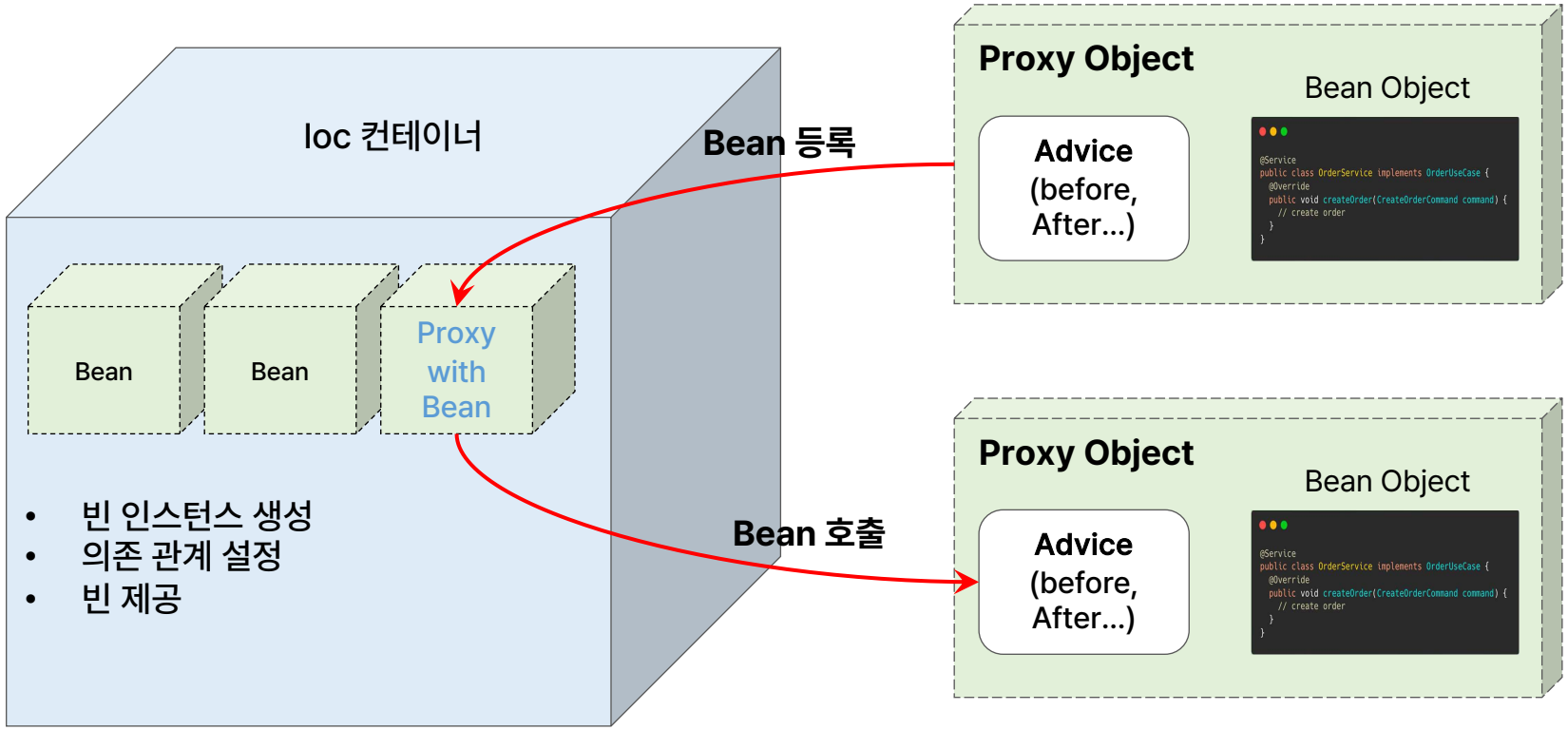
Object result = jointPoint.proceed(args);   // 수정된 인자 전달

return result;
```

# Bean 과 AOP

**Aspect Class 기반으로 Target이 된 @Component, @Service, @Component 등의 클래스는 Bean 으로 IoC Container를 통해 생성 후 BeanFactory에 등록 시**

1) Bean 객체 생성 후 AOP Proxy 객체로 Wrapping (Proxy 기반 Bean 으로 등록)
2) @Controller 등의 Bean 호출 시 Proxy Obejct를 호출 (Advice 실행 전/후 Bean Method 호출)

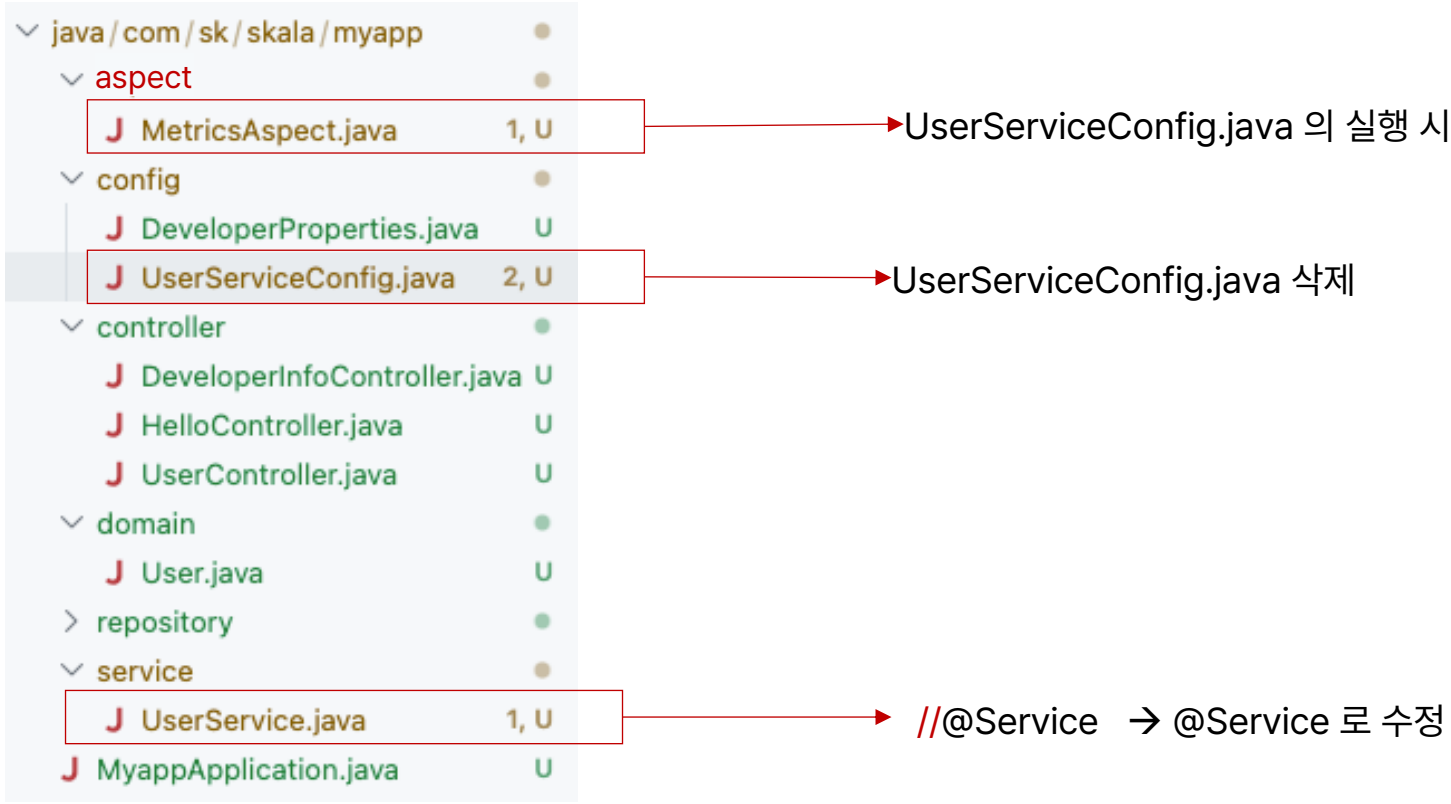


# [실습 1] @Aspect 적용하기

08.aop

07.proxy 과정을 통해 추가했던 config/UserServiceConfig.java 의 메소드 호출 전/후 시각 및 처리 소요 시간을 표준 출력한 것을 AOP (@Aspect 와 @Around) 로 변환해보자

1. config/UserServiceConfig.java에서 @Configuration annotation을 comment out한다 (또는 제거)
2. service/UserService.java에서 @Service annotation을 활성화한다. ( comment out 제거)
3. aspect package를 추가하고 MetricsAspect.java를 생성한다.
4. @Aspect 와 @Around를 지원하기 위한 MetricsAspect.java를 생성한다



# [실습 1] @Aspect 적용하기

aspect/MetricsAspect.java 코드

```
@Slf4j
@Aspect
@Component
public class MetricsAspect {

    private static final DateTimeFormatter TIME_FORMATTER =
            DateTimeFormatter.ofPattern("HH:mm:ss.SSS");

    @Around("execution(* com.sk.skala.myapp.service.UserService.*(..))")
    public Object measureExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        String methodName = joinPoint.getSignature().getName();

        LocalDateTime startTime = LocalDateTime.now();
        long startMillis = System.currentTimeMillis();

        log.info("[AOP Around] {} 메소드 시작: {}", methodName, startTime.format(TIME_FORMATTER));
        try {
            return joinPoint.proceed(); // 실제 메소드 실행
        } finally {
            long elapsed = System.currentTimeMillis() - startMillis;
            LocalDateTime endTime = LocalDateTime.now();
            log.info("[AOP Around] {} 메소드 종료: {} ¦ 총 소요 시간: {} ms",
                    methodName, endTime.format(TIME_FORMATTER), elapsed);
        }
    }
```

# [실습 1] @Aspect 적용하기

빌드 후 실행해보자. Apsect 가 정확하게 적용되는지 검토해보자

1. 먼저 Run and Debug 해보자

2. http://localhost:8080/api/users 접속하고 console log를 확인해보자

```
7-29T22:39:47.078+09:00  INFO 94315 --- [myapp] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
7-29T22:39:47.078+09:00  INFO 94315 --- [myapp] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet         : Initializing Servlet 'dispatcherServlet'
7-29T22:39:47.079+09:00  INFO 94315 --- [myapp] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet         : Completed initialization in 1 ms
7-29T22:39:47.127+09:00  INFO 94315 --- [myapp] [nio-8080-exec-1] c.sk.skala.myapp.aspect.MetricsAspect     : [AOP] getAllUsers 메소드 시작: 22:39:47.127
ate: select u1_0.id,u1_0.email,u1_0.name from users u1_0
7-29T22:39:47.242+09:00  INFO 94315 --- [myapp] [nio-8080-exec-1] c.sk.skala.myapp.aspect.MetricsAspect     : [AOP] getAllUsers 메소드 종료: 22:39:47.242 | 총 소요 시간: 115 ms
```

# [실습 2] Annotation AOP 적용하기



[실습 2] MetricsAspect.java에 @Before를 적용해보자

아래의 @Before 를 MetricsAspect.java에 추가한다

```
    @Before("execution(* com.sk.skala.myapp.service.UserService.*(..))")
    public void logControllerStart(JoinPoint joinPoint) {
        String targetName = joinPoint.getTarget().getClass().getSimpleName();
        String methodName = joinPoint.getSignature().getName();
        log.info("[AOP before] @Metrics 시작 - 대상: {}, 메소드: {}", targetName, methodName);
    }
```

1. 먼저 Run and Debug 해보자

2. http://localhost:8080/api/users 접속하고 console log를 확인해보자

```
-07-29T22:51:46.504+09:00  INFO 4394 --- [myapp] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
-07-29T22:51:46.504+09:00  INFO 4394 --- [myapp] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
-07-29T22:51:46.506+09:00  INFO 4394 --- [myapp] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms
-07-29T22:51:46.559+09:00  INFO 4394 --- [myapp] [nio-8080-exec-1] c.sk.skala.myapp.aspect.MetricsAspect    : [AOP Around] getAllUsers 메소드 시작 : 22:51:46.559
-07-29T22:51:46.560+09:00  INFO 4394 --- [myapp] [nio-8080-exec-1] c.sk.skala.myapp.aspect.MetricsAspect    : [AOP before] @Metrics 시작 - 대상 : UserService, 메소드 : getAllUsers
rnate: select u1_0.id,u1_0.email,u1_0.name from users u1_0
-07-29T22:51:46.685+09:00  INFO 4394 --- [myapp] [nio-8080-exec-1] c.sk.skala.myapp.aspect.MetricsAspect    : [AOP Around] getAllUsers 메소드 종료 : 22:51:46.685 | 총 소요 시간 : 126 ms
```

# [실습] Annotation을 이용한 AOP 구성하기

- **Metrics Annotation 생성하기**

**apsect/Metrics.java**

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Metrics {
}
```

- **MetricsAspect.java의 기능을 MetricsAnnotationAspect.java로 만들기**

**apsect/MetricsAnnotationAspect.java**

```
public class MetricsAnnotationAspect {

    private static final DateTimeFormatter TIME_FORMATTER =
            DateTimeFormatter.ofPattern("HH:mm:ss.SSS");

    @Pointcut("@annotation(com.sk.skala.myapp.aspect.Metrics)")
    public void metricsAnnotation() {}


    @Before("metricsAnnotation()")
    public void logControllerStart(JoinPoint joinPoint) {}

    @Around("metricsAnnotation()")
    public Object measureExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {}
```

# [실습] Annotation을 이용한 AOP 구성하기

애플리케이션이 실행되는 중에 **Annotation이 붙은 메서드나 클래스를 감지하여 앞/뒤로 로직을 수행**

- **구현 위치**: @Aspect가 붙은 클래스 (Aspect Class)
- **사용 기술**: Spring AOP (Proxy 패턴), Java Reflection

**UserController.java**

```
@Metrics
@GetMapping("/users/{id}")
public User getUserById(@PathVariable long id) {
    return userService.getUserById(id).orElse(null);
}
```



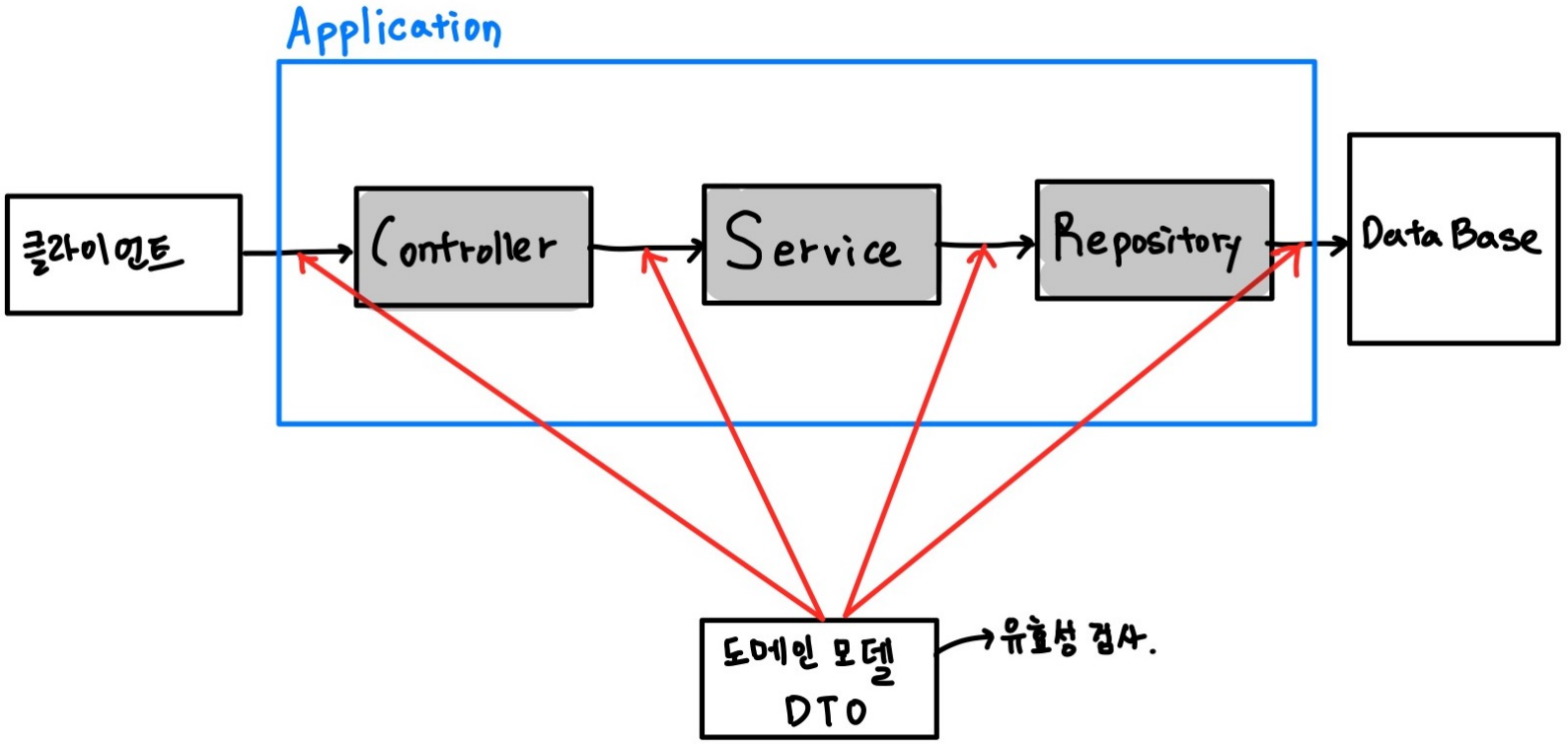
# 입력값 검증

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# 입력값 검증이란?

- 클라이언트(사용자)로부터 받은 데이터가 사전에 정의된 규칙에 맞는지 확인하는 과정
- 검증 과정 없이 잘못된 데이터가 시스템으로 들어오면, 예상치 못한 오류 또는 데이터베이스에 원치 않는 값이 저장될 수 있음
- 입력값 검증 과정을 통해 애플리케이션의 안정성과 데이터의 신뢰성을 확보



# 의존성 추가하기

의존성 추가 설정

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

# DTO (Data Transfer Object)

- 소프트웨어 아키텍처에서 계층 간 데이터 전달만을 담당하는 순수한 자료 구조
- 비즈니스 로직은 포함하지 않고 데이터 속성(Field)와 제어 메서드 (Getter 생성자)만 존재
- ↔ 서비스 ↔ 컨트롤러 ↔ 외부 API/클라이언트 사이에서 전달 형식을 안정적으로 유지

**DTO를 쓰는 이유**

- 엔티티(Entity) 보호 및 보안: 전달해야 하는 데이터만 선별해서 전달 가능
- 클라이언트 요구 사항과 도메인 로직과 격리
- 데이터 전송 도중 변하지 않는 불변성 권장 (Java 14 부터 record 지원)
- 네트워크 오버헤드 감소 (원격 호출 시)

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class UserResponse {
    private Long id;
    private String name;
    private String email;

    // 정적 팩토리 메소드로 Entity → DTO 변환
    public static UserResponse from(User user) {
        return new UserResponse(
            user.getId(),
            user.getName(),
            user.getEmail()
        );
    }
}
```

# [참고] DTO 작성 시 주의점

- **Don't Include Business Logic (비즈니스 로직을 포함하지 마시오)**
  - DTO는 순수하게 데이터를 전달하는 "데이터 상자" 역할만 해야 함.
  - 계산, 로직 처리, 데이터베이스 접근 등의 코드는 서비스(Service)나 도메인(Domain) 계층에 위치
  - 나쁜 예: DTO 내부에 할인율을 계산하는 메서드를 만드는 것.

- **Don't Use Entities as DTOs (JPA 엔티티를 DTO로 사용하지 마시오) – 귀차니즘**
  - 엔티티는 데이터베이스 테이블과 직접 매핑되는 핵심 도메인 객체
  - 엔티티를 클라이언트에 직접 노출하면 원치 않는 필드(비밀번호 등)가 외부에 노출될 수 있음
  - UI 요구사항이 변경될 때마다 데이터베이스 설계에 영향을 주게 되어 유연성이 떨어짐.
  - 양방향 연관관계가 설정된 엔티티를 직렬화할 때 무한 루프에 빠져 StackOverflowError가 발생할 수 있음

- **Name and Separate Appropriately (목적에 맞는 네이밍과 분리) – 귀차니즘**
  - DTO의 역할을 이름에 명시하면 가독성이 높아짐.
  - 요청(Request)용 DTO: UserCreateRequest, ProductUpdateRequest
  - 응답(Response)용 DTO: UserResponse, OrderDetailResponse
  - 하나의 DTO를 요청과 응답에 모두 사용하려고 하면, 용도가 다른 필드가 섞여 혼란을 줄 수 있음.

- **Consider Serialization (직렬화를 고려하시오)**
  - DTO는 네트워크를 통해 JSON 등의 형태로 변환되어 전송되는 경우가 많음
  - Jackson과 같은 라이브러리가 객체를 JSON으로 변환할 때 **기본 생성자나 Getter**가 없으면 오류 발생 가능 → Lombok으로 해결

- **Validation annotation belongs in the DTO (입력값 검증 어노테이션은 DTO에서)**
  - 클라이언트로부터 들어오는 데이터에 대한 검증은 DTO의 필드에 @NotNull, @Size, @Email 등의 어노테이션을 붙여 처리
  - 유효하지 않은 데이터가 서비스 계층으로 넘어가지 않도록 사전 차단

# @Valid 검증 적용

- **DTO + @Valid** → IoC 컨테이너가 아니라 **Spring MVC의 바인딩 과정**에서 검증

```
// DTO
public class UserRequest {
    @NotBlank
    private String name;

    @Email
    private String email;

    @Min(18)
    private int age;
}
```

```
// 컨트롤러에서 @Valid 사용 검증
@PostMapping("/users")
public ResponseEntity<?> createUser(@Valid @RequestBody UserRequest request) {
    // 유효성 검증 통과 시 로직 진행
    return ResponseEntity.ok("가입 완료");
}
```

❖ 검증 실패 시 **400 Bad Request**로 예외 발생

참고: RequestMappingHandlerAdaptor내 HandlerMethodArgumentResolvoer가 Body/Parameter를 DTO로 바인딩 수행

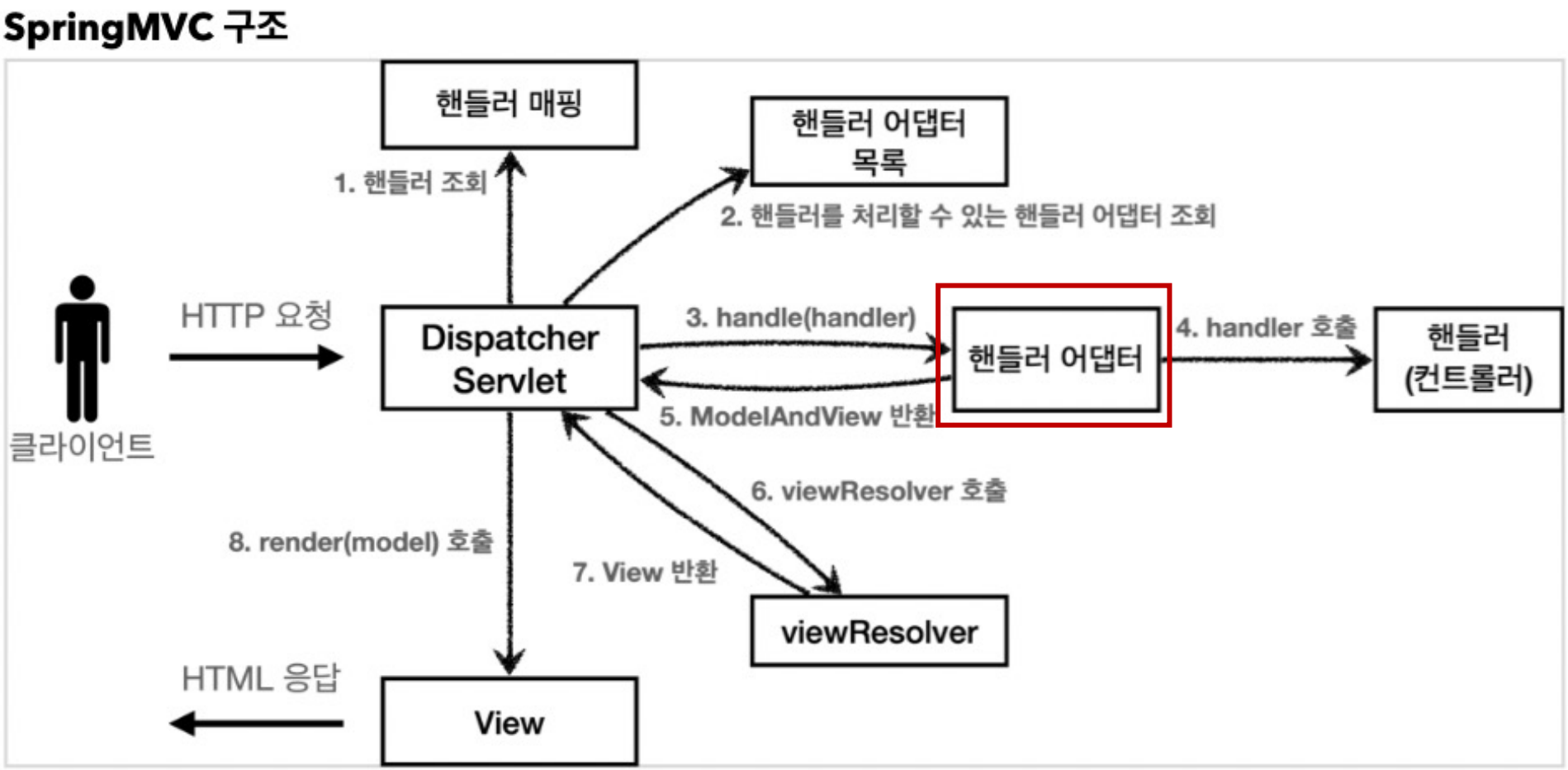
# 입력값 검증 어노테이션

- 개별 데이터에 설정하고 @Valid 또는 @Validated를 통해 검증

| 어노테이션 | 설명 | 예시 |
|---|---|---|
| @NotNull | Null이면 안됨 | 필수 입력값 |
| @NotEmpty | 개수가 0이면 안됨 | String, Array, Collection 대상 |
| @NotBlank | Null, 0, 스페이스 모두 안됨 | String 전용 |
| @Size | 문자열, 컬렉션, 배열의 길이/크기 제한 | @Size(min=2, max=20) |
| @Min | 최소값(숫자) | @Min(18) |
| @Max | 최대값(숫자) | @Max(100) |
| @Positive | 양수여야 함 | 1, 2, ... |
| @PositiveOrZero | 0 또는 양수여야 함 | 0, 1, 2, ... |
| @Negative | 음수여야 함 | -1, -2, ... |
| @NegativeOrZero | 0 또는 음수여야 함 | 0, -1, -2, ... |
| @Email | 이메일 형식 | @Email |
| @Pattern | 정규표현식 패턴 일치 | @Pattern(regexp="^[0-9]+$") |
| @Past | 과거 날짜여야 함 | 생년월일 등 |
| @PastOrPresent | 과거나 오늘 날짜 | |
| @Future | 미래 날짜여야 함 | 예약일 등 |
| @FutureOrPresent | 오늘 또는 미래 날짜여야 함 | |
| @Digits | 자릿수 및 소수점 자리 제한 | @Digits(integer=5, fraction=2) |
| @AssertTrue | 반드시 true | 체크박스 등 |
| @AssertFalse | 반드시 false | |
| @Null | null이어야 함 | |

# [참고] spring MVC 와 @Valid 적용

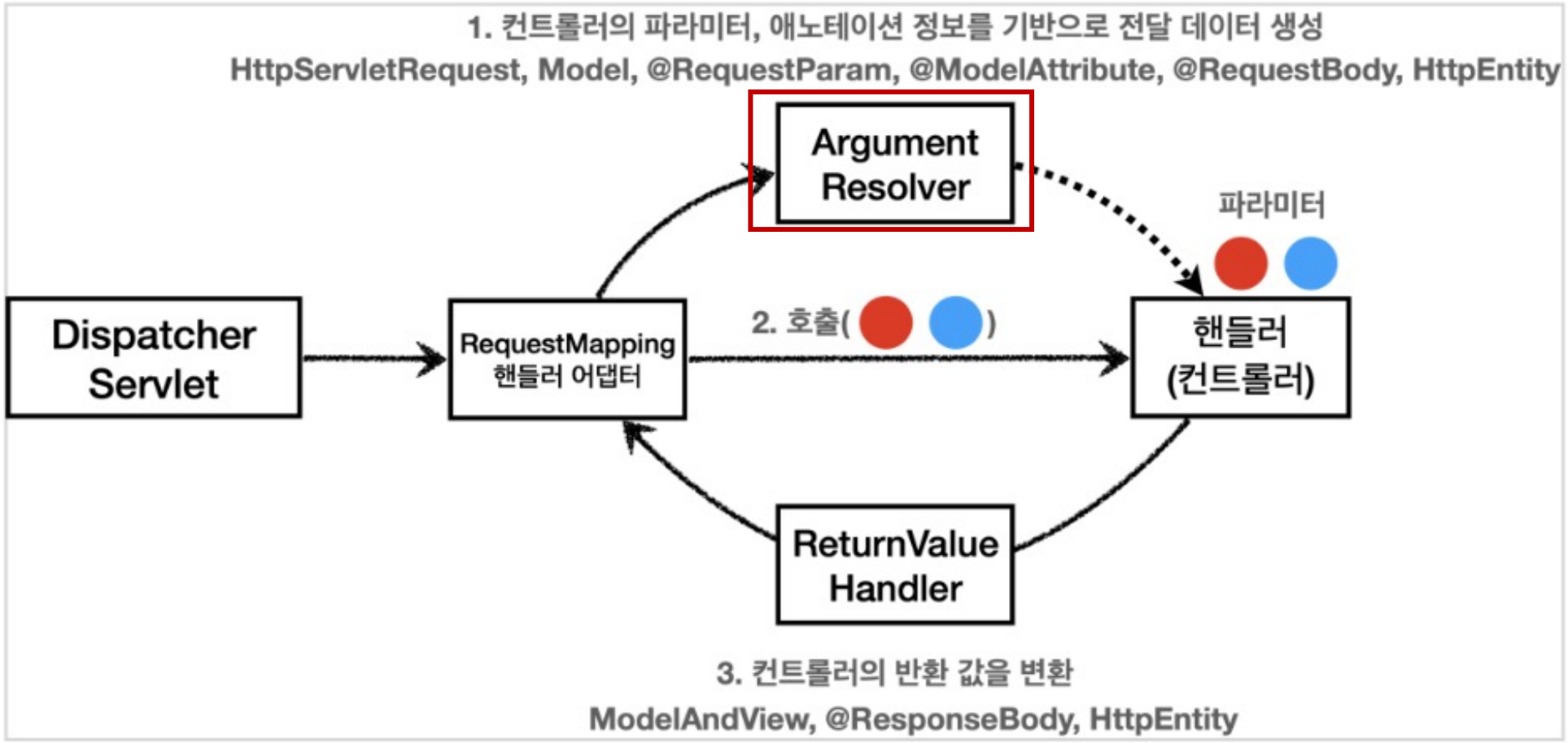
**RequestMappingHandlerAdaptor에서 @Valid 처리를 수행**



# [참고] spring MVC 와 @Valid 적용

**@Valid는 ArgumentResolver 과정에서 DTO request의 Validation 처리 수행**

**RequestMappingHandlerAdapter 동작 방식**




**MethodArgumentNotValidException 발생**

# Controller외 Bean에서 Valid 적용 방법

- @Valid는 **Spring MVC의 메서드 파라미터 바인딩 과정**에서 동작
- Controller의 메서드 파라미터는 HTTP 요청 데이터 → 객체 변환 → 검증 흐름을 거치는데,
  이 변환과 검증 과정은 HandlerMethodArgumentResolver와 Validator가 담당합니다.
- @Service의 메서드 파라미터는 IoC 컨테이너가 생성자나 메서드 호출로 직접 주입하는 객체이기 때문에,
  Spring MVC의 요청 바인딩 + 검증 흐름이 아예 적용되지 않습니다.

### 1) Controller에서 @Valid 동작

```
HTTP 요청 → DispatcherServlet → HandlerMapping → HandlerAdapter
→ Argument Resolver (데이터 바인딩 + @Valid 검증) → Controller 메서드 실행
```

### 2) Service에서 @Valid 불가능한 이유

```
Controller → (이미 DI로 주입된) Service Bean 메서드 호출
→ 이 시점에는 Argument Resolver가 아님 → @Valid 자동 검증 안 함
```

### 3) Service에서 @Validated 적용으로 AOP Proxy 로 동작하도록 구성해서 Service에서 @Valid 사용 지원

# 입력값 검증 - @Valid vs @Validated

**@Valid : Java 표준 어노테이션**

- Controller DTO 검증시

**@Validated : Spring 제공 확장 검증 어노테이션**

- Controller의 단일 파라메터에 @NotEmpty, @Min 같은 제약을 검증 시 Controller Class에 @Valiated 설정 필요
- Spring AOP를 통한 Service/component 메서드 호출 시 검증

❖ **검증 실패 시 발생하는 예외 (@Valid와 @Validated 모두 동일)**

| 어노테이션 적용 위치 | 발생 예외 |
|---|---|
| @RequestBody | MethodArgumentNotValidException |
| @ModelAttribute | BindException |
| @RequestParam | ConstraintViolationException |

# 서비스에 @Validated 검증 적용

**Service 에 @Validated 적용 → AOP Proxy 기반으로 동작**

```
@Service
@Validated // Method Validation AOP Proxy 활성화
public class UserService {

    // DTO 검증
    public void createUser(@Valid UserDto user) {
        System.out.println("유저 생성: " + user.getName());
    }

    // 단일 파라미터 검증
    public void deleteUser(@Min(value = 1, message = "ID는 1 이상이어야 합니다.") Long id) {
        System.out.println("유저 삭제: " + id);
    }
}
```

**동작 방식**

1. Spring IoC Container가 UserService 빈을 생성 시 @Validated 감지.
2. MethodValidationPostProcessor가 해당 빈을 AOP 프록시로 감싼후 이것을 Bean으로 등록
3. Controller가 AOP Proxy로 감싸진 Service의 메서드를 호출하면:
   - AOP 프록시가 메서드 호출을 가로챔
   - Hibernate Validator로 파라미터 제약 검사
   - 검증 실패 시 ConstraintViolationException 발생
4. 검증 성공 시 원래 메서드 실행

# Controller에 @Validated 검증 적용

- **단일 파라미터 + @Validated** → Spring AOP(Method Validation)로 검증

```
@RestController
@RequestMapping("/users")
@Validated // Method Validation AOP 활성화
public class UserController {

    @GetMapping("/search")
    public String search(@NotEmpty String name) {
        return "검색: " + name;
    }
}
```

**동작 방식**

1. Spring IoC Container가 UserController 빈을 생성 시 @Validated 감지.
2. MethodValidationPostProcessor가 해당 빈을 AOP 프록시로 감싼후 이것을 Bean으로 등록
3. IoC Container가 AOP Proxy로 감싸진 Controller의 메서드를 호출하면:
   - AOP 프록시가 메서드 호출을 가로챔
   - Hibernate Validator로 파라미터 제약 검사
   - 검증 실패 시 ConstraintViolationException 발생
4. 검증 성공 시 원래 메서드 실행

# [실습] DTO UserRequest /UserResponse 만들기

기존 @Entity  User 클래스에 대해 UserRequest / UserResponse DTO를 작성하자

DTO는 @lombak 을 사용해도 되고 record를 사용해도 됩니다. 단, @lombak 적용 시 setter 는 제외해야 합니다



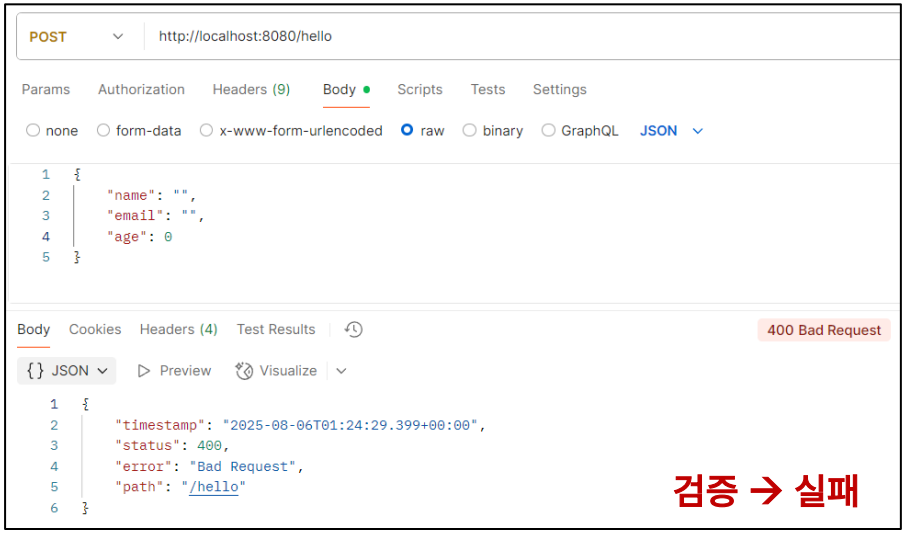

```
User Request 는
email 과 name field 를 가지고 있다
id field는 DB 저장시에 생성되므로 필요없다.
```

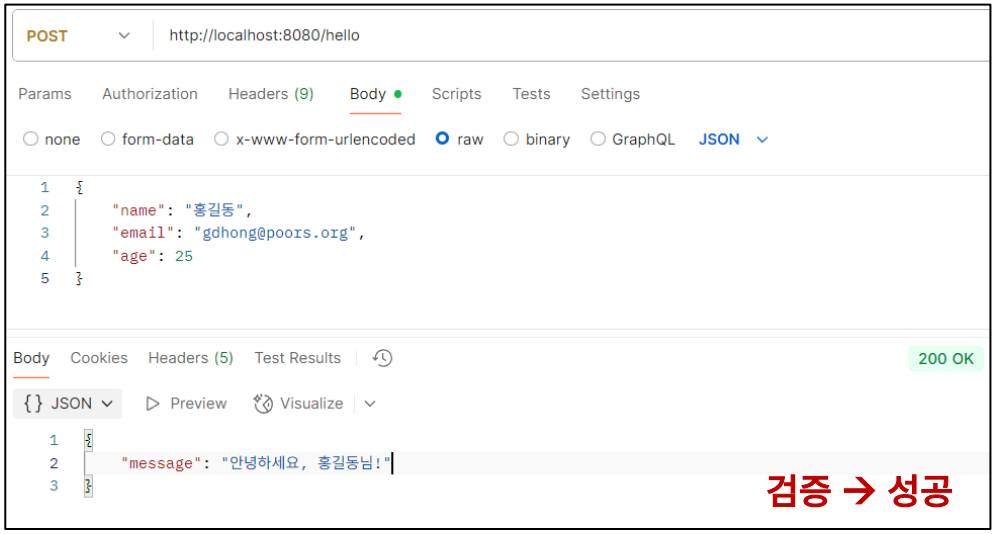
```
User Response 는
id, name, email field를 가지고 있다.
```

# [실습] DTO UserRequest /UserResponse

1. @Valid를 DTO에 적용합니다.
   - id : @Positive(message = "ID는 양수여야 합니다")
   - name: @NotBlank(message = "이름은 필수입니다")
   - email: @NotBlank(message = "이메일은 필수입니다"),
     @Email(message = "올바른 이메일 형식이 아닙니다")
2. UserService.java에 createUser, updateUser 에 @Valid를 적용합니다.
3. postman을 통해 post 요청 처리하고 그 결과 확인해보자

# [코드] 입력값 검증 요청 결과



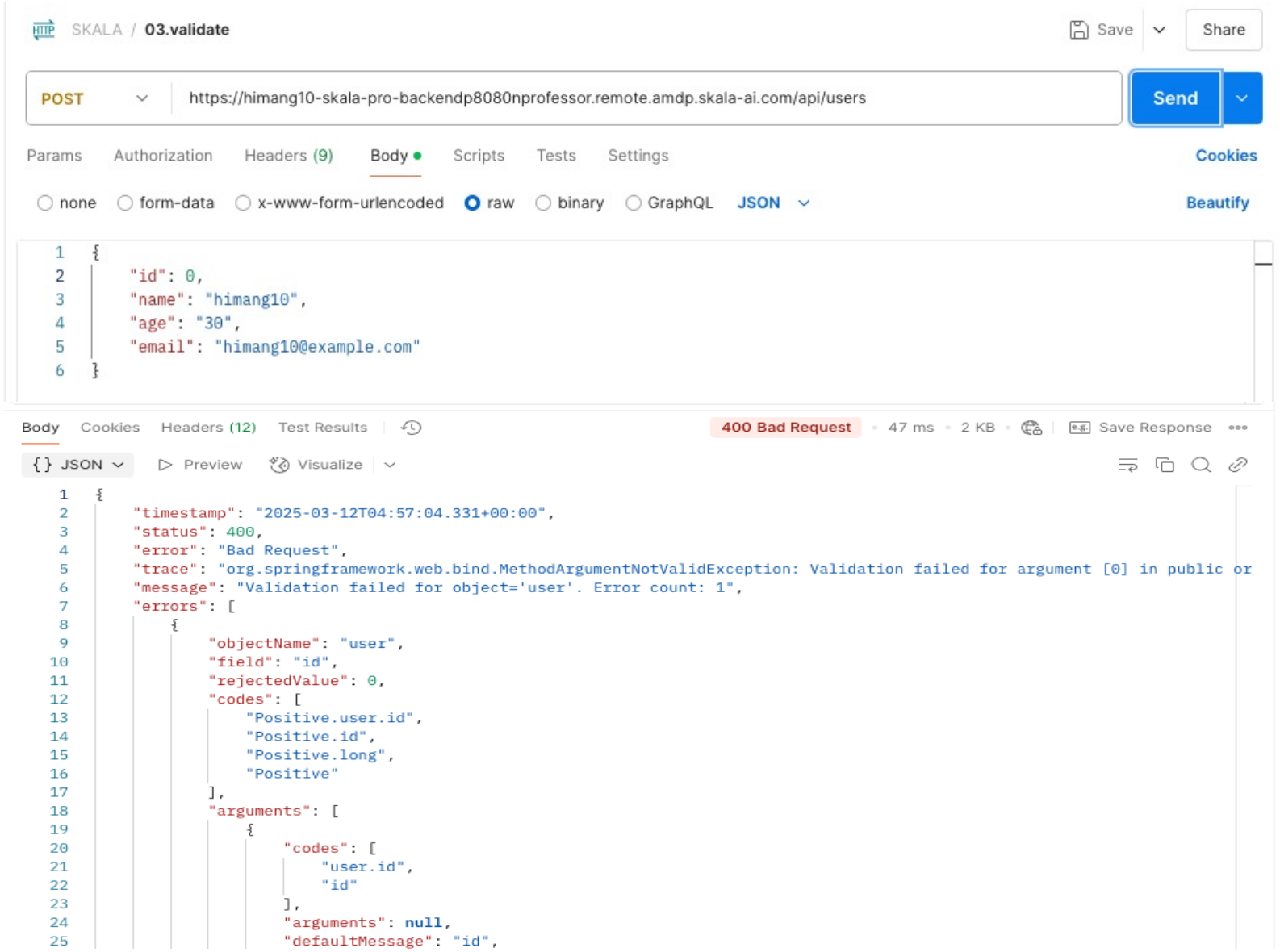




```
# 검증 결과 로그

Validation failed for argument [0] in public com.skala.basic.data.HelloResponse
com.skala.basic.controller.HelloController.postHello(com.skala.basic.data.HelloRequest) with 2 errors: [Field error
in object 'helloRequest' on field 'name': rejected value []; codes
[NotBlank.helloRequest.name,NotBlank.name,NotBlank.java.lang.String,NotBlank]; arguments
[org.springframework.context.support.DefaultMessageSourceResolvable: codes [helloRequest.name,name]; arguments [];
default message [name]]; default message [공백일 수 없습니다]] [Field error in object 'helloRequest' on field 'age':
rejected value [0]; codes [Min.helloRequest.age,Min.age,Min.int,Min]; arguments
[org.springframework.context.support.DefaultMessageSourceResolvable: codes [helloRequest.age,age]; arguments [];
default message [age],18]; default message [18 이상이어야 합니다]] ]
```

❖ 검증 실패 시 **400 Bad Request 대신** 검증 조건을 전달할 수 없을까?

# [참고] Valid 적용


SKALA / 03.validate
Save
Share
POST
https://himang10-skala-pro-backendp8080nprofessor.remote.amdp.skala-ai.com/api/users
Send
Params
Authorization
Headers (9)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"id": 0,
"name": "himang10",
"age": "30",
"email": "himang10@example.com"
}
Body
Cookies
Headers (12)
Test Results
400 Bad Request
47 ms
2 KB
Save Response
{ } JSON
Preview
Visualize
{
"timestamp": "2025-03-12T04:57:04.331+00:00",
"status": 400,
"error": "Bad Request",
"trace": "org.springframework.web.bind.MethodArgumentNotValidException: Validation failed for argument [0] in public or
"message": "Validation failed for object='user'. Error count: 1",
"errors": [
{
"objectName": "user",
"field": "id",
"rejectedValue": 0,
"codes": [
"Positive.user.id",
"Positive.id",
"Positive.long",
"Positive"
],
"arguments": [
{
"codes": [
"user.id",
"id"
],
"arguments": null,
"defaultMessage": "id",



# Async

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# Async란?

- 메서드를 호출한 스레드와 분리하여 비동기(Async)로 실행하도록 지시하는 Spring 기능 Annotation
- 호출자는 즉시 반환하고, 실제 실행은 별도 스레드 풀에서 처리되도록 구성

**특징**

- 긴 작업(메일 전송, 파일 처리, 외부 API 호출 등)을 메인 흐름에서 분리하여 성능 향상
- 웹 요청 처리 시간을 단축하고, 시스템의 병렬 처리 능력 향상
- 비동기 로직을 직접 스레드 생성 없이 Annotation만으로 쉽게 구현

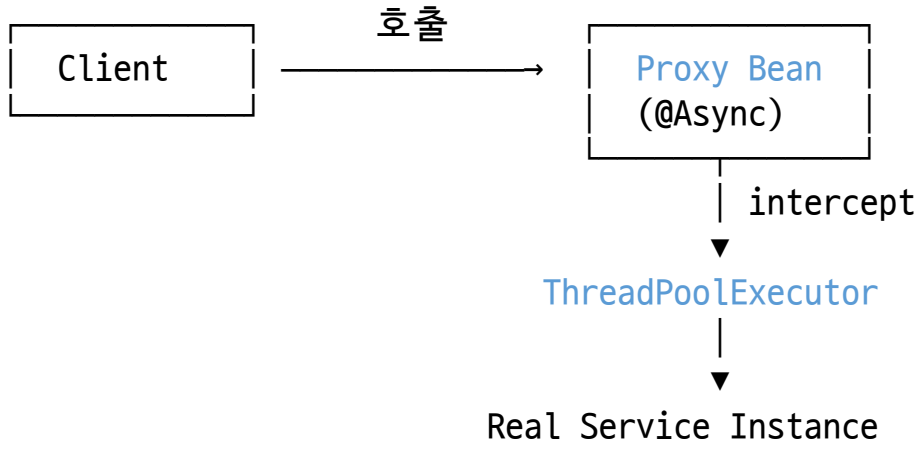
| 구분 | 동기 방식 | 비동기 방식 |
|---|---|---|
| 실행 방식 | 한 작업 끝나야 다음 작업 수행 | 여러 작업 병렬 수행 |
| 응답 | 처리 끝까지 대기 | 즉시 응답 |
| 스레드 | 동일 스레드 | 별도 스레드 풀 |
| 사용 예 | 단순 로직 | I/O, 외부 연동, 시간이 긴 작업 |

# Async 동작 원리

Spring은 Bean을 Proxy로 감싸 비동기 실행을 구현

**동작 순서**

① ComponentScan → Bean 등록
② @EnableAsync Bean → AsyncAnnotationBeanPostProcessor 등록
③ AsyncAnnotationBeanPostProcessor가 모든 Bean을 확인
**④ @Async 메서드가 존재하는 Bean 발견**
⑤ 해당 Bean을 **AOP Proxy Bean**으로 교체
⑥ 메서드 호출 시 프록시가 가로채서 ThreadPoolExecutor에 위임
⑦ 호출자는 즉시 응답



# Async 활성화 방법: 디폴트 TaskExecutor

- @Async는 기본적으로 동작하지 않음
- Spring에서 비동기 기능을 활성화하기 위해 아래 설정이 반드시 필요

```
@Configuration
@EnableAsync
public class AsyncConfig {
}
```

@EnableAsync → Spring이 AsyncAnnotationBeanPostProcessor를 등록하여
@Bean 중 @Async 메서드를 자동 감지하고 **프록시 생성**

### SimpleAsyncTaskExecutor 사용

- 기본 executor 사용: Spring의 기본 SimpleAsyncTaskExecutor 사용
- 스레드 풀 없음: 매번 새로운 스레드 생성 (성능 저하 가능)
- 설정 불가: 스레드 풀 크기, 큐 등을 제어할 수 없음

예시

```
@Async
public void asyncMethodWithoutReturn(String message) { ... }
```

# Async 활성화 방법: TaskExecutor 지정

- 프로덕션 환경, 많은 비동기 작업 필요시 Task Executor 지정

```
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "taskExecutor")
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(5);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
        executor.initialize();
        return executor;
    }
}
```

- **corePoolSize**: 기본 스레드 수
- **maxPoolSize**: 최대 스레드 수
- **queueCapacity**: 대기 큐
- **threadNamePrefix:** 스레드 이름 Prefix

## ThreadPoolTaskExecutor 사용

- **사용자 정의 executor 사용**: AsyncConfig에서 설정한 ThreadPoolTaskExecutor 사용
- **스레드 풀 활용**: 설정한 풀 크기(2~5)에서 재사용
- **성능 최적화**: 효율적인 스레드 관리, 큐 처리 가능

```
@Async("taskExecutor")
public CompletableFuture<String> asyncMethodWithReturn(String message) { ... }
```

# 비교

- 메서드를 호출한 스레드와 분리하여 비동기(Async)로 실행하도록 지시하는 Spring 기능 Annotation
- 호출자는 즉시 반환하고, 실제 실행은 별도 스레드 풀에서 처리되도록 구성

| 항목 | @Async | @Async("taskExecutor") |
|---|---|---|
| Executor | SimpleAsyncTaskExecutor | ThreadPoolTaskExecutor |
| 스레드 생성 | 매번 새로 생성 | 풀에서 재사용 |
| 코어 스레드 | 없음 | 2개 (설정값) |
| 최대 스레드 | 무제한 | 5개 (설정값) |
| 큐 용량 | 없음 | 100 (설정값) |
| 스레드 명 | SimpleAsyncTaskExecutor-N | async-1, async-2 |

# Async 사용 방식

- **기본 방식**

```
@Service
public class MailService {

    @Async
    public void sendMail(String email) {
        System.out.println("Send mail to " + email);
        System.out.println("Thread: " + Thread.currentThread().getName());
    }
}
```

- **호출 코드 (비동기 실행)**

```
@RestController
public class MailController {

    private final MailService mailService;

    public MailController(MailService mailService) {
        this.mailService = mailService;
    }

    @GetMapping("/mail")
    public String send() {
        mailService.sendMail("user@test.com");
        return "Request Accepted!";
    }
}
```

Main Thread는 바로 응답 발송

sendMail Method는 별도의 Thread에서 실행

# Async 사용 방식 : Blocking 방식 사용

- **CompletableFuture 리턴**

```
@Service
public class ReportService {

    @Async
    public CompletableFuture<String> generateReport() {

        Thread.sleep(2000); // 무거운 작업

        return CompletableFuture.completedFuture("Report Complete!");
    }
}
```

- **Blocking 대기 방식**

```
String status = reportService.generateReport().get();     //"Report Complete"
```

**실행 흐름:**

① @Async → generateReport() 는 별도 스레드에서 실행
② 하지만 .get() 을 호출하면
③ 현재 스레드(예: 톰캣 요청 처리 스레드)가 작업 완료까지 대기
④ generateReport() 가 끝나야 get()이 반환됨

→ 즉, 비동기 실행이지만 get()을 호출하는 순간 다시 동기 방식이 된다.

# Async 사용 방식 : Non-Blocking 방식 사용

- **CompletableFuture 리턴**

```
@Async("taskExecutor")
public CompletableFuture<String> asyncMethodWithReturn(String message) {
    try {
        Thread.sleep(3000); // 3초 대기
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
    String result = "처리 완료: " + message;

    return CompletableFuture.completedFuture(result);
}
```

- **Non-Blocking 대기 방식**

```
@GetMapping("/future")
public CompletableFuture<String> callAsyncFuture(@RequestParam String message) {

    //즉시 리턴
    CompletableFuture<String> future = asyncService.asyncMethodWithReturn(message);

    return future;
}
```

**호출 흐름: asyncService**는 순수한 **원본 객체가 아니라 스프링이 만든 프록시 객체**

컨트롤러는 원본 메서드를 직접 실행하는 게 아니라 **프록시의 메서드를 호출**하게 됨

# [참고] CompletableFuture 주요 메서드

CompletableFuture의 핵심 메서드 분류(등록, 조합, 실행)

| 메서드 | 설명 | 사용 예시 코드 |
|---|---|---|
| **thenApply** | **Stream의 map과 유사** | CompletableFuture.supplyAsync(() -> "hello")<br>.thenApply(s -> s + " world"); |
| **thenAccept** | 결과를 수신해서 처리하며 리턴 없음 | CompletableFuture.supplyAsync(() -> "data")<br>.thenAccept(d -> System.out.println(d)); |
| **whenComplete** | 결과값을 관찰하고 PASS | future.whenComplete((res, ex) -> {<br>  if (ex != null) log.error("Error", ex);<br>}); |
| **thenCompose** | FlatMap과 유사 | CompletableFuture.supplyAsync(() -> userId)<br>.thenCompose(id -> getUserOrderAsync(id)); |
| **thenCombine** | **서로 독립적인 두 비동기 작업**을 병합 | futureA.thenCombine(futureB,<br>(resA, resB) -> resA + resB); |
| **complete** | 대기 중인 CompletableFuture를 **외부에서 수동으로 완료 처리**하고 결과 값을 강제로 채움 | CompletableFuture<String> future =<br>  new CompletableFuture<>();<br>future.complete("Manual Result"); |

# [실습] Async

- @EnableAsync / @Async 어노테이션의 동작 방식을 이해한다.
- 커스텀 스레드 풀(ThreadPoolTaskExecutor)을 직접 설정할 수 있다.
- 반환값 없는 비동기 메서드와 CompletableFuture 반환 메서드의 차이를 설명할 수 있다.
- 동기 호출과 비동기 호출의 응답 시간 차이를 직접 확인한다.

### 작성파일 목록 및 설명

| 파일 | 패키지 | 설명 |
|---|---|---|
| `AsyncConfig.java` | `config` | 비동기 활성화 및 스레드 풀 설정 |
| `AsyncService.java` | `service` | 비동기/동기 메서드 구현 |
| `AsyncController.java` | `controller` | REST API 엔드포인트 제공 |

> 세 파일을 **순서대로** 작성하세요. Config → Service → Controller

# [실습] Async

## AsyncConfig.java

**요구사항**

- 클래스에 `@Configuration`, `@EnableAsync` 를 붙인다.
- `"taskExecutor"` 이름의 `Executor` Bean 을 등록한다.
- 스레드 풀 설정값은 아래 표를 따른다.

| 속성 | 값 |
| --- | --- |
| corePoolSize | `2` |
| maxPoolSize | `5` |
| queueCapacity | `100` |
| threadNamePrefix | `"async-"` |

# [실습] Async 코드

### AsyncService.java

```
@Slf4j
@Service
public class AsyncService {

    @Async
    public void asyncMethodWithoutReturn(String message) {
        try {
            Thread.sleep(2000); // 2초 대기
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        log.info("비동기 메서드 완료 - Thread: {}, Message: {}",
                 Thread.currentThread().getName(), message);
    }

    @Async("taskExecutor")
    public CompletableFuture<String> asyncMethodWithReturn(String message) {
        try {
            Thread.sleep(3000); // 3초 대기
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        String result = "처리 완료: " + message;
        log.info("비동기 메서드(반환값) 완료 - Thread: {}, Result: {}",
                 Thread.currentThread().getName(), result);
        return CompletableFuture.completedFuture(result);
    }
}
```

# [실습] Async

### AsyncController.java

| HTTP | URL | 호출 메서드 | 설명 |
|---|---|---|---|
| GET | `/api/void?message=` | `asyncMethodWithoutReturn` | 즉시 응답, 백그라운드 처리 |
| GET | `/api/future?message=` | `asyncMethodWithReturn` | 비동기 처리 결과를 응답 |

### 아래 코드로 구성 : AOP @Metrics 를 이용한 결과

```
    @GetMapping("/future")
    @Metrics
    public CompletableFuture<String> callAsyncFuture(@RequestParam String message) {
        CompletableFuture<String> future = asyncService.asyncMethodWithReturn(message);
        return future;
    }

    @GetMapping("/void")
    @Metrics
    public String callAsyncVoid(@RequestParam String message) {
        asyncService.asyncMethodWithoutReturn(message);
        return "비동기 작업이 시작되었습니다. (반환값 없음)";
    }
```

# [실습] Async

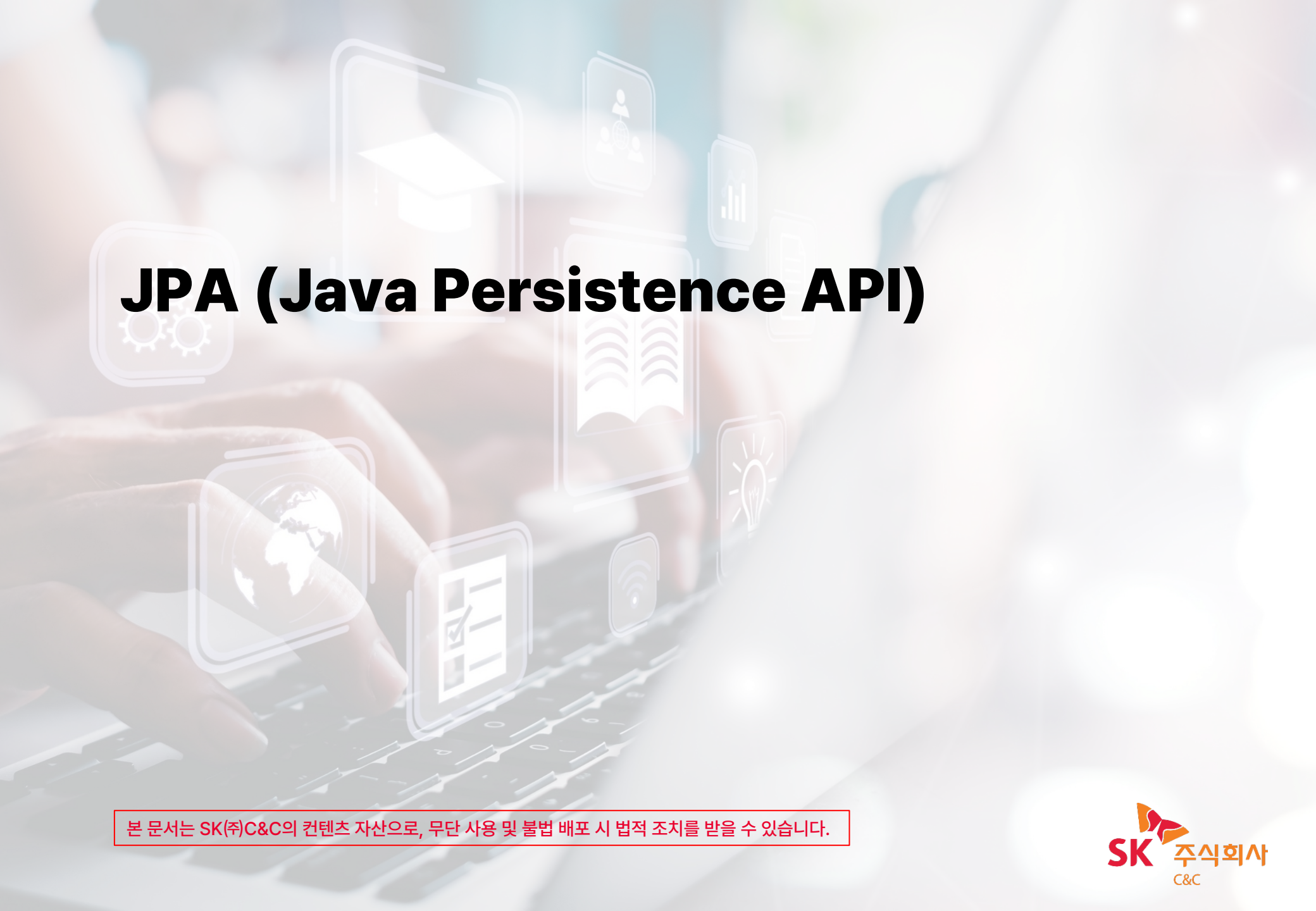
**직접 호출해보고 console 로그를 확인해보고  Async 구조를 이해해보자**

**1. 비동기 서비스 호출 :응답 있음**

http://localhost:8080/api/future?message=hello

**1. 비동기 서비스 호출: 응답 없음**

http://localhost:8080/api/void?message=hello



# JPA (Java Persistence API)

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# Entity

Entity는  관계형 데이터베이스의 **테이블과 1:1로 매핑되는 자바 클래스**
각 인스턴스는 테이블의 **한 행(Row)** 을 의미

## 특징

- 클래스 = 테이블
- 객체(instance) = 행(row)
- 필드(field) = 컬럼(column)

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name="user_name", nullable = false, length = 50)
    private String name;
}
```

- @Entity: 테이블과 매핑할 클래스에 붙여주는 Annotation
- @Id: Primary Key(PK)로 지정할 필드에 붙여주는 Annotation
- @Column:  자바 필드를 DB 테이블의 컬럼으로 매핑 ( user_name VARCHAR(50) NOT NULL)

# Entity 와 Table 매핑 시 이름 연결 방법

- Entity 객체/필드의 이름은 Camel Case 로 구성시 Table 명은 자동적으로 Snake Case로 전환

  예) userName → user_name

- 명시적으로 이름을 매핑하려면 @Table, @Column 사용

```
@Entity
@Table(name = "users") // 테이블명을 snake_case로 명시
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

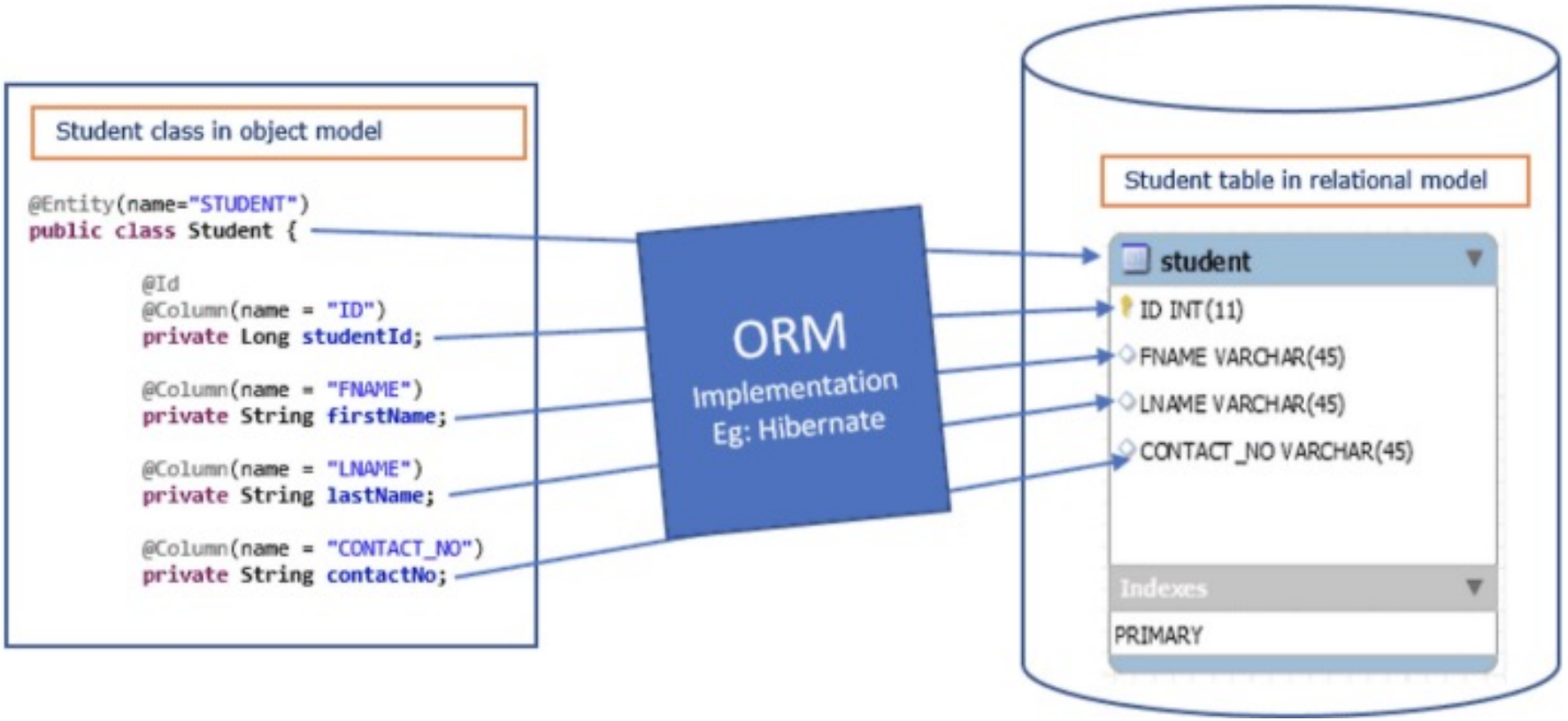
    @Column(name = "user_name")    // 명시적 매핑 (camelCase → snake_case)
    private String userName;

    @Column(name = "user_email")  // 명시적 매핑
    private String userEmail;

    @Column(name = "total_price") // 명시적 매핑
    private BigDecimal totalPrice;
}
```

# ORM (Object-Relational Mapping)

객체와 관계형 데이터베이스 간 의 매핑을 자동으로 처리해주는 기술
ORM에서는 Entity 들을 객체의 형태로 다룰 수 있게 지원



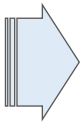
# ORM (Object-Relational Mapping)

**Object** **Mapping** **Relational**

```java
@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(unique = true, nullable = false, length = 150)
    private String email;
}
```

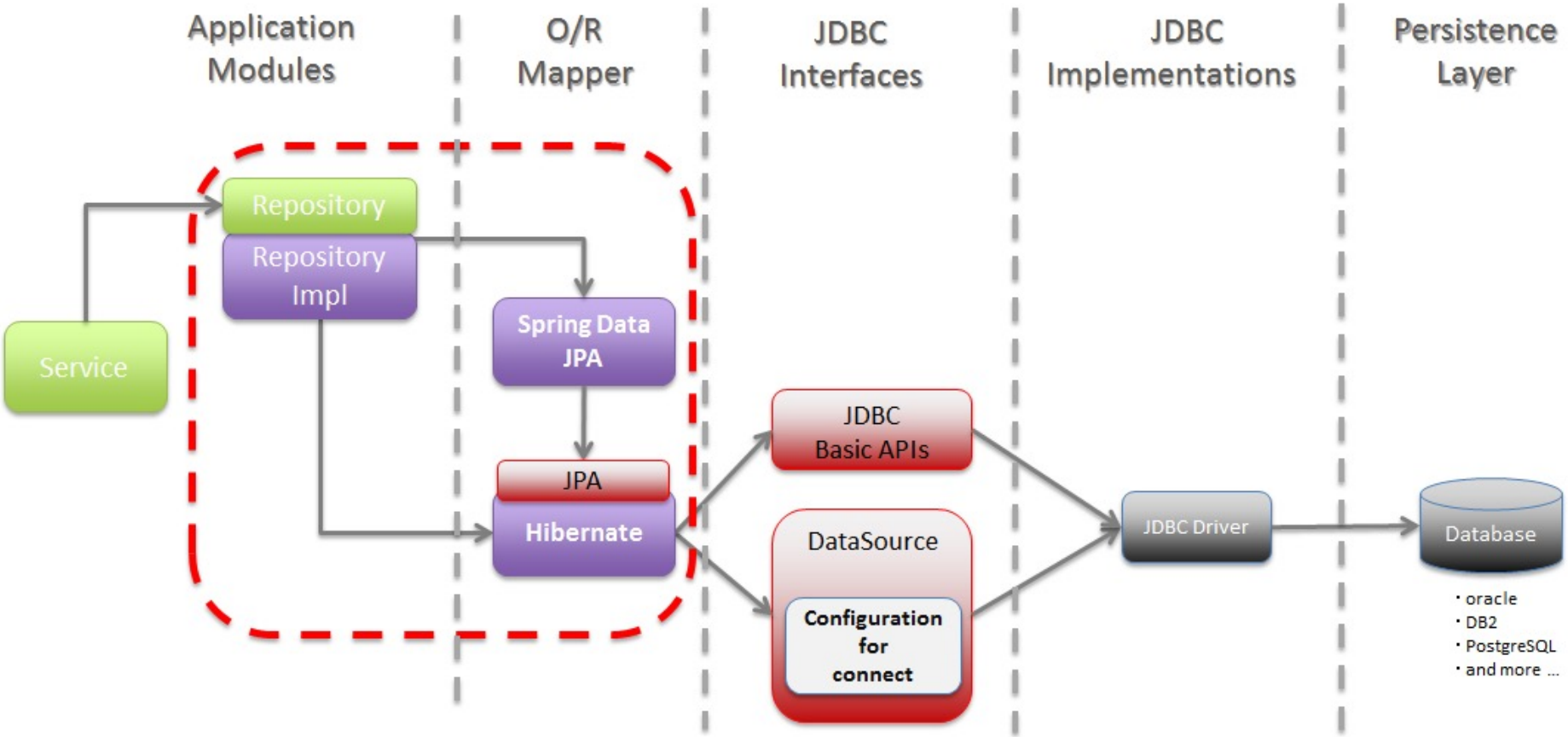


```sql
CREATE TABLE users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL
);
```

| Object (객체) | Relational Table (테이블) |
|---|---|
| User (클래스) | users (테이블) |
| id (Long, Primary Key) | id (BIGINT, PRIMARY KEY, AUTO_INCREMENT) |
| name (String) | name (VARCHAR(100), NOT NULL) |
| email (String) | email (VARCHAR(150), UNIQUE, NOT NULL) |

# JPA (Java Persistence API)

ORM 기술에 대한 **Java의 ORM(Object-Relational Mapping) 프레임워크 구현체**
JPA는 "인터페이스"와 "어노테이션"만 제공하고 Hibernate, EclipseLink 등의 구현체가 별도로 존재
(spring boot에서는 주로 Hibernate를 사용하며 다른 것으로 교체 가능)



# [참고] JPA vs JDBC

- JDBC: Java Database Connectivity의 약자로, 자바에서 데이터베이스에 접근하기 위한 표준 API

| 구분 | JPA (Java Persistence API) | JDBC (Java Database Connectivity) |
|---|---|---|
| 목적 | 자바 객체와 DB 테이블 매핑(ORM) | 자바에서 SQL을 직접 실행하여 DB와 연동 |
| 코딩 방식 | 객체지향적(Entity/Repository/Service 계층)<br>SQL 최소화 | SQL을 직접 작성<br>Connection/Statement/ResultSet 사용 |
| 자동화 수준 | CRUD, 트랜잭션, 매핑, 변경감지 등 많은 부분 자동화 | 모든 DB 작업을 개발자가 직접 처리 |
| SQL 작성 | 메서드 자동 생성, @Query를 통한 쿼리 작성 가능 | 모든 쿼리를 직접 작성 |
| 트랜잭션 | @Transactional로 선언적 관리 | Connection 객체로 직접 트랜잭션 제어 |
| 예외 처리 | DataAccessException 등 추상화된 예외 사용 | SQLException 사용, 직접 예외 처리 필요 |
| 데이터 매핑 | 객체(Entity)와 테이블 자동 매핑 | ResultSet에서 직접 필드 추출, 수동 매핑 |
| 학습 난이도 | 개념(ORM, 엔티티, 영속성 컨텍스트 등) 필요 | JDBC와 SQL만 알면 바로 사용 가능 |
| 성능 최적화 | 캐시, 배치 처리 등 별도 튜닝 필요 | SQL 및 JDBC 커넥션 풀 등 직접 튜닝 |
| 대표 사용 예 | Spring Data JPA, Hibernate | JdbcTemplate, MyBatis, 순수 JDBC 코드 |
| 커넥션 관리 | 스프링이 자동 관리, 커넥션 풀 연동 | 직접 관리(try-finally 등으로 close 필수) |

**호출 흐름**

```
개발자 코드
   ↓ (EntityManager API, JPQL)
JPA (표준)
   ↓ (스펙 위임)
Hibernate, EclipseLink (구현체)
   ↓ (SQL 생성 + JDBC 호출)
JDBC
   ↓ (드라이버)
Database
```

# Database 연결 설정

- 스프링부트에서 JPA 및 데이터베이스 연결 정보는 application.yml에 설정

```
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver
    username: sa
    password:
  h2:
    console:
      enabled: true
      path: /h2-console
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true
    defer-datasource-initialization: true
  sql:
    init:
      mode: always
```

❖ ddl-auto: update, create, create-drop, validate 등

❖ h2.console 기본 path: /h2-console

❖ show-sql: Entity에서 JDBC로 연결되어 실제 Query를 수행할ㄸ 사용되는 SQL 문을 System.out 출력

# Database 연결 설정

스프링부트에서 JPA 및 데이터베이스 연결 정보는 application.yml에 설정

```
# HikariCP 튜닝 (옵션)

spring:
  datasource:
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2
      connection-timeout: 30000
      idle-timeout: 600000
```

❖ Spring Boot는 2.0 버전부터 기본 데이터베이스 커넥션 풀로 HikariCP를 사용

세밀한 커넥션 풀 설정 필요 시 hikari 속성으로 조정 가능

```
애플리케이션
   ↓ (DataSource 요청)
HikariCP (Connection Pool)
   - 미리 만들어둔 DB Connection 보관
   - 필요 시 getConnection()
   - 사용 후 close() → 실제로는 반납
   ↓
JDBC 드라이버
   ↓
Database
```

# [참고] mariadb 기반 JPA 설정 방법

- 스프링부트에서 JPA 및 데이터베이스 연결 정보는 application.yml에 설정

```
spring:
  jpa:
    open-in-view: true
    generate-ddl: true
    show-sql: false
  datasource:
    url: jdbc:mariadb://localhost:3306/skala?allowMultiQueries=true
    username: admin
    password: xxxx
    driver-class-name: org.mariadb.jdbc.Driver
    hikari:
      idle-timeout: 30000
      max-lifetime: 30000
      maximum-pool-size: 20
      connection-timout: 10000
```

# JPA 주요 어노테이션

| 어노테이션 | 목적/용도 (상세 설명) |
|---|---|
| @Entity | **클래스를 JPA가 관리하는 엔티티로 지정**<br>JPA는 이 클래스를 테이블에 매핑해서 CRUD 등 데이터베이스 작업을 처리할 수 있게 됨. |
| @Table | **엔티티와 매핑할 데이터베이스 테이블의 이름·스키마 지정**<br>클래스명과 실제 테이블명이 다르거나, DB 스키마를 지정해야 할 때 사용 |
| @Id | **기본키(Primary Key)로 사용할 필드 지정**<br>각 엔티티마다 식별자가 필요하고 이를 통해 객체의 고유성을 보장 |
| @GeneratedValue | **기본키 값을 자동 생성하는 전략 설정**<br>DB의 auto-increment나 시퀀스 등을 활용해 PK를 자동 할당하고 싶을 때 사용 |
| @Column | **클래스 필드와 DB 컬럼을 세부적으로 매핑**<br>컬럼명, 길이, null 허용, 유니크 등 제약조건 지정(필드가 DB 컬럼명과 다를 때도사용) |
| @Enumerated | **Enum 타입 필드를 DB에 저장할 때 방식 지정**<br>값 자체를 숫자로 저장할지(ORDINAL), 문자열로 저장할지(STRING) 선택 |
| @Lob | **대용량 데이터(텍스트/이미지 등)를 DB에 저장**<br>일반적인 문자열이 아닌, 긴 텍스트(CLOB)나 바이너리 데이터(BLOB) 매핑 |
| @Transient | **DB에 저장하지 않는 필드 선언**<br>계산된 값, 일시적 데이터 등 DB 컬럼으로 만들 필요 없는 필드에 사용 |
| @{One/Many}To{One/Many} | **엔티티 간의 관계를 명시적으로 선언**<br>관계형 DB의 외래키와 매핑하며, 객체지향적으로 연관관계를 관리(1:1, 1:N, N:1, N:N 등) |
| @JoinColumn | **관계 매핑시 외래키 컬럼을 직접 지정**<br>관계 엔티티의 어떤 컬럼이 외래키인지 명확히 표기할 때 사용 |
| @Embedded | **내장 값 타입을 엔티티에 포함**<br>별도의 클래스를 필드로 사용하여, 컬럼을 그룹화해서 재사용 |
| @Embeddable | **내장 값 타입 클래스 지정**<br>@Embedded로 포함될 클래스에 선언 (여러 엔티티에서 재사용 가능) |

# JPA 어노테이션 - @Entity

- 해당 클래스를 JPA가 관리하는 엔티티(테이블)로 지정 - 자바 클래스와 데이터베이스의 테이블을 1:1로 매핑하는 역할
- @Entity가 선언된 클래스는 JPA의 CRUD 대상이 됨. (Persistent Object)
- JPA가 Persistence Context 안에서 생명주기를 관리하는 대상으로 지정

```
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity                 // 이 클래스를 JPA가 관리하는 엔티티로 지정
@Table(name = "stocks") // (선택) stocks 테이블에 매핑
public class Stock {

    @Id
    private String ticker; // 주식의 고유 코드 (ex: "AAPL")

    private String name;   // 주식 이름 (ex: "Apple Inc.")

    private String market; // 시장 구분 (ex: "NASDAQ")

    // getter, setter 등 생략
}
```

- @Entity가 붙은 클래스는 반드시
  **기본 생성자(파라미터 없는 생성자)**가 필요
- @Entity만 선언하면 클래스 이름이 그대로 테이블명으로 매핑
- 실제 운영에서는 테이블 명과 클래스명이 다를 수 있음
  보통 @Table을 함께 사용해서 명시적으로 테이블명을 지정

# JPA 어노테이션 - @Table

- @Entity로 지정한 클래스가 어느 DB 테이블과 매핑될지 명시
- 주로 테이블 이름이 클래스명과 다를 때 사용하거나, 스키마 지정 등 추가 설정이 필요할 때 사용(생략 가능)

| 속성 | 설명 | 예시 |
|---|---|---|
| name | 매핑할 DB 테이블 이름을 지정 | @Table(name = "stocks") |
| schema | 사용할 DB 스키마 지정 (DBMS가 지원할 때만) | @Table(name = "stocks", schema = "skala") |

```
@Entity
@Table(
    name = "stocks",
    schema = "skala"
)
public class Stock {
    @Id
    private String ticker;
    private String name;
    private String market;
}
```

# JPA 어노테이션 - @Id

- 엔티티의 "기본키(Primary Key)" 필드를 지정
- JPA는 모든 엔티티에 반드시 하나 이상의 @Id가 필요

```
// 기본키가 하나인 경우 (단일 필드)
// ticker 필드가 stocks 테이블의 Primary Key

@Entity
@Table(name = "stocks")
public class Stock {
    @Id
    private String ticker; // 예: "AAPL"

    private String name;
    private String market;
}
```

```
// 자동 생성되는 숫자 기본키
// MySQL 등에서 auto_increment로 PK가 자동 할당

import javax.persistence.*;

@Entity
@Table(name = "transactions")
public class Transaction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // 자동 증가되는 기본키

    private String ticker;     // 주식 코드
    private int quantity;      // 거래 수량
    private double price;      // 거래 가격
    private LocalDateTime date; // 거래 일시
}
```

# JPA 어노테이션 - @Id (복합키)

주로 두 개 이상의 컬럼 조합이 고유해야 하는 경우 사용

```java
@Embeddable
public class UserStockId implements Serializable {
    private Long userId;
    private String ticker;
}

@Entity
@Table(name = "user_stocks")
public class UserStock {
    @EmbeddedId
    private UserStockId id;

    private int quantity;
}
```

- @Embeddable 클래스로 복합키 타입 정의
- @EmbeddedId로 엔티티에 복합키 사용

```sql
CREATE TABLE user_stocks (
    user_id BIGINT NOT NULL,
    ticker VARCHAR(255) NOT NULL,
    quantity INT NOT NULL,
    PRIMARY KEY (user_id, ticker)
);
```

예시)

```java
public interface UserStockRepository extends Jparepository<UserSotck, UserStockId> {}
```

```java
UserStockId id = new UserStockId();
id.setUserId(1L);
id.setTicker("AAPL");

Optional<UserStock> result = userStockRepository.findById(id);
result.ifPresent(s -> System.out.println(s.getQuantity()));
```

# JPA 어노테이션 - @GeneratedValue

- 기본키(PK) 값을 자동으로 생성하는 전략을 지정할 때 사용 - 반드시 @Id와 함께 사용
- 데이터베이스의 auto_increment, 시퀀스, 별도 테이블 등 다양한 자동 생성 방식을 지원
- 주요 속성
  - strategy : PK 자동 생성 방식 지정
  - generator : 시퀀스/테이블 방식일 때 커스텀 생성기 이름 지정

```
// MySQL, MariaDB 기준 - DB의 auto_increment 사용

@Entity
@Table(name = "transactions")
public class Transaction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;  // 자동 생성 PK

    private String ticker;
    private int quantity;
    private double price;
    private LocalDateTime tradedAt;
}
```

**[strategy : GenerationType.IDENTITY]**

- AUTO: DB에 맞는 전략을 JPA가 알아서 선택 (기본값)
- IDENTITY: INSERT 시점에 ID 생성 (Mariadb, SQL Server etc)
- SEQUENCE: PK를 미리 뽑아서 INSERT 시 사용 (Oracle, Postgres)

# JPA 어노테이션 - @Column

- 엔티티 필드를 데이터베이스의 컬럼과 매핑할 때 사용
- 컬럼 이름, 길이, null 허용, 유니크 여부, 숫자 자릿수 등 DB 컬럼 속성을 세밀하게 조정 가능

| 속성 | 설명 | 예시 |
|---|---|---|
| name | 컬럼명 지정 | @Column(name = "user_name") |
| nullable | null 허용 여부 (기본값: true) | @Column(nullable = false) |
| length | 문자 길이(문자열 필드에만 사용) | @Column(length = 50) |
| unique | 유니크 제약조건 부여(단일 컬럼일 때) | @Column(unique = true) |
| updatable | UPDATE 쿼리에서 수정 가능 여부 | @Column(updatable = false) |
| precision, scale | 소수점 숫자 자리수 지정(숫자형만) | @Column(precision = 10, scale = 2) |

# @Column 속성 매핑

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_name", nullable = false, length = 30)
    private String name; // DB 컬럼: user_name, NOT NULL, 최대 30자

    @Column(unique = true, length = 50)
    private String email; // DB 컬럼: email, UNIQUE, 최대 50자

    @Column(length = 100)
    private String password; // 비밀번호 컬럼, 길이 100자 제한

    @Column(updatable = false)
    private LocalDateTime createdAt; // 최초 생성일(수정 불가)
}
```

# @Column 속성 매핑 - 숫자 자리 지정

```
@Entity
public class Transaction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(precision = 12, scale = 4)
    private BigDecimal price; // 최대 9999999999.9999까지 표현 가능
}
```

# JPA 어노테이션 - @Enumerated

- Enum(열거형) 타입 필드를 데이터베이스에 어떻게 저장할지 지정할 때 사용
- 자바의 enum 타입은 RDB에는 직접 저장할 수 없으므로, 숫자(ORDINAL)나 문자열(STRING)로 변환하여 저장
- 실무에서는 STRING 방식을 강력 권장

```
public enum StockStatus {
    ACTIVE,
    INACTIVE
}

@Entity
public class Stock {
    @Id
    private String ticker;

    @Enumerated(EnumType.STRING)   //  or 숫자 (ORDINAL)
    @Column(length = 10)
    private StockStatus status; // "ACTIVE" 또는 "INACTIVE"로 저장
}
```

# JPA 어노테이션 - @Lob

- 대용량 데이터(텍스트 또는 바이너리 데이터)를 데이터베이스에 저장할 때 사용 - 필드 타입에 따라 자동으로 CLOB 또는 BLOB으로 매핑
  - **CLOB(Character Large Object)**: 대용량 텍스트 (ex. 긴 설명, 본문 등)
  - **BLOB(Binary Large Object)**: 대용량 바이너리 (ex. 이미지, 파일 등)

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @Lob
    private String description; // @Lob + String 타입 → CLOB(대용량 텍스트)로 매핑됨

    @Lob
    private byte[] profileImage; // @Lob + byte[] 타입 → BLOB(대용량 바이너리)로 매핑됨
}
```

❖ **@Lob vs @Column(columnDefinition = "TEXT")**

- **@Lob은 JPA 표준 방식, DB 독립적** → 유지보수·확장성에 유리
- **columnDefinition은 DB 종속적**, 유연성 적음 → 특별한 경우에만 사용

# JPA 어노테이션 - @Transient

- 엔티티 필드 중에서 DB 컬럼과 매핑하지 않을 필드에 사용
  - 해당 필드는 DB 테이블에 컬럼으로 생성되지 않으며,
  - JPA의 영속화(저장/조회) 대상이 아님을 명시
- 계산값, 임시 데이터, 화면 출력 전용 속성, 패스워드 등에 활용

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;

    private String password;

    @Transient
    private String passwordConfirm; // 입력폼 전용, DB 저장 X
}
```

DTO 객체를 사용하여 내부에서 임시 저장하고 실제 데이터베이스 저장시에만 Entity를 사용하게 되면 실제 Transient는 필요 없음



# JPA Repository

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# JPA Repository 정의

JPA에서 Repository는 데이터베이스와 직접적인 상호작용을 담당하는 인터페이스
Spring Data JPA에서 제공하는 JpaRepository를 사용하면 기본적인 CRUD(Create, Read, Update, Delete) 기능을 별도의 SQL 코드 없이 사용 지원

```
// CRUD
// C : CREATE  → 생성
// R : READ    → 읽기 (SELECT)
// U : UPDATE  → 업데이트
// D : DELETE  → 삭제

package com.example.study.repository;

import com.example.study.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    // JpaRepository를 상속받으면 기본적인 CRUD 메서드 제공
    // save(), findById(), findAll(), delete() 등
}
```

참고) 이구조는 JpaRepository가 구현한 Method (findAll, findById, save, delete)를 상속받아서 제공
UserRepository는 UserRepository를 상속받은 **Proxy**를 동적으로 생성해서 Bean으로 등록

# JPA Repository

- Spring Data JPA가 제공하는 인터페이스로 JPA를 더 쉽게 다루기 위한 추상화 계층
- 개발자가 데이터베이스에 접근하는 코드 DAO(Data Access Object)를 직접 구현하지 않고도 데이터베이스 조작을 가능하게 하는 인터페이스 기반의 프레임워크

| | |
|---|---|
| **Repository** | 가장 상위의 기본 인터페이스로, 아무런 메서드도 정의되어 있지 않은 마커(Marker) 인터페이스 |
| 상속 | |
| **CrudRepository** | Repository를 상속받으며, 데이터의 CRUD 기능을 위한 기본적인 메서드들을 제공 |
| 상속 | |
| **PagingAndSortingRepository** | CrudRepository를 상속받으며, 페이징과 정렬 기능을 위한 메서드를 추가로 제공 |
| 상속 | |
| **JpaRepository** | PagingAndSortingRepository를 상속받으며, JPA에 특화된 기능들을 추가로 제공합니다. 일반적으로 가장 많이 사용되는 인터페이스 |

# JPA Repository 기본 Method

JPA 선언 시 자동 구성되는 기본 Method

| 메서드 | 설명 | SQL 동작 |
|---|---|---|
| save(S entity) | 엔티티 저장 (INSERT or UPDATE) | INSERT 또는 UPDATE |
| saveAll(Iterable<S> entities) | 여러 엔티티를 한 번에 저장 | BULK INSERT 또는 UPDATE |
| findById(ID id) | ID(PK)로 엔티티 조회 | SELECT * WHERE id = ? |
| existsById(ID id) | 특정 ID의 엔티티 존재 여부 확인 | SELECT COUNT(*) WHERE id = ? |
| findAll() | 모든 엔티티 조회 (Page 단위로 조회 가능) | SELECT * FROM table |
| findAllById(Iterable<ID> ids) | 여러 ID에 해당하는 엔티티 조회 | SELECT * WHERE id IN (?) |
| count() | 엔티티 개수 반환 | SELECT COUNT(*) FROM table |
| deleteById(ID id) | 특정 ID의 엔티티 삭제 | DELETE FROM table WHERE id = ? |
| delete(T entity) | 특정 엔티티 삭제 | DELETE FROM table WHERE id = ? |
| deleteAllById(Iterable<ID> ids) | 여러 ID의 엔티티 삭제 | DELETE FROM table WHERE id IN (?) |
| deleteAll(Iterable<? extends T> entities) | 여러 엔티티 삭제 | DELETE FROM table WHERE id IN (?) |
| deleteAll() | 모든 엔티티 삭제 | DELETE FROM table |
| flush() | 현재 영속성 컨텍스트의 변경 내용을 즉시 반영 | FLUSH |
| saveAndFlush(S entity) | 저장 후 즉시 데이터베이스에 반영 | INSERT/UPDATE + FLUSH |
| deleteAllInBatch() | 전체 데이터 삭제 (개별 DELETE 없이 한 번에) | DELETE FROM table |
| deleteAllByIdInBatch(Iterable<ID> ids) | 여러 개의 데이터를 한 번에 삭제 | DELETE FROM table WHERE id IN (?) |

✔ deleteAllById()는 엔티티를 먼저 조회한 후 삭제하는 반면,
✔ deleteAllByIdInBatch()는 조회 없이 한 번의 SQL 쿼리로 바로 삭제합니다.
✔ 따라서 대량 데이터 삭제 시 deleteAllByIdInBatch()가 성능이 더 좋습니다.

# JPA Repository 기본 Method

| 메서드 시그니처 | 설명 | 리턴값 |
|---|---|---|
| `<S extends T> S save(S entity)` | 엔티티 저장 또는 병합(수정) | `S` |
| `<S extends T> List<S> saveAll(Iterable<S> entities)` | 여러 엔티티 저장/수정 | `List<S>` |
| `<S extends T> S saveAndFlush(S entity)` | 엔티티 저장/수정 후 즉시 flush | `S` |
| **`void flush()`** | **영속성 컨텍스트 변경 내용을 DB에 즉시 반영** | **`void`** |

❖ **flush()** : JPA에서 영속성 컨텍스트(Persistence Context)의 변경 내용을 즉시 DB에 반영(동기화)하도록 하는 메서드

- JPA는 기본적으로 트랜잭션이 커밋(commit) 되는 시점에 자동으로 flush를 호출하여 DB에 쿼리를 실행
- 저장, 수정, 삭제를 해도 즉시 DB 쿼리가 실행되는 것이 아니라, 영속성 컨텍스트에 반영된 뒤, 트랜잭션이 커밋될 때 한 번에 반영(Flush)되는 구조

❖ **flush()를 직접 호출하는 대표적 상황**

- DB에 즉시 쿼리를 반영해야 할 때 : 예를 들어, 바로 이후에 Native Query 또는 다른 JDBC 작업을 수행해야 할 때, 변경 내용을 DB에 먼저 반영해야 할 수 있음
- 트랜잭션 내에서 쿼리 동작 순서를 명확히 해야 할 때: 예를 들어, insert 후 select를 실행해야 할 때, 영속성 컨텍스트의 변경이 DB에 반영되어야 원하는 결과가 나올 때
- JPA 이벤트 리스너나 복잡한 이벤트 처리 로직: 엔티티의 상태를 DB에 반영해야만 이벤트가 정상적으로 처리되는 경우

# JPA 사용자 쿼리 메서드 (Query Methods)

JPA가 정한 약속된 메서드 명명 규칙에 맞춰 인터페이스 선언시 동적으로 쿼리 자동 생성 방식

- **Entity User**

```
@Entity
@Table(name = "users")
public class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private long id;

  private String name;
  private String email;
}
```

- **JpaRepository :** 이터페이스에 메소드 선언만으로 실제 Quey 구현체 자동 생성

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

# JPA 사용자 쿼리 메서드

- 쉽고 빠르며, 실수 방지. 단순 조건, 검색, 정렬에 매우 효율적
- 조인, 복잡 논리(AND/OR 혼합), 그룹/집계, 복잡 동적 쿼리에는 적합하지 않음
- 이름이 너무 길어질 땐 @Query로 전환 고려

```
// 1. 이름과 나이로 사용자 찾기 (자동 쿼리 생성)
List<User> findByNameAndAgeGreaterThan(String name, int age);

// 2. 상태와 이메일로 사용자 검색 (첫번째 1개만)
Optional<User> findByStatusAndEmail(UserStatus status, String email);

// 3. 생성일 기준 정렬, 페이징
Page<User> findByCreatedAtAfter(LocalDateTime date, Pageable pageable);
```

1. 이름과 나이로 사용자 찾기

```
SELECT *
FROM user
WHERE name = ?
  AND age > ?;
```

2. 상태와 이메일로 사용자 검색 (1개)

```
SELECT *
FROM users u
WHERE u.status = ?
  AND u.email = ?
LIMIT 1;
```

3. 생성일 기준 정렬, 페이징

```
SELECT *
FROM users
WHERE created_at > ?
```

# JPA 메소드를 위한 사전 정의 키워드 (1/2)

사전에 정의된 메소드 키워드

| 키워드 | 설명 | 예제 |
|---|---|---|
| findBy | 조회 | findByEmail(String email) |
| readBy | 조회 (find와 동일) | readByEmail(String email) |
| queryBy | 조회 (find와 동일) | queryByEmail(String email) |
| getBy | 조회 (find와 동일) | getByEmail(String email) |
| countBy | 카운트 조회 | countByStatus(String status) |
| existsBy | 존재 여부 (true, false) | existsByEmail(String email) |
| deleteBy | 엔티티 삭제 | deleteByEmail(String email) |
| removeBy | 엔티티 삭제 | removeByEmail(String email) |

### return 타입은 다양한 형태로 사용 가능 (선언에 따라 자동 구성)

- User , Optional<User>, List<User>, Page<User>, long, Boolean 등 가능
- 한개 리턴 시 Optional 권장
- 여러 개 리턴 시 Optional 사용 안됨

# JPA 메소드를 위한 사전 정의 키워드 (2/2)

여러 메소드를 조합하기 위한 연산자

| 키워드 | 설명 | 예제 |
|---|---|---|
| And | AND로 결합 | findByNameAndEmail(String name, String email) |
| Or | OR로 결합 | findByNameOrEmail(String name, String email) |
| OrderBy | 정렬 기준 추가 | findByNameOrderByEmailAsc(String name) |
| Between | 두 값 사이의 범위 검색 | findByAgeBetween(int min, int max) |
| GreaterThan | 특정 값보다 큰 값 검색 | findByAgeGreaterThan(int age) |
| LessThan | 특정 값보다 작은 값 검색 | findByAgeLessThan(int age) |
| Like | 특정 문자열 포함 검색<br>ex) '홍%','%홍%','%홍' | findByNameLike(String namePattern)<br><br>예) List<User> users = userRepository.findByNameLike("%홍%"); |
| In | 여러개 중 하나를 포함하는 조회 | findByStatusIn(List<String> statuses)<br><br>예) List<User> users = userRepository.findByStatusIn(List.of("ACTIVE", "PENDING")) |

# [참고] JPA 사용자 쿼리 메서드 키워드(Keyword)

| 구분 | 키워드(메서드 내 단어) | 의미 / 동작 | 예시 메서드명 |
|---|---|---|---|
| 조회 | *find, read, get, query, search, stream, count, exists, delete, remove* | 조회, 개수, 존재 여부, 삭제 등 | *findByName, countByAge, existsByEmail* |
| 조건 | *By* | 뒤에 오는 조건 필드를 지정 | *findByName, findByAge* |
| | *And, Or* | 조건 연결(AND/OR) | *findByNameAndAge, findByNameOrEmail* |
| | *Is, Equals* | 같음(=) | *findByNameIs, findByNameEquals* |
| | *Between* | 범위(사이) | *findByAgeBetween* |
| | *LessThan, Before* | 미만(<) | *findByAgeLessThan* |
| | *GreaterThan, After* | 초과(>) | *findByAgeGreaterThan* |
| | *LessThanEqual* | 이하(<=) | *findByAgeLessThanEqual* |
| | *GreaterThanEqual* | 이상(>=) | *findByAgeGreaterThanEqual* |
| | *IsNull, IsNotNull* | 널/널이 아님 | *findByEmailIsNull, findByEmailIsNotNull* |
| | *In, NotIn* | 컬렉션 내 포함/불포함 | *findByStatusIn, findByAgeNotIn* |
| | *StartingWith* | 문자열로 시작하는(Like 'xxx%') | *findByNameStartingWith* |
| | *EndingWith* | 문자열로 끝나는(Like '%xxx') | *findByNameEndingWith* |
| | *Containing, Contains* | 포함(Like '%xxx%') | *findByNameContaining* |
| | *Like, NotLike* | Like, Not Like | *findByNameLike, findByNameNotLike* |
| | *True, False* | 불리언 값 | *findByActiveTrue* |
| 정렬 | *OrderBy* | 정렬 조건 | *findByNameOrderByAgeDesc* |
| 페이징 | *Pageable, Slice* | 페이징 파라미터 사용 | *findByName(String name, Pageable pageable)* |

# JPA 사용자 쿼리 메서드: @Query 어노테이션 (1/2)

- 복잡 조건, 조인, 집계(AVG, COUNT), 부분 컬럼 추출 등에 적합
- 동적 조건이 많으면 코드가 지저분해짐, 컴파일러 타입 체크 어려움
- 반복되는 복잡 쿼리는 QueryDSL/커스텀 리포지토리로 이전 고려

## JPQL(Java Persistence Query Language)

- JPA Entity 객체 명과 필드명 사용
- 특정 DB 종류(MySQL, Oracle, PostgreSQL 등)에 종속되지 않는 객체 지향 쿼리 작성 가능

```
// JPQL (엔티티 객체 대상으로 쿼리)
@Query("select u from User u where u.status = :status and u.age >= :minAge")
List<User> findActiveUsers(@Param("status") UserStatus status, @Param("minAge") int minAge);

// 조인 포함 (User, Order)
@Query("select u from User u join u.orders o where o.price > :minPrice")
List<User> findByOrderPriceGreaterThan(@Param("minPrice") int minPrice);

// 특정 컬럼만 추출
@Query("select u.name, u.email from User u where u.status = :status")
List<Object[]> findUserNamesAndEmails(@Param("status") UserStatus status);
```

# JPA 사용자 쿼리 메서드: @Query 어노테이션 (2/2)

### Native SQL(nativeQuery= true)

- 실제 DB의 테이블명 및 컬럼명
- DB 전용 함수, 윈도우 함수(OVER()), 복잡한 집계 쿼리 등 DB 특화 기능을 그대로 사용

```
@Query(value = "SELECT * FROM users u WHERE u.status = :status AND u.age >= :minAge", nativeQuery = true)
List<User> findActiveUsers(@Param("status") UserStatus status, @Param("minAge") int minAge);

// 조인 포함 (User, Order)
@Query(value = "SELECT DISTINCT u.* FROM users u " + "JOIN orders o ON u.id = o.user_id " +
               "WHERE o.price > :minPrice", nativeQuery = true)
List<User> findByOrderPriceGreaterThan(@Param("minPrice") int minPrice);

// 특정 컬럼만 추출
@Query(value = "SELECT u.name, u.email FROM users u WHERE u.status = :status", nativeQuery = true)
List<Object[]> findUserNamesAndEmails(@Param("status") UserStatus status);
```

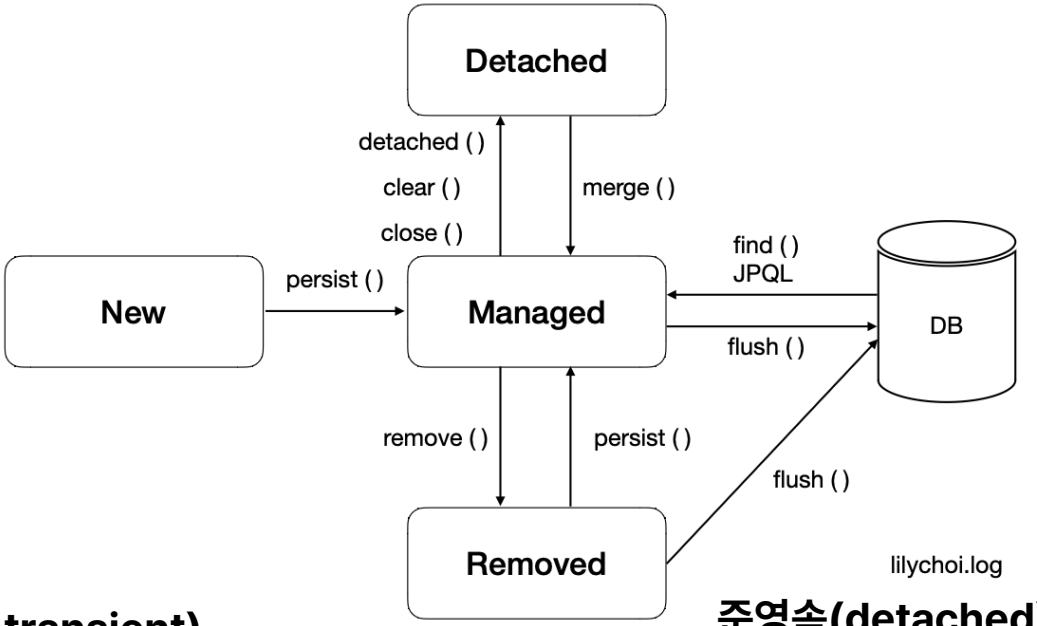
Native SQL을 사용시 Database 의존관계 발생하므로 가급적 사용 최소화 필요

# [참고] JPQL 주요 함수

| 분류 | 함수명 | 설명/용도 | 예시 코드/비고 |
|---|---|---|---|
| 집계함수 | *COUNT* | 행 개수 집계 | *select count(u) from User u* |
| | *SUM* | 합계 | *select sum(o.amount) from Order o* |
| | *AVG* | 평균 | *select avg(u.age) from User u* |
| | *MAX* | 최대값 | *select max(u.age) from User u* |
| | *MIN* | 최소값 | *select min(u.age) from User u* |
| 문자열 | *CONCAT* | 문자열 연결 | *select concat(u.firstName, ' ', u.lastName) from User u* |
| | *SUBSTRING* | 문자열 자르기 | *select substring(u.name, 1, 3) from User u* |
| | *LENGTH* | 문자열 길이 | *select length(u.name) from User u* |
| | *LOWER* | 소문자 변환 | *select lower(u.name) from User u* |
| | *UPPER* | 대문자 변환 | *select upper(u.name) from User u* |
| | *TRIM* | 앞뒤 공백 제거 | *select trim(u.name) from User u* |
| 날짜/시간 | *CURRENT_DATE* | 현재 날짜 | *select current_date from User u* |
| | *CURRENT_TIME* | 현재 시간 | *select current_time from User u* |
| | *CURRENT_TIMESTAMP* | 현재 날짜/시간 | *select current_timestamp from User u* |

# 영속성 컨텍스트 (Persistent Context)상태 관리

Entity를 저장하거나 검색하는 경우에는 영속화 (Persistent Context) 상태로 전환




**비영속(new/ transient)**

→ 순수 자바 객체 상태

예) Member member = new Member();

**영속(managed)**

→ 영속성 컨텍스트 메모리에 등록되고 관리되는 상태

예) 새 객체를 em.persist(entity) 할 때
em.find() 등을 통해 DB에서 데이터를 조회해 올 때

**준영속(detached)**

→ 생성된 Entity 객체가 영속성 컨텍스트에서 떨어져 나가 더이상 동기화되지 않는 상태

예) **em.detach(entity), em.clear()**
em.merge() 로 다시 managed 로 전환

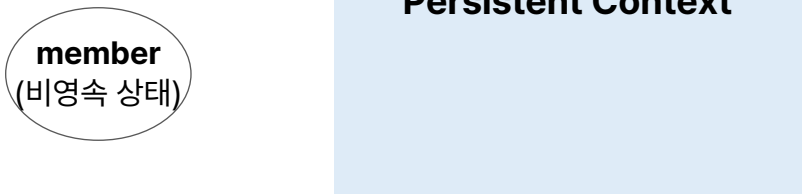
**삭제(removed)**

→ 삭제된 상태

예) em.remove(entity) 는 객체 삭제 표시

# 영속성 컨텍스트 (Persistent Context)상태 관리

### 비영속(new/ transient)




```
//Entity 객체를 생성한 상태(비영속)
Member member = new Member();
member.setId("member1");
member.setUsername("회원1");
```

### 영속(Managed)




```
//Entity 객체를 생성한 상태(비영속)
Member member = new Member();
member.setId("member1");
member.setUsername("회원1");

EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

//Entity 객체를 저장한 상태(영속)
em.persist(member);
```

### 준영속, 삭제

```
//회원 엔티티를 영속성 컨텍스트에서 분리, 준영속 상태
em.detach(member);

//객체를 삭제한 상태(삭제)
em.remove(member);
```

# [실습] Product Domain 구현하기

Product Entity를 추가해보자

**정의**

- Product Entity는 판매 상품 정보를 데이터베이스에 저장하고 조회하기 위한 JPA Entity
- products 테이블과 매핑하며, 상품명·가격·재고 수량을 관리
- 각 상품은 자동 생성되는 고유 ID로 식별

- 클래스명: Product
- 테이블명: products
- 기본키: id
- 속성:
  - 상품명 (name),
  - 가격 (price),
  - 재고 수량(stockQuantity),
  - 제품 상태 (status)
  - 제품 설명 (description)

제품 상태는 ProductStatus 를 표현하는 enum type :

- 판매중 (ON_SALE), 품절(SOLD_OUT), 단종(DISCONTINUED)

# [실습] Product Domain 구현하기

Product Domain 구성을 위한 클래스를 생성해보자

### 작성파일 목록 및 설명

| 파일 | 패키지 | 역할 |
|---|---|---|
| `ProductStatus.java` | `domain` | 상품 상태를 표현하는 Enum (판매중, 품절, 단종) |
| `Product.java` | `domain` | JPA 어노테이션이 적용된 엔티티 |
| `ProductRepository.java` | `repository` | JpaRepository 상속 인터페이스 |
| `ProductRequest.java` | `dto` | 클라이언트 요청 DTO |
| `ProductResponse.java` | `dto` | 클라이언트 응답 DTO |
| `ProductService.java` | `service` | 비즈니스 로직 |
| `ProductController.java` | `controller` | REST API 엔드포인트 |
| `data.sql` | `resources` | 애플리케이션 시작 시 자동으로 실행되는 초기 데이터 |

# [실습] Product Domain 구현하기

### domain/ProductStatus.java

```
/**
 * 상품 상태를 나타내는 Enum
 * - @Enumerated(EnumType.STRING) 실습에 사용
 */
public enum ProductStatus {
    ON_SALE,    // 판매중
    SOLD_OUT,   // 품절
    DISCONTINUED // 단종
}
```

### repository/ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    // 상태별 상품 목록 조회 (쿼리 메서드)
    List<Product> findByStatus(ProductStatus status);
}
```

# [실습] Product Domain 구현하기

## domain/Product.java

```
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT
    private Long id;

    @Column(name = "product_name", nullable = false, length = 100)
    private String name;                                // 상품명 (최대 100자, NOT NULL)

    @Column(nullable = false)
    private Integer price;                              // 가격 (NOT NULL)

    @Column(name = "stock_quantity", columnDefinition = "INT DEFAULT 0")
    private Integer stockQuantity;                      // 재고 수량 (기본값 0)

    @Enumerated(EnumType.STRING)                        // DB에 "ON_SALE" 같은 문자열로 저장
    @Column(nullable = false)
    private ProductStatus status;                       // 상품 상태 (ON_SALE / SOLD_OUT / DISCONTINUED)

    @Lob
    @Column(columnDefinition = "TEXT")
    private String description;                         // 상품 상세 설명 (대용량 텍스트)

    @Transient
    private String displayLabel;                        // "상품명 (상태)" 형태의 표시용 라벨, DB에 저장 안 됨

    public String getDisplayLabel() {
        return name + " (" + (status != null ? status.name() : "N/A") + ")";
    }
}
```

# [실습] Product Domain 구현하기

## service/ProductService.java

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // [전체 상품 목록 조회]
    public List<Product> getAllProducts() {}

    // [상품 단건 조회]
    public Optional<Product> getProductById(Long id) {}

    // [상태별 상품 목록 조회]
    public List<Product> getProductsByStatus(ProductStatus status) {}

    // [신규 상품 등록]
    public Product createProduct(Product product) {}

    // [기존 상품 정보 수정]
    public Optional<Product> updateProduct(Long id, Product updated) {}

    // [상품 삭제]
    public void deleteProduct(Long id) {}
}
```

# [실습] Product Domain 구현하기

## ProdcutController.java

**요구 사항**

- 기본 경로는 `/api/products`
- ProductRequest → Product 변환, Product → ProductResponse 변환 헬퍼 메서드를 만든다
- 아래 6개 엔드포인트를 구현한다

| Method | URL | 설명 |
|---|---|---|
| GET | `/api/products` | 전체 조회 |
| GET | `/api/products/{id}` | 단건 조회 |
| GET | `/api/products/status?value=ON_SALE` | 상태별 조회 |
| POST | `/api/products` | 등록 |
| PUT | `/api/products/{id}` | 수정 |
| DELETE | `/api/products/{id}` | 삭제 |

# [실습] Product Domain 구현하기

## data.sql 초기 데이터

```
INSERT INTO products (product_name, price, stock_quantity, status, description)
  VALUES ('노트북',      1500000, 10, 'ON_SALE',      '고성능 개발용 노트북입니다.');

INSERT INTO products (product_name, price, stock_quantity, status, description)
  VALUES ('무선 마우스',   35000, 50, 'ON_SALE',      '2.4GHz 무선 마우스입니다.');

INSERT INTO products (product_name, price, stock_quantity, status, description)
  VALUES ('기계식 키보드', 120000,  0, 'SOLD_OUT',    '청축 기계식 키보드입니다.');

INSERT INTO products (product_name, price, stock_quantity, status, description)
  VALUES ('27인치 모니터',350000,  5, 'ON_SALE',      'QHD 해상도 모니터입니다.');

INSERT INTO products (product_name, price, stock_quantity, status, description)
  VALUES ('USB 허브',     25000,  0, 'DISCONTINUED', '단종된 4포트 USB 허브입니다.');
```



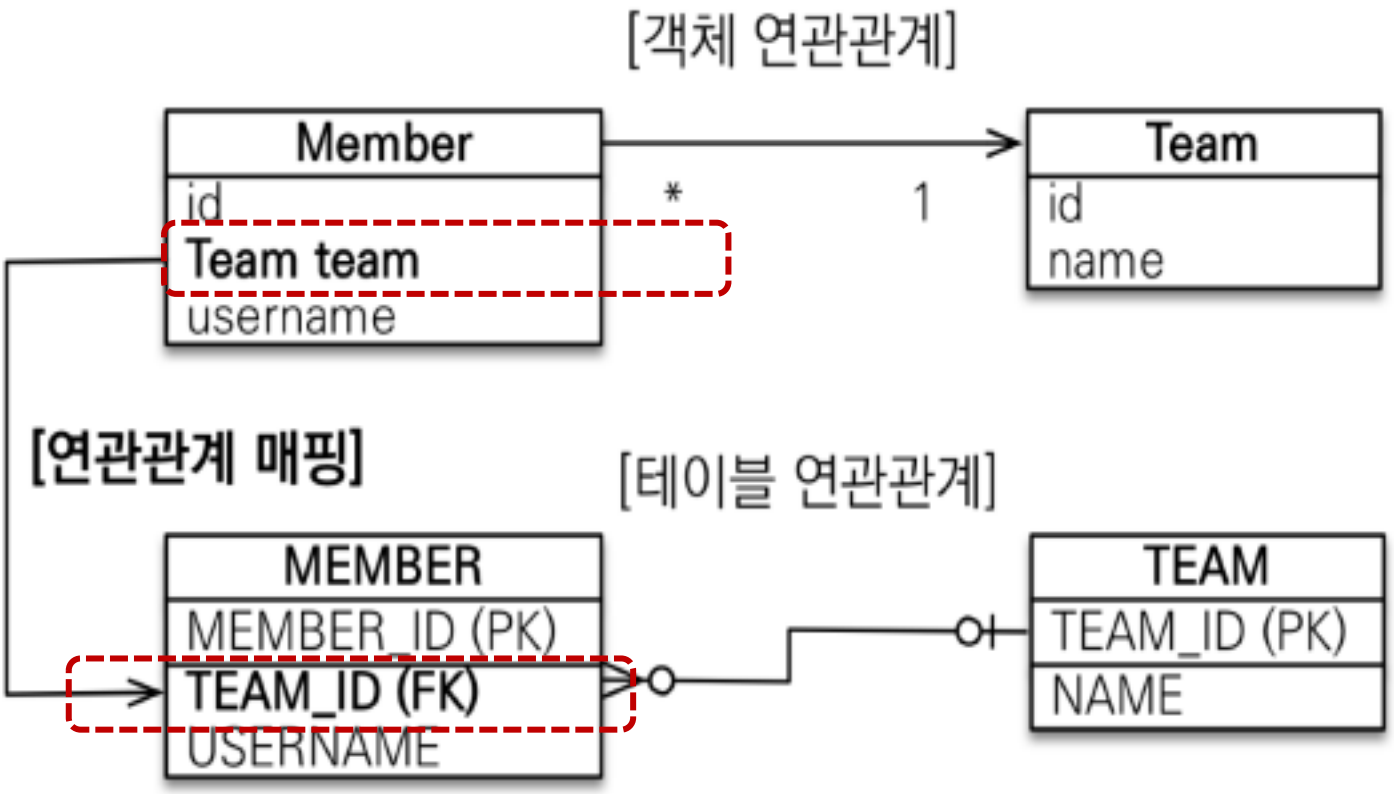
# JPA 연관 관계 매핑

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

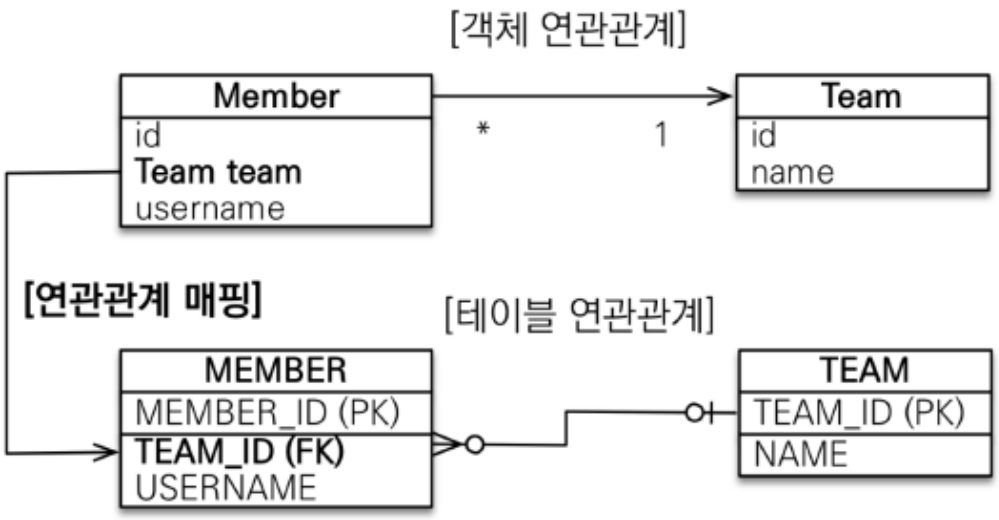
SK 주식회사 C&C

# 관계 연관 매핑

객체 지향 언어의 객체 참조와 관계형 데이터베이스의 외래키(FK)간의 차이를 연결해 주는 과정



# 관계 매핑 : ManyToOne




```java
@Entity
public class Member {
    @Id @GeneratedValue
    private Long id;

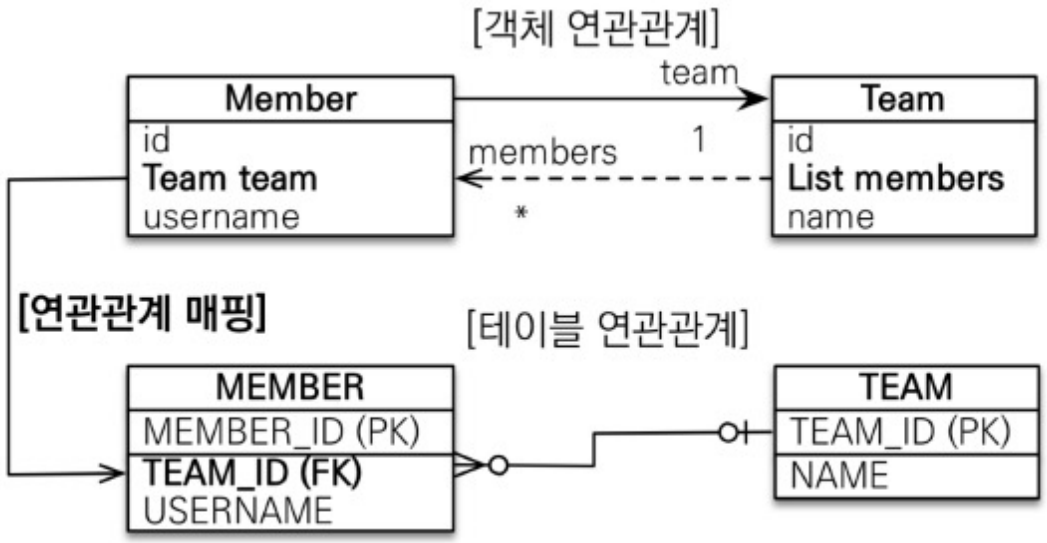
    private String username;

    @ManyToOne
    @JoinColumn(name = "TEAM_ID")
    private Team team;
}
```

```java
@Entity
public class Team {
    @Id @GeneratedValue
    @Column(name = "TEAM_ID")
    private Long id;

    private String name;
}
```

# 관계 매핑 : OneToMany




```java
@Entity
public class Team {

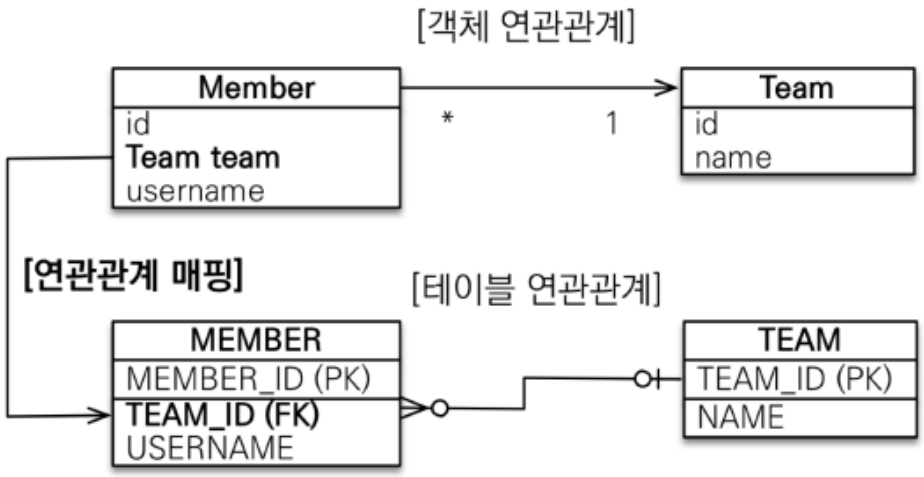
    @Id @GeneratedValue
    @Column(name = "TEAM_ID")
    private Long id;

    private String name;

    @OneToMany(mappedBy = "team")
    private List<Member> members = new ArrayList<>();
}
```

# FK 제어권에 대한 이해

TEAM 과 MEMBER 간 연관관계는 FK를 가지고 있는 MEMBER가 제어(변경) 할 수 있음




**FK의 제어권으로 인해**

- 객체의 연관관계의 변경은 Member에서만 수정
- **@ManyToOne**에서만 연관관계 객체를 변경 가능

**TEAM Entity가 연관관계를 변경할 수 없음**

- Team 객체가 연관관계를 변경할 수 없음
- @OneToMany의 members를 교체할 수 없음

```java
@Entity
public class Team {

    @Id @GeneratedValue
    @Column(name = "TEAM_ID")
    private Long id;

    private String name;

    @OneToMany(mappedBy = "team")
    private List<Member> members = new ArrayList<>();
}
```

```java
@Entity
public class Member {
    @Id @GeneratedValue
    private Long id;

    private String username;

    @ManyToOne
    @JoinColumn(name = "TEAM_ID")
    private Team team;
}
```

# JPA 어노테이션 - 관계 매핑

- 객체 간의 관계를 데이터베이스 테이블의 외래 키(FK)와 연결하는 방법을 정의
- 다중성(1:1, 1:N, N:1, N:M)과 방향성(단방향, 양방향)을 조합하여 설정

| 어노테이션 | 관계 | 대표 예시 |
|---|---|---|
| @ManyToOne | N:1 | 여러 거래 → 하나의 주식 |
| @OneToMany | 1:N | 하나의 사용자 → 여러 거래 |
| @OneToOne | 1:1 | 유저 ↔ 프로필 |
| @ManyToMany | N:M | 유저 ↔ 관심종목(주식) |
| @JoinColumn | 외래키 지정 | 거래에 user_id, ticker 등 |
| @JoinTable | 중간테이블 | user_watchlist(user_id + ticker) |

연관관계의 주인은 FK가 있는 쪽이 됨

- @OneToMany(mappedBy = “user”) 라면 추가/삭제/변경의 주체는 @ManyToOne을 가진 Entity
- SQL 은 항상 주체 쪽 변경을 기준으로 생성

# FetchType.LAZY or EAGER

## @OneToMany는 fetch.LAZY가 default, @ManyToOne은 fetch.EAGER 가 default

```
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT
    private Long id;

    @Column(name = "product_name", nullable = false, length = 100)
    private String name;                                  // 상품명 (최대 100자, NOT NULL)
…
    @Lob
    @Column(columnDefinition = "TEXT")
    private String description;                           // 상품 상세 설명 (대용량 텍스트)

    @ManyToOne(fetch = FetchType.LAZY)                   // N:1 관계 - 여러 상품이 한 명의 사용자에 속함
    @JoinColumn(name = "user_id")                         // products 테이블에 user_id FK 컬럼 생성
    private User user;                                    // 이 상품을 등록한 사용자
```

## FetchType.LAZY 처리

```
em.getTransaction().begin();

Product p = em.find(Product.class, 1L);

User u = p.getUser();        // 아직 쿼리 안 나감(프록시 반환)
String name = u.getName();       // 이때 초기화

em.getTransaction().commit();
```

### FetchType.LAZY 인경우

Product를 조회 시 user는 즉시 쿼리로 가져오지 않고 Proxy로 user을 채운다.
getName()에 접근하는 순간 별도 SELECT가 나가며 초기화

# [실습] ManyToOne 관계 추가하기

현재 User와 Product은 서보 독립적인 엔티티
이번 실습에서는 "상품은 한명의 사용자에게 속한다"는 N:1 관계를 코드에 반영해봅니다

- @ManyToOne 어노테이션으로 두 엔티티 간 N:1 연관관계를 설정할 수 있다.
- @JoinColumn으로 FK 컬럼명을 지정하는 방법을 이해한다.
- 연관관계 엔티티를 DTO 변환 과정에서 올바르게 처리할 수 있다.
- Service 계층에서 연관 엔티티를 조회하고 연결하는 흐름을 구현할 수 있다.

```
Product (N) ──────────────► User (1)
  여러 상품이 한 명의 사용자에 속함
```

### 작성파일 목록 및 설명

| 파일 | 역할 |
|---|---|
| `domain/Product.java` | `@ManyToOne`으로 User 참조 필드 추가 |
| `dto/ProductRequest.java` | 상품 등록/수정 시 사용자 ID를 받는 필드 추가 |
| `dto/ProductResponse.java` | 응답에 사용자 ID와 이름 포함 |
| `controller/ProductController.java` | DTO 변환 메서드에 User 정보 처리 추가 |
| `resources/data.sql` | 초기 데이터에 `user_id` 컬럼 추가 |

# [실습] ManyToOne 관계 추가하기

현재 User와 Product은 서보 독립적인 엔티티
이번 실습에서는 "상품은 한명의 사용자에게 속한다"는 N:1 관계를 코드에 반영해봅니다

- @ManyToOne 어노테이션으로 두 엔티티 간 N:1 연관관계를 설정할 수 있다.
- @JoinColumn으로 FK 컬럼명을 지정하는 방법을 이해한다.
- 연관관계 엔티티를 DTO 변환 과정에서 올바르게 처리할 수 있다.
- Service 계층에서 연관 엔티티를 조회하고 연결하는 흐름을 구현할 수 있다.

```
Product (N) ──────────────► User (1)
  여러 상품이 한 명의 사용자에 속함
```

### 작성파일 목록 및 설명

| 파일 | 역할 |
|---|---|
| `domain/Product.java` | `@ManyToOne`으로 User 참조 필드 추가 |
| `dto/ProductRequest.java` | 상품 등록/수정 시 사용자 ID를 받는 필드 추가 |
| `dto/ProductResponse.java` | 응답에 사용자 ID와 이름 포함 |
| `controller/ProductController.java` | DTO 변환 메서드에 User 정보 처리 추가 |
| `resources/data.sql` | 초기 데이터에 `user_id` 컬럼 추가 |

# [실습] ManyToOne 관계 추가하기

## Product.java

Product가 User를 참조하도록 연관관계 필드를 추가합니다.

- @ManyToOne을 사용하고 fetch 전략은 LAZY로 설정합니다.
- @JoinColumn으로 FK 컬럼명을 user_id로 지정합니다.
- 필드명은 user이고 타입은 User입니다.

## ProductRequest.java

API 요청 바디에서 사용자 ID를 받을 수 있도록 필드를 추가합니다.

- 필드 타입은 Long, 이름은 userId
- 필드 타입은 String, 이름은 name

## ProductResponse.java

응답 DTO에 연관된 사용자 정보를 포함합니다.

- userId : 필드 타입 Long, 상품을 등록한 사용자 ID
- userName : 필드 타입 String

## ProductRepository.java

@ManyToOne으로 연결된 User 필드를 활용해 두 가지 쿼리 메서드를 추가합니다.

- findByUserId : user_id FK 값으로 상품 목록을 조회합니다.
- findByUserName : 연관된 User의 name 필드 값으로 상품 목록을 조회합니다.

# [실습] Product 수정

Product Entity에 ManyToOne을 추가

```
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT
    private Long id;

    @Column(name = "product_name", nullable = false, length = 100)
    private String name;                                 // 상품명 (최대 100자, NOT NULL)

…
    @ManyToOne(fetch = FetchType.LAZY)                  // N:1 관계 - 여러 상품이 한 명의 사용자에 속함
    @JoinColumn(name = "user_id")                        // products 테이블에 user_id FK 컬럼 생성
    private User user;                                   // 이 상품을 등록한 사용자

    @Transient
    private String displayLabel;                         // "상품명 (상태)" 형태의 표시용 라벨, DB에 저장 안 됨

    /**
     * DB에서 조회한 뒤 displayLabel을 조합해 반환하는 편의 메서드
     * @Transient 필드는 이처럼 런타임에 계산되는 값에 활용
     */
    public String getDisplayLabel() {
        return name + " (" + (status != null ? status.name() : "N/A") + ")";
    }
}
```

# [실습] Product Request / Product Response 에 user 추가

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class ProductRequest {

    @NotBlank(message = "상품명은 필수입니다")
    private String name;

    @NotNull(message = "가격은 필수입니다")
    @Min(value = 0, message = "가격은 0 이상이어야 합니다")
    private Integer price;

    @Min(value = 0, message = "재고는 0 이상이어야 합니다")
    private Integer stockQuantity;

    @NotNull(message = "상품 상태는 필수입니다")
    private ProductStatus status;

    private String description;

    private Long userId;
    private String userName:
}
```

```
public class ProductResponse {

    private Long id;
    private String name;
    private Integer price;
    private Integer stockQuantity;
    private ProductStatus status;
    private String description;
    private String displayLabel;  // @Transient 필드 – 런
타임 계산값 노출
    private Long userId;          // 등록한 사용자 ID
    private String userName;      // 등록한 사용자 이름
}
```

# [실습] Product Repository 수정

User ID 또는 Name으로 사용자별 상품 목록 조회하기

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    // 상태별 상품 목록 조회 (쿼리 메서드)
    List<Product> findByStatus(ProductStatus status);

    // 사용자별 상품 목록 조회 (@ManyToOne 연관관계 활용)
    List<Product> findByUserId(Long userId);

    // 사용자 이름으로 상품 목록 조회 (연관 엔티티 필드 탐색)
    List<Product> findByUserName(String userName);
}
```

# [실습] ProductService 수정

userId로 상품 등록, 상품 수정, 상품 삭제되도록 수정

```
    // userId 기반으로 상품 목록 검색
    public List<Product> getProductsByUserId(Long userId) {
        return productRepository.findByUserId(userId);
    }


    // 사용자 이름으로 상품 목록 조회
    public List<Product> getProductsByUserName(String userName) {
        return productRepository.findByUserName(userName);
    }
```

# [실습] Product Controller

**API 추가**

userId와 userName으로 제품 목록을 검색하는 API 를 추가해보자

```
    // GET /api/products/user?userId=123 – userId 기반 상품 목록 조회
    @GetMapping(value = "/user", params = "userId")
    public List<ProductResponse> getProductsByUserId(@RequestParam Long userId) {
        return productService.getProductsByUserId(userId).stream()
                .map(this::toResponse)
                .toList();
    }

    // GET /api/products/user?name=홍길동 – 사용자 이름으로 상품 목록 조회
    @GetMapping(value = "/user", params = "name")
    public List<ProductResponse> getProductsByUserName(@RequestParam String name) {
        return productService.getProductsByUserName(name).stream()
                .map(this::toResponse)
                .toList();
    }
```

# [실습] data.sql

기존 products INSERT 문에 user_id 컬럼을 추가합니다.

```
INSERT INTO products (product_name, price, stock_quantity, status, description, user_id)
VALUES ('노트북', 1500000, 10, 'ON_SALE', '고성능 개발용 노트북입니다.', 1 /* userId */);

INSERT INTO products (product_name, price, stock_quantity, status, description, user_id)
VALUES ('무선 마우스', 35000, 50, 'ON_SALE', '2.4GHz 무선 마우스입니다.', 1 /* userId */);

INSERT INTO products (product_name, price, stock_quantity, status, description, user_id)
VALUES ('기계식 키보드', 120000, 0, 'SOLD_OUT', '청축 기계식 키보드입니다.', 2 /* userId */);

INSERT INTO products (product_name, price, stock_quantity, status, description, user_id)
VALUES ('27인치 모니터', 350000, 5, 'ON_SALE', 'QHD 해상도 모니터입니다.', 2 /* userId */);

INSERT INTO products (product_name, price, stock_quantity, status, description, user_id)
VALUES ('USB 허브', 25000, 0, 'DISCONTINUED', '단종된 4포트 USB 허브입니다.', 3 /* userId */);
```

# [실습] 접속해서 확인하기

아래 Path로 접속해서 동작을 확인해보자

| Method | URL | 설명 |
|---|---|---|
| GET | /api/products | 전체 조회 |
| GET | /api/products/{id} | 단건 조회 |
| GET | /api/products/status?value=ON_SALE | 상태별 조회 |
| GET | /api/products/user?userId=123 | userId 기반 상품 목록 조회 |
| GET | /api/products/user?name=홍길동 | 사용자 이름으로 상품 목록 조회 |
| POST | /api/products | 등록 |
| PUT | /api/products/{id} | 수정 |
| DELETE | /api/products/{id} | 삭제 |



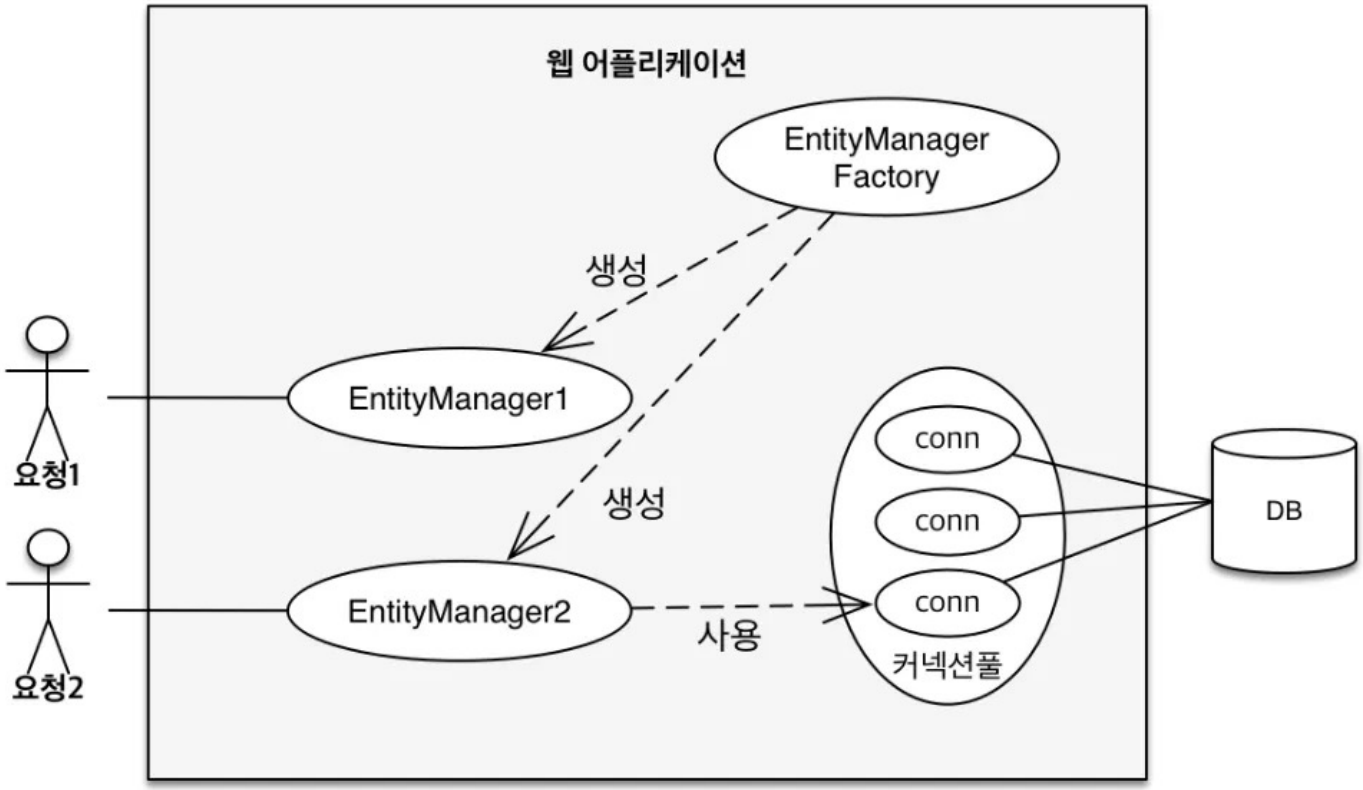
# EntityManager

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# EntityManager

- JPA에서 엔티티(객체)를 DB와 동기화하기 위해 사용하는 핵심 관리 객체
- 엔티티의 생성, 조회, 수정, 삭제(CRUD)와 트랜잭션, 영속성 컨텍스트 관리 등을 수행
- JPA가 DB와 대화하는 모든 기능의 중심이 **EntityManager**




웹 어플리케이션이 구동하는 시점에 EntityManagerFactory 를 생성하여 가지고 있으며,
사용자의 요청이 있을 때 EntityManager를 생성하여 커넥션 풀(Connection Pool) 을 사용해서 DB를 핸들링

# EntityManager의 주요 메소드

Spring Boot + JPA(Hibernate) 에서 가장 자주 쓰는 메서드는 find, persist, merge, remove, flush, clear

| 분류 | 메서드 | 설명 |
|---|---|---|
| 조회 (Read) | **find(Class<T> entityClass, Object primaryKey)** | PK 기반 즉시 조회 (1차 캐시 먼저 확인) |
| | getReference() | 실제 사용 시까지 프록시 – 지연 로딩 |
| | createQuery(String qlString) | JPQL 기반 조회 |
| | createNamedQuery(String name) | @NamedQuery 조회 |
| | createNativeQuery(String sqlString) | 네이티브 SQL 조회 |
| 저장/변경 (Write) | **persist(Object entity)** | 신규 엔티티 영속화 (INSERT 예정) |
| | **merge(Object entity)** | Detached 객체(일반객체)를 다시 영속상태로 병합 |
| | **remove(Object entity)** | 영속 엔티티 제거(DELETE 예정) |
| | **flush()** | 영속성 컨텍스트 변경 내용을 DB에 즉시 반영 |
| | **clear()** | 영속성 컨텍스트 초기화 (1차 캐시 제거) |
| 트랜잭션/ 상태 관리 | contains(Object entity) | 해당 객체가 영속 상태인지 여부 |
| | detach(Object entity) | 특정 엔티티만 영속성 컨텍스트에서 분리 |
| | setFlushMode(FlushModeType) | flush 시점 제어(COMMIT, AUTO) |
| 동시성 관련 | lock(Object entity, LockModeType lockMode) | 낙관/비관적 Lock 적용 |
| 저장소 접근 | unwrap(Class<T>) | 하위 구현체(ex: Hibernate Session) 얻기 |
| | getEntityManagerFactory() | 연결된 EMF 조회 |
| 기타 | getTransaction() | 리소스 로컬 트랜잭션 획득<br>(JTA 환경에서는 사용 X) |

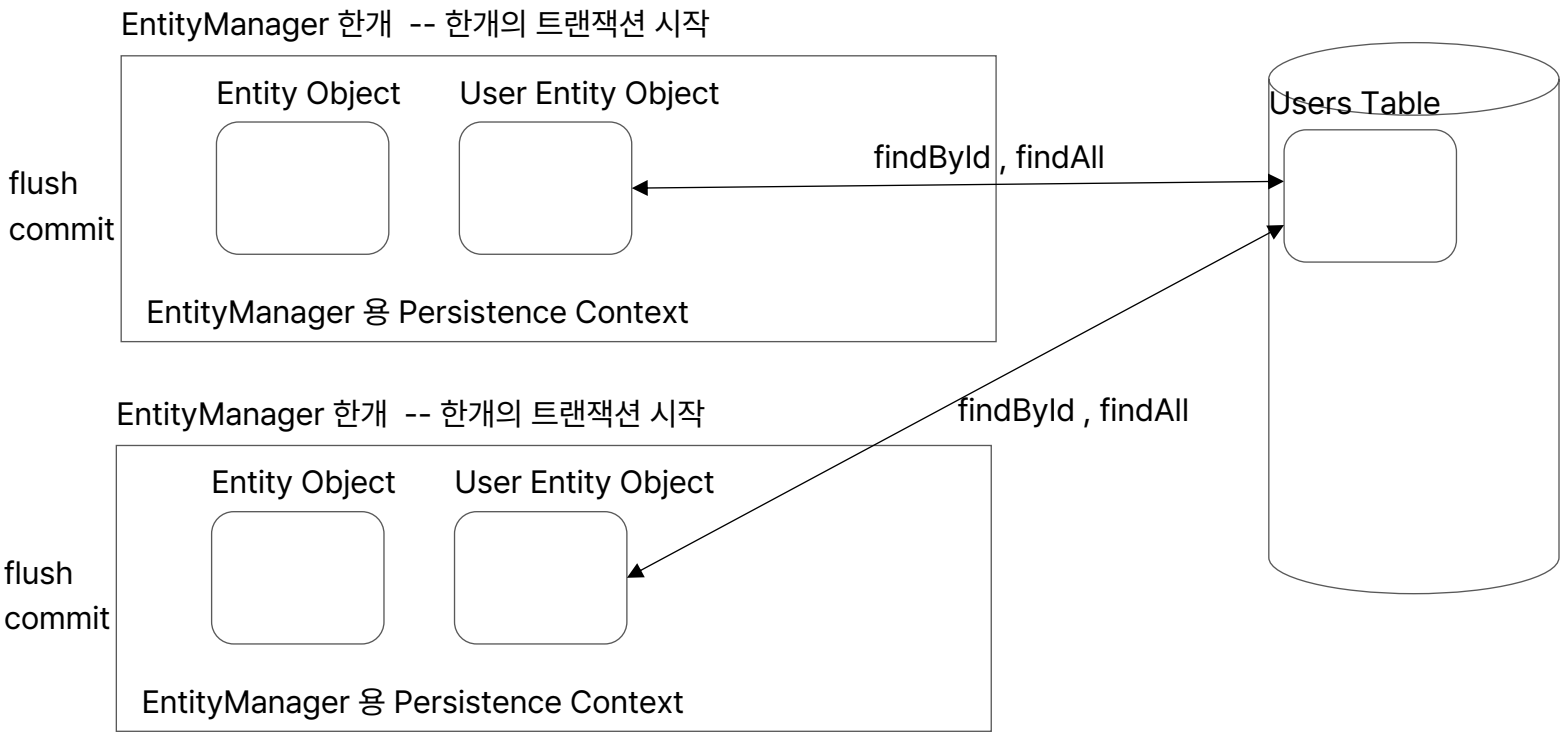
# Entity Manager vs JpsRepository

JpaRepository는 **EntityManager를 더 쓰기 쉽게 감싸놓은 상위 수준의 추상화 도구**
내부적으로 JpaRepository는 결국 EntityManager를 사용해 동작

| 구분 | EntityManager (JPA 기본) | JpaRepository (Spring Data JPA) |
|---|---|---|
| **정의** | JPA 표준 스펙의 핵심 인터페이스 | Spring이 JPA를 쉽게 쓰도록 만든 추상화 라이브러리 |
| **작동 레벨** | 로우 레벨 (데이터베이스와 직접 소통) | 하이 레벨 (개발자 친화적 인터페이스) |
| **구현 방식** | 개발자가 직접 메서드를 호출하고 쿼리 작성 | 인터페이스 선언만으로 Spring이 알아서 구현체 생성 |
| **코드 양** | 상대적으로 많음 (보일러플레이트 코드 발생) | 매우 적음 (메서드 이름만으로 쿼리 생성) |

# 영속성 컨텍스트 (Persistent Context)

- EntityManager는 하나의 트랜잭션 별로 Persistent Context 를 생성 관리
- @Transactional을 사용하여 Persistent Context 활성화




영속성 컨텍스트(Persistence Context)는

**엔티티 객체를 1차 캐시로 보관**하여,

**해당 엔티티와 데이터베이스 레코드를 동기화**하기 위해 **상태를 추적·관리하는 메모리 공간**

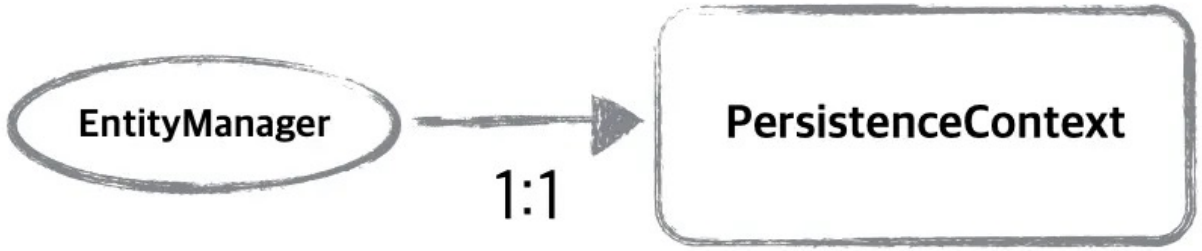
# 영속성 컨텍스트 (Persistent Context) 역할

JPA의 작업장을 모아두는 1차 캐시 + 변경 감지 시스템

| 기능 | 설명 |
|---|---|
| **1차 캐시** | 같은 엔티티를 다시 조회하면 DB hit 없이 캐시에서 반환 |
| **엔티티 동일성 보장** | 같은 PK 조회 시 항상 동일 객체(== true) |
| **변경 감지(Dirty Checking)** | 엔티티 변경을 자동 감지 → UPDATE SQL 자동 실행 |
| **지연 로딩(Lazy Loading)** | 필요할 때만 연관 엔티티 로딩 |
| **쓰기 지연(Write-behind)** | SQL을 모아서 트랜잭션 끝에서 한 번에 flush |

### EntityManager ↔ Persistence Context 관계

- EntityManager는 단순한 컨트롤러
- Persistence Context는 실제 엔티티를 저장하고 관리하는 공간



# [참고] JPARepository 와 EntityManager 관계

JPARepository는 인터페이스만 선언해도 자동으로 구현체가 생성되는데 Proxy pattern으로 EntityManager를 생성하여 호출하는 구조로 지원

1. UserRepository.save(user) --> (프록시 객체, 동적 구현체)



proxy를 통해 기본 JpaRepository 호출

2. SimpleJpaRepository.save(user) --> (Spring Data JPA 기본 구현체)



EntityManager.persist 호출

3. EntityManager.persist(user) --> (JPA 표준 Inerface 정의한 구현체)



Hibernate의 Persistence Context 저장 요청

4. Hibernate: session.persist(user)|flush|commit --> (Hibernate의 Persistence Context 저장)



JDBC API 호출

5. JDBC: PreparedStatement.executeUpdate() --> (JDBC 드라이버)



SQL 실행 요청

6. Database: (INSERT INTO user VALUES (...)) --> (DB에 SQL 실행)

# EntityManager 사용 예시

모든 Method에 @Transactional을 설정

```
@Service
@Transactional   // Class Level 적용
public class UserService {

    @PersistenceContext
    private EntityManager em;   // EntityManager가 아니라 Proxy EntityManager

    public void saveUser() {
        User user = new User("홍길동");

        em.persist(user); // INSERT 실행 (트랜잭션 commit 시점)
    }

    public User getUser(Long id) {
        return em.find(User.class, id);  // SELECT 실행
    }

    public void deleteUser(Long id) {
        User user = em.find(User.class, id);
        em.remove(user); // DELETE 실행
    }
}
```



# JPA Transactional

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# JPA 트랜잭션 (Transaction)

- 트랜잭션은 데이터베이스에서 일련의 작업들을 하나의 논리적 단위로 묶어서 처리하는 것
- 모든 작업이 성공해야만 실제로 DB에 반영되고, 중간에 오류가 발생하면 전체 작업이 취소(롤백) 됨
- JPA의 변경 내용(저장/수정/삭제)은 트랜잭션이 있을 때만 데이터베이스에 안전하게 반영됨

| 트랜잭션의 원칙 (ACID) | 설명 |
|---|---|
| Atomicity(원자성) | 모두 반영되거나 모두 반영되지 않아야 한다.(all or Nothing) |
| Consistency(일관성) | 트랜잭션 전후로 데이터가 일관된 상태 유지해야 한다. |
| Isolation(격리성) | 여러 트랜잭션이 동시에 실행돼도 서로 영향 없어야 한다. |
| Durability(지속성) | 커밋된 결과는 영구적으로 저장되어야 한다. |

❖ **JPA에서는 서비스 계층에서 트랜잭션을 시작** – 스프링은 **@Transactional** 어노테이션을 사용해 트랜잭션을 자동 관리

# @Transactional

- 트랜잭션 관리를 지원하는 선언적 어노테이션
- 메서드(또는 클래스)에 붙이면, 해당 범위 내의 모든 데이터베이스 작업을 트랜잭션 단위로 처리
- 모든 작업이 성공하면 커밋(commit), 중간에 예외가 발생하면 자동으로 롤백(rollback) 처리됨

**속성 설정 방식** org.springframework.transaction.annotation.Transactional을 사용해야 함.

```
@Service
@Transactional(readOnly = true)    // class Level 적용
public class UserService {

    // 1. 위의 클래스 레벨 설정을 그대로 이어받아 readOnly = true로 동작 (조회 전용)
    public UserProfile getUserProfile(Long userId) {
        return userRepository.findById(userId).orElseThrow();
    }

    // 2. 쓰기 작업이 필요한 메서드는 readOnly = false(기본값)로 덮어쓰기
    @Transactional
    public void registerUser(UserDto userDto) {
        userRepository.save(userDto.toEntity());
    }
```

**readOnly=true 일때 특징**

- 변경 감지, 추적을 수행하지 않기 때문에 성능 향상
- write 가 원천적으로 차단
- Master-Slave 분기 시 Slave DB 커넥션을 맺도록 부하 분산

# @Transactional 사용 케이스

- DB 변경이 있는 오퍼레이션을 수행하는 서비스 메서드에는 @Transactional을 꼭 사용하는 것이 권장
- @Transactional을 너무 남발하면 트랜잭션 범위가 넓어져 DB lock, 동시성 저하 등 성능 문제가 생길 수 있음

| 케이스 | 트랜잭션 적용 방법 |
|---|---|
| **데이터 변경(insert/update/delete)** | **반드시 @Transactional 사용** |
| 단순 조회(select) | @Transactional(readOnly = true) |
| 외부 API/파일 등 장시간 IO 포함 | **트랜잭션 범위를 DB 오퍼레이션만 포함하도록 최대한 좁게 유지** |

@Transactional을 붙이지 않으면 save 시 commit 이 일어나지 않나?

JpaRepository 를 사용하는 경우 기본적으로 save 호출 시 @Transactional을 method에 붙이고 있음
하지만 원자성 및 성능 저하 /비효율 발생

# [참고] @Transactional 설정 범위 예시

```java
@Service
public class FileService {

    @Transactional
    public void processAndSave(User user, MultipartFile file) throws IOException {

        // 1. 파일 저장 (네트워크/디스크 IO) - 작업 시간이 길 수 있음
        String filePath = saveFileToDisk(file);

        // 2. DB 저장 (트랜잭션)
        user.setProfileImagePath(filePath);
        userRepository.save(user);

        // 3. 외부 API 호출 (트랜잭션) - 네트워크 작업
        notifyExternalSystem(user);
    }
}
```

**트랜잭션 범위가 너무 넓음**

```java
@Service
public class FileService {

    public void processAndSave(User user, MultipartFile file) throws IOException {
        String filePath = saveFileToDisk(file);    // 파일 저장
        saveUserProfileImagePath(user, filePath);  // DB 저장
        notifyExternalSystem(user);                // 외부 API 호출
    }

    @Transactional  // DB 오퍼레이션에만 트랜잭션 적용
    public void saveUserProfileImagePath(User user, String filePath) {
        user.setProfileImagePath(filePath);
        userRepository.save(user);
    }
}
```

**트랜잭션 범위를 DB 작업에 한정**

→ 트랜잭션은 AOP Proxy를 통해 호출

→ 그러므로 내부 사용 함수지만 public 선언

# JPA 트랜잭션 커밋(Commit) 시점

- JPA에서 트랜잭션이 커밋 되는 시점은 기본적으로 트랜잭션을 시작한 메서드가 성공적으로 종료될 때
- 스프링 환경에서는 @Transactional 어노테이션이 붙은 메서드가 **예외 없이** 실행을 마치는 시점에 커밋 발생

❖ **@Transactional 어노테이션**

- 스프링에서 **서비스 클래스의 메서드에 @Transactional을 붙이면**, 스프링 컨테이너는 해당 서비스 객체 대신 트랜잭션 기능이 포함된 **프록시(Proxy) 객체를 생성하여 빈(Bean)으로 등록** - 서비스 메서드를 호출하면 실제로는 이 프록시 객체의 메서드가 먼저 호출됨.
- **메서드 시작 시점**: 프록시는 데이터베이스 트랜잭션을 시작(BEGIN)
- **메서드 종료 시점**: 메서드가 정상적으로 종료되면, 프록시는 트랜잭션을 커밋(COMMIT) 하고, 메서드 실행 중 예외(기본적으로 RuntimeException)가 발생하면, 트랜잭션을 롤백(ROLLBACK) 처리

❖ **영속성 컨텍스트(Persistence Context)와 쓰기 지연**

- 트랜잭션이 시작되면, JPA는 **영속성 컨텍스트라는 논리적인 영역을 생성** : 이곳은 엔티티(Entity)를 임시로 저장하고 관리하는 일종의 1차 캐시 이자 작업 공간
- **repository.save(entity)를 호출**하면, **엔티티는 데이터베이스에 바로 저장되는 것이 아니라 영속성 컨텍스트에 저장**되고, 실행해야 할 SQL(INSERT, UPDATE, DELETE 등)은 **쓰기 지연 SQL 저장소에 쌓이게 됨**.
- 이렇게 **SQL을 바로 실행하지 않고 모아두는 것을 쓰기 지연(Write-Behind)** 이라고 함.

# 트랜잭션 전파 (Propagation)

- 트랜잭션이 이미 존재할 때, 현재 실행 중인 메서드에서 트랜잭션을 어떻게 처리할지를 결정하는 옵션
  → 기존 트랜잭션에 참여할지, 새로운 트랜잭션을 생성할지, 아니면 트랜잭션 없이 실행할지 선택

| 옵션명 | 설명 |
| --- | --- |
| **REQUIRED (기본값)** | **기존 트랜잭션이 있으면 참여, 없으면 새로 생성**<br>- 대부분의 서비스 로직 |
| **REQUIRES_NEW** | **항상 새로운 트랜잭션 생성, 기존 트랜잭션은 일시 중단(suspend), 별도의 영속성 관리**<br>- A 트랜잭션의 성공/실패와 무관하게, B 작업을 반드시 독립적으로 커밋시키고 싶을때,<br>- 로그 기록, 감사(audit) 등 |

# 트랜잭션 전파 오류 – Self Invocation 문제

- 하나의 클래스 내에서 @Transactional이 붙은 다른 메소드를 호출하면 트랜잭션 전파 설정이 적용되지 않음.

← 프록시를 거치지 않고 실제 객체의 내부 메소드를 직접 호출하기 때문

```
@Service
public class UserService {
    @Transactional  // (propagation = Propagation.REQUIRED) 기본값
    public void outerMethod() {
        // DB 작업 1
        innerMethod();
    }

    @Transactional (propagation = Propagation.REQUIRES_NEW)
    public void innerMethod() {
        // DB 작업 2
    }
}
```

❖ **해결 방안** : 트랜잭션 단위가 다른 메소드는 **별도의 클래스(Bean)로 분리**하여 주입받아 사용

# 트랜잭션 전파 오류 – 해결 방안

- 트랜잭션 단위가 다른 메소드는 별도의 클래스(Bean)로 분리하여 주입받아 사용

```
@Service
public class OuterService {
    @Autowired
    private InnerService innerService;

    @Transactional // (propagation = Propagation.REQUIRED)
    public void outerMethod() {
        // ... DB 작업 1 ...
        innerService.innerMethod();
        // ... DB 작업 3 ...
    }
}
```

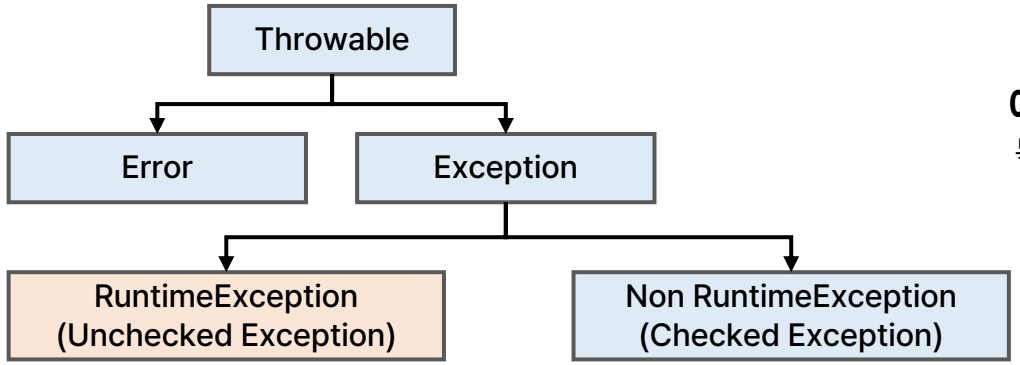
InnerService는 OuterService Bean 생성 시점에
AOP Proxy로 Wrapping 된 후 InnerService가 DI
→ 개별 트랜잭션으로 처리

```
@Service
public class InnerService {
    @Transactional // (propagation = Propagation.REQUIRED)
    public void innerMethod() {
        // ... DB 작업 2 ...
        // 만약 여기서 예외가 발생하여 롤백 되면, outerMethod의 'DB 작업 1'까지 모두 롤백 된다.
    }
}
```

# [참고] 트랜잭션 예외와 롤백 규칙

- @Transactional은 기본적으로 RuntimeException(Unchecked Exception)과 Error가 발생했을 때만 롤백
- Checked Exception: Exception을 상속받는 Checked Exception을 try-catch로 잡고 아무것도 하지 않으면, 트랜잭션은 정상적으로 커밋 되어 데이터 정합성 문제 발생 가능
- 해결 방안: 모든 예외(Exception)에 대해 롤백하고 싶다면 rollbackFor 속성을 사용

```
@Transactional(rollbackFor = Exception.class)
public void process() throws MyCustomException {
    // ... 로직 ...
}
```




코드문제는 알아서

NullPointException,
IllegalArgumentException

IOException, FileNotFoundException, ClassNotFoundException,
SQLException (DB 접근 오류) 등

**예외를 반드시 인지하고 대책을 세워라**

복구 가능한 예외(Recoverable Exception)

- 네트워크 임시 오류 (재시도)
- 파일 잠김(대기후 재시도)
- DB Lock 대기 (sleep 후 retry)
- 외부 API (BackOff)

# 트랜잭션 동시성 제어 (Consistency Control)

- 여러 사용자가 **동시에 데이터에 접근하고 수정**할 때 발생할 수 있는 데이터 불일치 문제 **(Multi-Thread, Multi-Process)**
- JPA에서 동시성 제어(concurrency control) 는 @Transactional 자체가 해결해주는 것이 아님

| 문제 유형 | | 발생 여부 | 설명 |
|---|---|---|---|
| Lost Update | 두 사용자가 동시에 같은 User의 point를 증가시키는 경우<br>예) 두 Thread가 동시에 값을 읽고, 각각 업데이트 | O | 충돌 감지 불가 |
| Dirty Read | Isolation 이 필수적으로 지원하기 때문에 발생하지 않음 | X | 기본 ReadCommitted로 방지됨 |
| 반복 읽기 불일치<br>Non-repeatable Read | 한 트랜잭션에서 같은 엔티티를 두 번 읽는 사이에<br>다른 트랜잭션이 데이터를 바꾼 경우 | O | JPA 1차 캐시와 DB 상태 불일치 |
| 환영 읽기<br>Phantom Read | **트랜잭션이 동일한 조건으로 동일한 쿼리를 두 번 실행했을 때,**<br>두 번째 조회에서 **새로운 Row가 추가되어 나타나는 현상** | O | Lock 없으면 방지 불가 |
| 경쟁 조건<br>Race Condition | **동시성 경쟁으로 비즈니스 로직이 잘못 처리**<br>예) 두 스레드 모두 재고 감소 실행 | O | 재고 감소/포인트 증가 등에서 치명적 |

# 트랜잭션 동시성 제어 (Concurrency Control)

- 여러 사용자가 동시에 데이터에 접근하고 수정할 때 발생할 수 있는 데이터 불일치 문제를 방지하기 위한 제어
- JPA는 낙관적(Optimistic) Lock과 비관적(Pessimistic) Lock을 제공

| 구분 | 개념 |
|---|---|
| 낙관적 Lock | 데이터 충돌이 "드물 것"이라고 낙관적으로 가정하고,락을 걸지 않은 채 자유롭게 읽고 수정하되,<br>커밋 시점에 충돌을 감지하여 처리하는 방식.<br>✓ 락을 걸지 않음<br>✓ 충돌은 commit 단계에서 version 비교로 감지<br>✓ 충돌 발생 시 → 예외(OptimisticLockException) 발생 → 재시도 필요 |
| 비관적 Lock | 데이터 충돌이 **자주 발생할 것**이라고 비관적으로 가정, 데이터를 조회하는 순간부터 다른 트랜잭션이<br>읽거나 수정하지 못하도록 DB에 락을 걸어 충돌을 사전에 차단하는 방식<br>• 조회 시점부터 Lock되며 해당락이 해재될때 까지 접근 불가<br>• 다른 트랜잭션은 UPDATE 또는 SELECT FOR UPDATE 시 대기(blocking)<br>• 충돌 자체를 원천적으로 방지데이터에 접근 시점부터 **데이터베이스 레코드에 직접 Lock**을 거는 방식 |

# 트랜잭션 동시성 제어 – 낙관적 락 (Optimistic Lock)

- **장점**: DB에 직접적인 lock을 걸지 않으므로 부하가 적고 성능이 좋음 (읽기 작업이 많은 환경에 유리)
- **단점**: 충돌이 발생하면 개발자가 직접 예외를 처리하고 재시도 로직 등을 구현해야 함.

```
@Entity
public class Article {
    @Id
    @GeneratedValue
    private Long id;

    private String content;

    @Version
    private Long version; // 버전을 관리할 필드
}
```

- 엔티티에 @Version 어노테이션이 붙은 필드(주로 숫자 타입)를 추가
- 데이터를 조회할 때 이 버전 정보도 함께 가져옴.
- 데이터를 수정하고 트랜잭션을 커밋할 때, 현재 데이터베이스의 버전과 내가 조회했던 시점의 버전을 비교
- 버전이 일치하면 데이터를 수정하고 버전을 1 증가시킴
- 버전이 일치하지 않으면 (다른 트랜잭션이 먼저 수정했다는 의미) ObjectOptimisticLockingFailureException 예외를 발생시키고 수정을 실패 처리
- 재 실행 과정 필요

@Version을 Entity에 설정하고 @Lock을 @Transactioal에서 사용하지 않으면 자동으로 낙관적 Lock으로 동작

# 낙관적 락 (Optimistic Lock) Retry

- 낙관적 Lock은 @Transcational 이 종료되면서 commit 시점에 다른 곳에서 업데이트를 먼저 했으면 Failure
- 재 처리를 위해서는 @Retryable 사용해서 재처리

```
@Service
@RequiredArgsConstructor
public class StockService {
    private final ProductRepository productRepository;

    @Retryable(
        include = { OptimisticLockingFailureException.class, OptimisticLockException.class },
        maxAttempts = 3,
        backoff = @Backoff(delay = 100, multiplier = 2) // 100ms → 200ms → 400ms
    )
    @Transactional
    public void decreaseStock(Long productId, int qty) {
        var p = productRepository.findById(productId).orElseThrow();
        if (p.getStock() < qty) throw new IllegalArgumentException("재고 부족");
        p.setStock(p.getStock() - qty); // flush/commit 시 버전 체크
    }

    @Recover
    public void recover(OptimisticLockingFailureException e, Long productId, int qty) {
        // 재시도 끝나도 실패하면 알림/대안 처리
        throw e;
    }
}
```

# 비관적 락 (Pessimistic Lock)

- **장점**: 데이터 정합성을 확실하게 보장 (충돌이 잦은 환경에서 데이터 무결성을 유지하는 데 효과적)
- **단점**: 데이터베이스에 직접 락을 걸기 때문에 성능 저하가 발생 가능
  (다른 트랜잭션의 대기 시간이 길어져 전체적인 시스템 성능에 영향을 주고, 데드락 발생 가능)

**Lock 모드:**

- **PESSIMISTIC_WRITE** : 데이터를 수정하기 위해 설정(다른 트랜잭션은 해당 데이터를 읽거나 쓸 수 없음)
- **PESSIMISTIC_READ** : 다른 트랜잭션이 해당 데이터를 수정하는 것을 방지(DB에 따라 읽기는 허용될 수 있음)
- **default 는 NONE**: 그냥 조회만 하고 DB 락 없음

**사용 방법**

```
public interface ArticleRepository extends JpaRepository<Article, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Article a where a.id = :id")
    Optional<Article> findByIdWithWriteLock(@Param("id") Long id);
}
```

# [실습] ProductService와 UserService에 트랜잭션 적용하기

**현행 코드에 다음 사항을 반영해보자**

- ProductService와 UserService Class Level로 @Transactional(readOnly=true)로 설정해보자
- 생성/변경/삭제 메소드에 대해서는 @Transactional 로 method 단위로 설정하자

이렇게 설정하는 이유에 대해 서로에게 설명해보자

# [실습] ProductService와 UserService에 트랜잭션 적용하기

- Product 과 User에 대한 낙관적 락을 설정할 수 있도록 변경해보자
- resources/data.sql에서 users와 products 컬럼에 version=0을 추가해보자

낙관적 락은 어떤 방식으로 동시성 제어를 수행하는지 설명해보자



# REST API 문서화

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# REST API 문서화

- Spring Boot 3.0 이상 버전에서는 springdoc-openapi 라이브러리를 사용하여 API 문서 자동화 지원
- OpenAPI 3 명세 기반의 문서와 Swagger UI를 자동 생성

```
<!-- Tomcat 기반 OpenAPI 지원 -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>3.0.3a</version>
</dependency>
```

```
<!-- WebFlux (Netty 기반) OpenAPI 지원 -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webflux-ui</artifactId>
    <version>3.0.3</version>
</dependency>
```

springdoc : Open API Specification (OAS) 문서를 자동 생성
swagger ui: 그 문서를 보기 쉽게 표시

❖ 별도의 추가 설정 없이 위 의존성만 추가하면 기본 문서 생성

- Swagger UI: http://localhost:8080/swagger-ui/index.html
- OpenAPI JSON 문서: http://localhost:8080/v3/api-docs

# REST API 문서화

- application.yaml 설정 방법

```
springdoc:
  api-docs:
    path: /api-docs
    enabled: true
  swagger-ui:
    path: /swagger-ui.html
    enabled: true
    operations-sorter: alpha
    tags-sorter: alpha
    display-request-duration: true
    try-it-out-enabled: true
  show-actuator: false
```

- Swagger UI: http://localhost:8080/swagger-ui.html
- OpenAPI JSON 문서: http://localhost:8080/api-docs

# [참고] OAS 어노테이션 태그

| 어노테이션 | 적용 위치 | 주요 용도/설명 |
|---|---|---|
| @Tag | 클래스(Controller) | API 그룹 및 설명. Swagger UI에서 그룹핑 |
| @Operation | 메서드 | 개별 API의 요약/설명/주석 작성 |
| @Parameter | 메서드 파라미터 | 개별 파라미터 설명 및 예시/기본값 지정 |
| @RequestBody | 메서드 파라미터 | 요청 본문 객체의 설명 및 스키마 자동생성 |
| @ApiResponse | 메서드 | 응답 코드별 설명 및 스키마 지정 |
| @Schema | DTO 클래스/필드 | 필드별 설명, 예시, 타입, 필수여부, 형식 등 |
| @Hidden | 클래스/메서드/필드 | 문서화에서 숨기기(표시되지 않음) |

# [참고] OAS 문서 작성 예시

- 컨트롤러, 메서드, DTO에 어노테이션으로 부가정보

```
@Tag(name = "Hello API", description = "Hello 엔드포인트 관련 API")
@RestController
public class HelloController {

    @Operation(summary = "Hello 메시지 반환", description = "입력받은 이름을 이용하여 환영 메시지를 반환합니다.")
    @ApiResonses({
        @ApiResponse(responseCode = "200", description = ”정상적인 인사”),
        @ApiResponse(responseCode = ”400", description = ”잘못된 요청”)
    })
    @GetMapping("/hello")
    public HelloResponse hello(
        @Parameter(description = "사용자 이름", example = "홍길동")
        @RequestParam String name
    ) {
        return new HelloResponse("안녕하세요, " + name + "님!");
    }
}

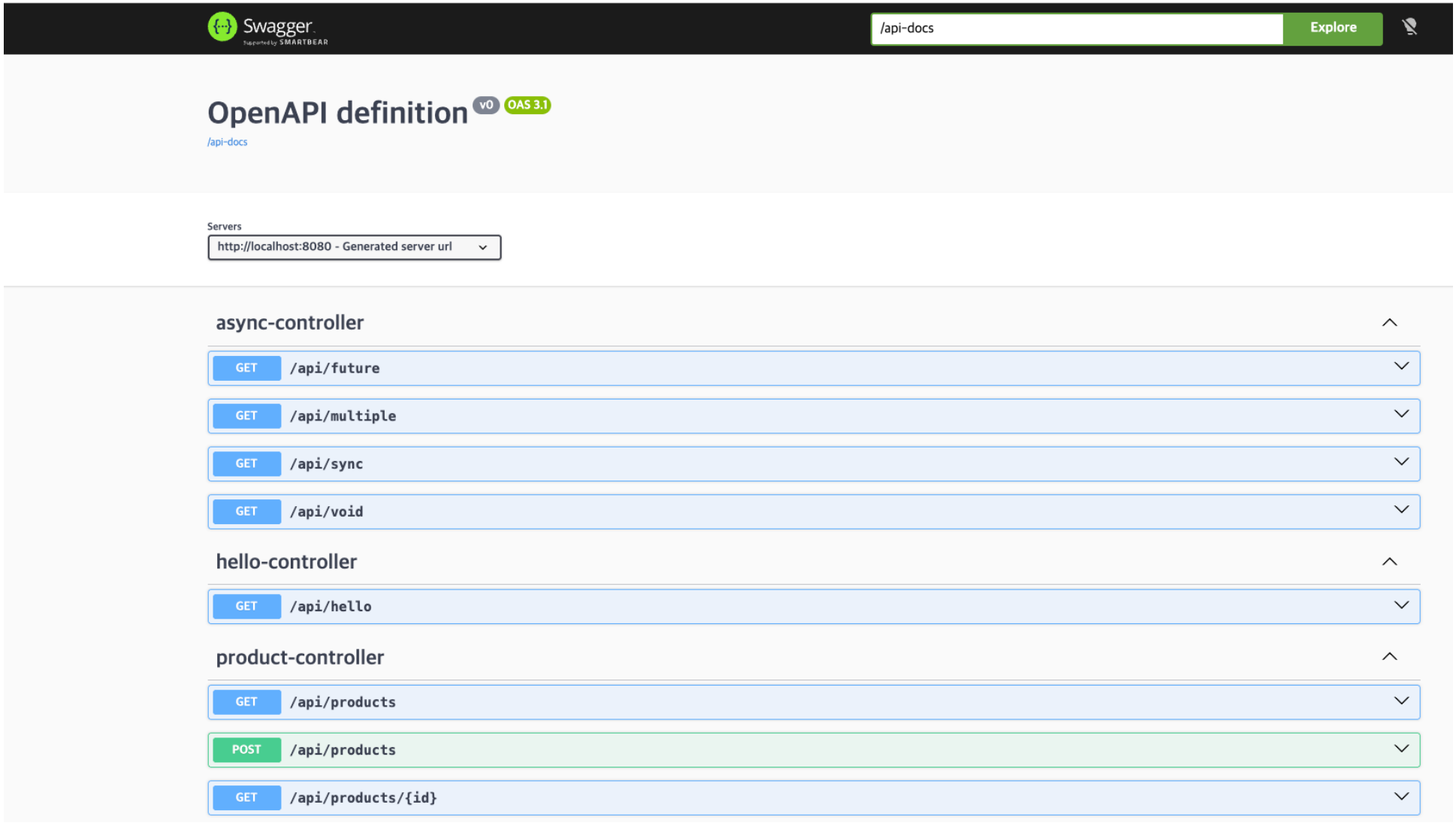
public class HelloResponse {
    @Schema(description = "환영 메시지", example = "안녕하세요, 홍길동님!")
    private String message;

    // getter, setter
}
```

# REST API 문서화

소스 위치: spring/11.swagger/

**http://localhost:8080/swagger-ui.html**



# REST API 문서화

## http://localhost:8080/v3/api-docs


```
{
  "openapi": "3.1.0",
  "info": {
    "title": "OpenAPI definition",
    "version": "v0"
  },
  "servers": [
    {
      "url": "http://localhost:8080",
      "description": "Generated server url"
    }
  ],
  "paths": {
    "/api/users/{id}": {
      "get": {
        "tags": [
          "user-controller"
        ],
        "operationId": "getUserById",
        "parameters": [
          {
            "name": "id",
            "in": "path",
            "required": true,
            "schema": {
              "type": "integer",
              "format": "int64"
            }
          }
        ],
        "responses": {
          "200": {
            "description": "OK",
            "content": {
              "*/*": {
                "schema": {
                  "$ref": "#/components/schemas/UserResponse"
                }
              }
            }
          }
        }
      },
```

# [실습] myapp에 Swagger 적용하기

소스 위치: spring/11.swagger/

myapp에 open api를 적용해보자

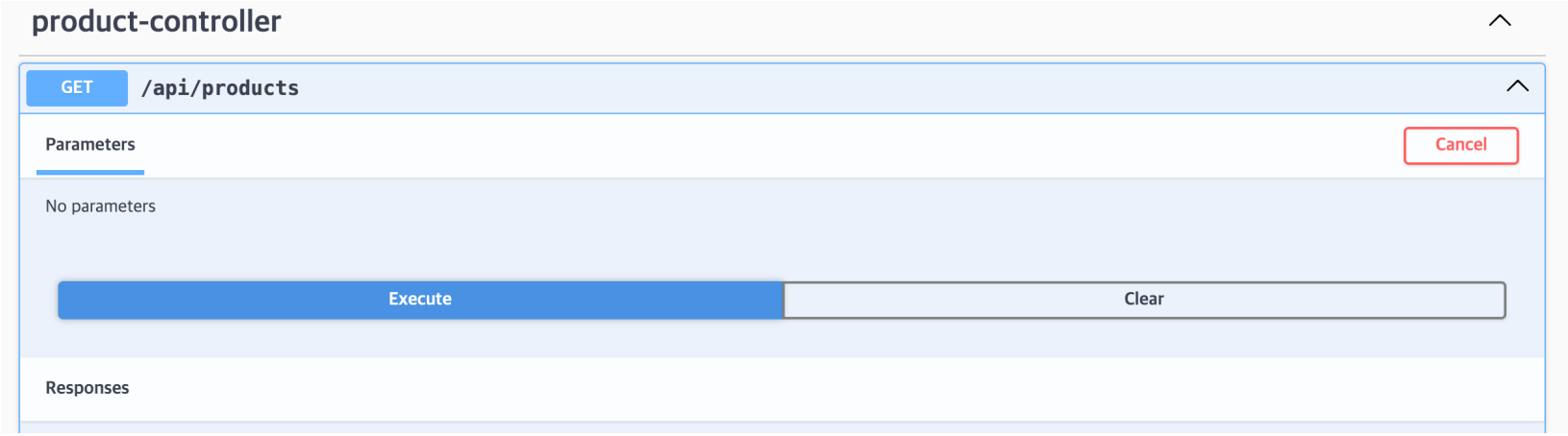
## 설정 하기

1. pom.xml 에 springdoc-openapi-starter-webmvc-ui 3.0.3 을 주입
2. application.yaml 에 springdoc 속성 주입

## myapp 실행 및 접속하기

1. myapp을 실행하고 난뒤
2. http://localhost:8080/swagger-ui.html

## swagger-ui에서 직접 테스트해보자





# Actuator

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

# Actuator 란?

Spring Boot Actuator는 애플리케이션의 모니터링 및 관리를 위한 다양한 기능을 제공하는 라이브러리

운영 환경에서 애플리케이션의 상태(health), 메트릭(metrics), 환경 설정(properties), 로그 설정 등을 확인하고 관리 지원

| 기능 | 설명 | 엔드포인트 |
|---|---|---|
| **Health Check** | 애플리케이션 상태 확인 | /actuator/health |
| **Metrics** | JVM, CPU, 메모리, GC, 요청 수 등 성능 지표 제공 | /actuator/metrics |
| **Prometheus Metrics** | Metrics 정보를 Prometheus 표준으로 전환하여 제공 | /actuator/prometheus |
| **Info** | 앱 버전, 빌드 정보 등 메타데이터 제공 | /actuator/info |
| **Env** | 환경 변수, 설정 값 노출 | /actuator/env |
| **Loggers** | 로깅 레벨 동적 변경 가능 | /actuator/loggers |
| **Beans** | IoC 컨테이너에 등록된 Bean 정보 제공 | /actuator/beans |

```
<!-- Actuator -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<!-- Actuator metrics 정보를 Prometheus 포맷으로 노출 지원 -->
<dependency>
<groupId>io.micrometer</groupId>
<artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

# Actuator 설정 방법

- Actuator 활성화를 위한 application.yaml 설정

```
management:
  server:
    port: 8080 # Actuator 포트 설정
  endpoints:
    web:
      exposure:
        include: "*" # Prometheus endpoint 노출
  endpoint:
    health:
      probes:
        enabled: true # LivenessState, ReadinessState 활성화
  health:
    livenessState:
      enabled: true # LivenessState 활성화
    readinessState:
      enabled: true # ReadinessState 활성화
  metrics:
    enable:
      all: true # 모든 기본 메트릭 활성화
```

# Actuator 접속 방법

Actuator 접속 방법: http://localhost:8080/actuator

- management.server.port 지정에 따라 port 가 변경될 수 있음
- default는 server.port 와 동일


```
{
  "_links": {
    "self": {
      "href": "http://localhost:8081/actuator",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8081/actuator/beans",
      "templated": false
    },
    "caches-cache": {
      "href": "http://localhost:8081/actuator/caches/{cache}",
      "templated": true
    },
    "caches": {
      "href": "http://localhost:8081/actuator/caches",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8081/actuator/health",
      "templated": false
    },
```

# Actuator 제공 API

- Spring Boot 애플리케이션의 상태 확인, 시스템 모니터링, 관리(Management), 운영 자동화 등을 위한 운영용 엔드 포인트(REST API)를 자동으로 제공하는 모듈

| 기능 구분 | 설명 | 엔드 포인트 |
|---|---|---|
| 헬스 체크 | 시스템, DB 등 주요 컴포넌트 상태 확인 | /actuator/health |
| 애플리케이션 정보 | 버전, 이름 등 info 정보 | /actuator/info |
| 메트릭 | JVM, CPU, 메모리, 트래픽, GC, 요청 수, response time 등 | /actuator/metrics |
| 환경변수 | 환경, 프로퍼티, 시스템 변수 | /actuator/env |
| 빈 정보 | 스프링 빈 목록, 상태, 의존관계 | /actuator/beans |
| 스레드 덤프 | 스레드 상태/스택 추적 | /actuator/threaddump |
| 로그 레벨 동적 변경 | 런타임에 로그 레벨 변경 | /actuator/loggers |
| 캐시 정보 | Spring Cache 정보 | /actuator/caches |
| HTTP 트레이스 | 최근 HTTP 요청/응답 추적 | /actuator/httptrace |
| Mappings | RequestMapping 정보 | /actuator/mappings |

# [실습] actuator 적용하기

01.training-code/15.actuator

myapp에 actuator 와 prometheus를 적용해보자

## 설정 하기

1. pom.xml 에 actuator 와 Prometheus 의존성을 주입
2. application.yaml 에 management 속성을 추가하기

## myapp 실행 및 접속하기

1. myapp을 실행하고 난뒤

## metrics endpoint 확인하기

| 기능 구분 | 설명 | 엔드 포인트 |
|---|---|---|
| 헬스 체크 | 시스템, DB 등 주요 컴포넌트 상태 확인 | /actuator/health |
| 애플리케이션 정보 | 버전, 이름 등 info 정보 | /actuator/info |
| 메트릭 | JVM, CPU, 메모리, 트래픽, GC, 요청 수, response time 등 | /actuator/metrics |
| 환경변수 | 환경, 프로퍼티, 시스템 변수 | /actuator/env |
| 빈 정보 | 스프링 빈 목록, 상태, 의존관계 | /actuator/beans |
| 스레드 덤프 | 스레드 상태/스택 추적 | /actuator/threaddump |
| 로그 레벨 동적 변경 | 런타임에 로그 레벨 변경 | /actuator/loggers |
| 캐시 정보 | Spring Cache 정보 | /actuator/caches |
| HTTP 트레이스 | 최근 HTTP 요청/응답 추적 | /actuator/httptrace |
| Mappings | RequestMapping 정보 | /actuator/mappings |

# [실습] actuator 적용하기

**아래의 API를 직접 호출하고 그 결과를 확인해보자**

| API PATH | 설명 |
|---|---|
| /actuator/health | 애플리케이션의 전체적인 헬스 상태(UP/DOWN)를 반환하는 기본 엔드포인트 |
| /actuator/health/liveness | Liveness 상태(생존 신호)를 확인<br>애플리케이션이 살아있는지 판별하며,<br>Kubernetes 환경에서 컨테이너의 재시작 여부를 결정할 때 주로 연동 |
| /actuator/health/readiness | Readiness 상태(준비 완료 신호)를 확인<br>트래픽을 받을 준비가 되었는지 판별하며,<br>준비가 안 되었다면 쿠버네티스 서비스 라우팅 대상에서 일시적으로 제외 |
| /actuator/metrics | 수집 가능한 모든 메트릭 지표(JVM 메모리, CPU 사용량, HTTP 요청 수 등)의<br>이름 목록을 JSON으로 반환 |
| /actuator/metrics/{metric.name} | 특정 메트릭 명칭(예: jvm.memory.used, http.server.requests)을 경로에<br>입력하여 구체적인 수치와 세부 지표를 단건 조회 |
| /actuator/prometheus | 프로메테우스 서버가 데이터를 긁어갈 수 있도록,<br>수집한 메트릭들을 프로메테우스 전용 텍스트 포맷으로 변환하여 노출 |

# Actuator - Logging level 설정하기

Spring Boot Actuator의 Loggers 엔드포인트(POST /actuator/loggers/{name})는

애플리케이션을 재배포하거나 재시작하지 않고도 특정 패키지/클래스의 로깅 레벨을 동적으로 변경

## 주요 유스케이스

**운영 장애 추적**

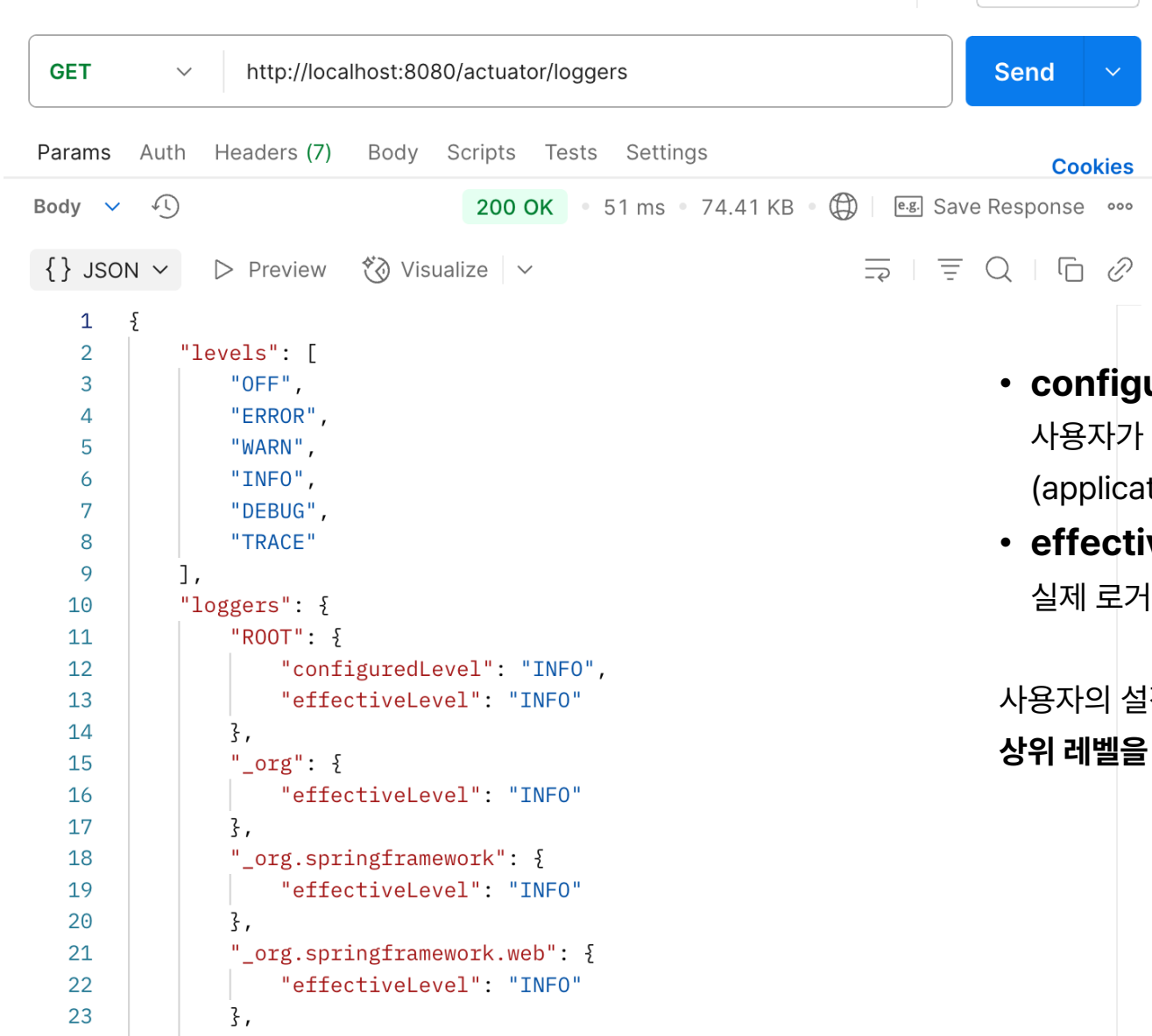
- 평소 로깅 레벨 INFO, 운영 에러 추적 시 DEBUG/TRACE 로 전환

**특정 써드파티 라이브러리/프레임워크 내부 내부 쿼리/동작 확인**

- 사용중인 라이브러리 패키지만 핀포인트로 로그 레벨을 조정
- 예) org.hibernate.SQL 또는 org.springframework.security 패키지 level 변경

# Actuator - Logging level (1/2)

GET http://localhost:8080/actuator/loggers



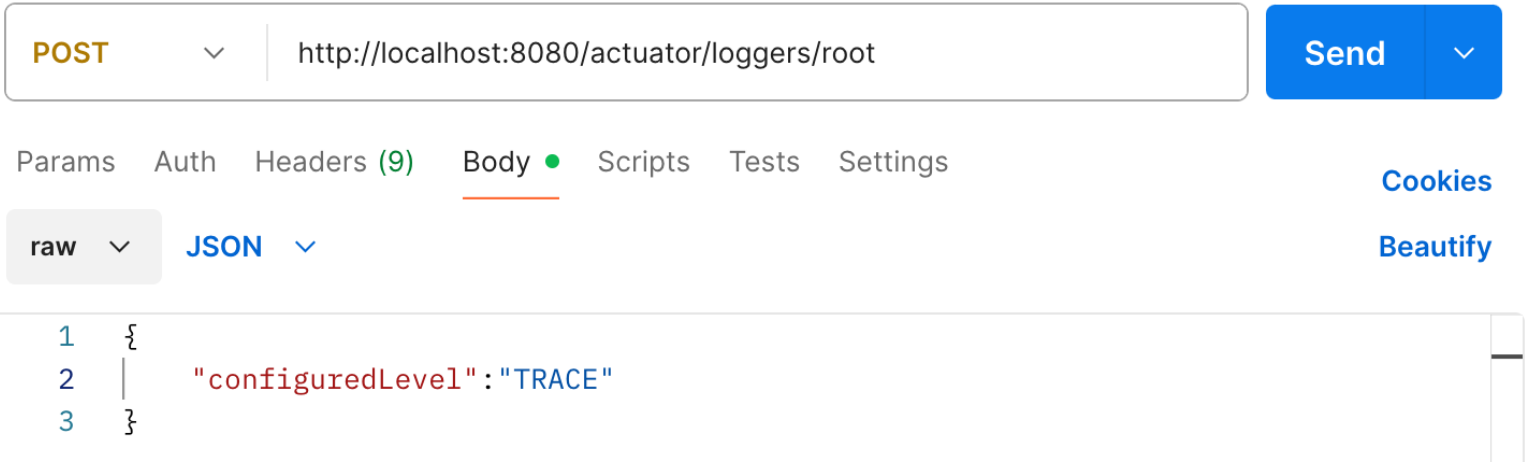

- **configuredLevel**:
  사용자가 명시적으로 설정한 로그 레벨
  (application.yaml 설정 값)
- **effectiveLevel**:
  실제 로거에 적용되는 레벨

사용자의 설정이 명시적으로 없으면
**상위 레벨을 상속**받아서 effectiveLevel이 결정

# Actuator Logging level (2/2)

- 실행되고 있는 Spring boot application의 logger 설정을 변경을 통해 추적

POST http://localhost:8080/actuator/loggers/{packageName}



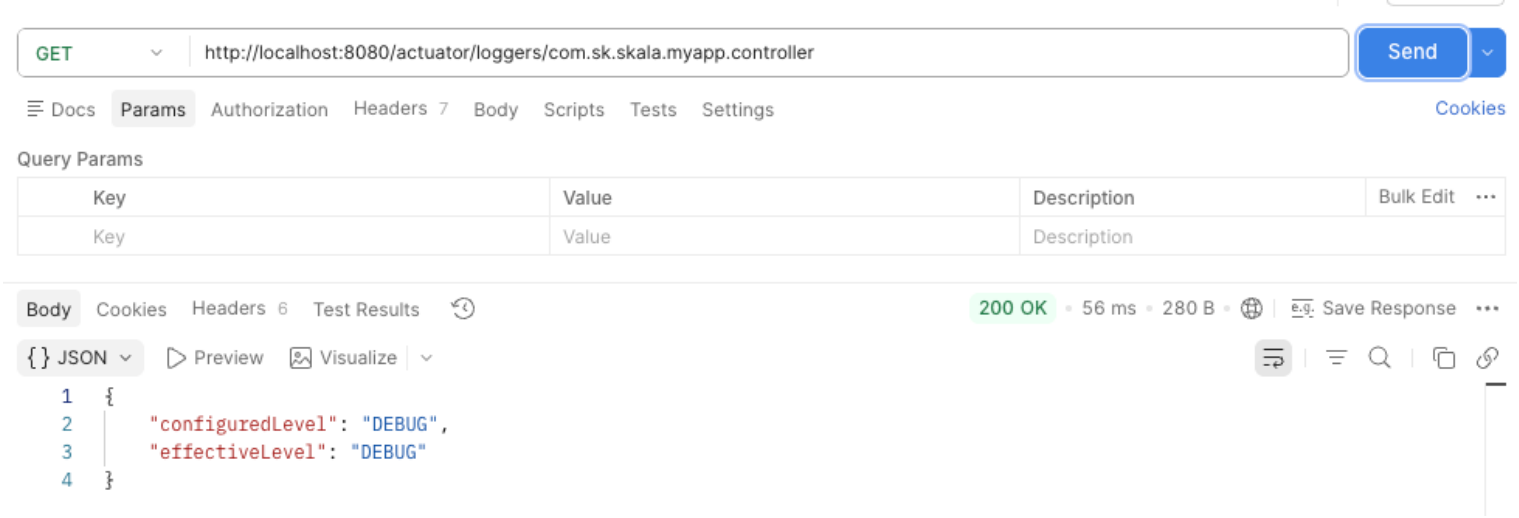

- root 대신 자신의 패키지 명을 지정해서 패키지 별 로그 레벨을 변경할 수 있음
  예) http://localhost:8080/actuator/loggers/com.sk.skala.myapp.controller

# [실습] Log Level 변경해보기

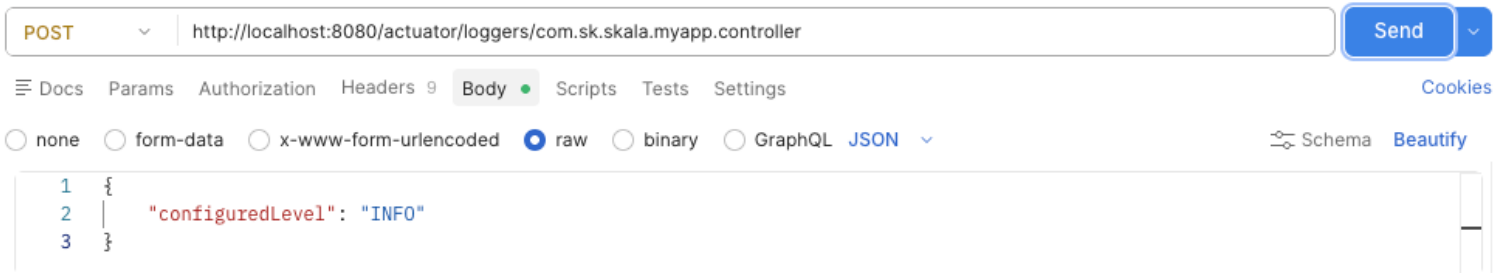
myapp에 actuator를 적용하고 acatuator metrics를 검색해보자

com.sk.skala.myapp.controller의 LOG_LEVEL을 INFO로 변경해보자

## 1. 패키지  com.sk.skala.myapp.controller 에 대한 Log Level 확인하기




## 2. 패키지 com.sk.skala.myapp.controller에 대한 Log Level 변경하기 (DEBUG → INFO)




## 3. package com.sk.skala.myapp.controller에 대한 Log LEVEL 변경 확인하기

# Actuator - Health

- 기본 Endpoint:  /actuator/health
- 애플리케이션의 전반적인 상태를 표시 (UP, DOWN, OUT_OF_SERVICE 등)

### Health 상태 유형

- **UP** → 애플리케이션 및 주요 컴포넌트가 정상 동작 중
- **DOWN** → 핵심 컴포넌트 중 하나라도 장애 발생
- **OUT_OF_SERVICE** → 애플리케이션이 살아는 있으나 외부 요청을 처리할 수 없는 상태
  (ex. 관리자가 의도적으로 서비스 중지)
- **UNKNOWN** → 상태를 알 수 없음 (HealthIndicator 미구현 or 예외 발생)

### 기본 제공 Health Indicator

| Indicator | 역할 | 판단 기준 |
|---|---|---|
| DiskSpaceHealthIndicator | 디스크 용량 확인 | 여유 공간이 기준치 미만이면 DOWN |
| DataSourceHealthIndicator | DB 연결 확인 | DB 연결 불가 시 DOWN |
| RedisHealthIndicator | Redis 연결 확인 | Redis 서버 미응답 → DOWN |
| MongoHealthIndicator | MongoDB 연결 확인 | 연결 실패 → DOWN |
| CassandraHealthIndicator | Cassandra 연결 확인 | 연결 실패 → DOWN |
| PingHealthIndicator | 단순 UP 체크 | 기본은 항상 UP |

참고) 사용자가 Indicator 를 별도 구성 가능 (HealthIndicator를 상속받아서 구현 가능)

# Actuator – Readiness / Liveness Probe

- spring boot의 생존 여부와 준비 상태를 모니터링하기 위한 상태 정보를 제공
- 단지 정보를 제공만 실제 실행은 외부 상태 확인 및 Post-Action은 별도 플랫폼 필요 (Kubernetes 연동)

## /actuator/health/readiness

**의미**: 지금 트래픽을 **받을 준비**가 되었나?

**상태 정보**

- ACCEPTING_TRAFFIC → 준비됨 (200 OK)
- REFUSING_TRAFFIC → 준비 안 됨 (503)
  (시작 전/그레이스풀 셧다운 중/사용자 전환 등)

## /actuator/health/liveness

**의미**: 프로세스가 살아있어(진행 가능 상태야)?

**상태 정보**

- CORRECT → 정상 (200 OK)
- BROKEN → 회복 불가 (503)
  (앱 실패/컨텍스트 깨짐 등)

# Actuator - Health Readiness Probe

## 특징

- 기본 Endpoint:  /actuator/health/readiness
- 애플리케이션이 요청을 받을 준비가 되었는지 확인
- 상태
  - 정상(UP) 상태가 되면 200 OK 응답
  - 비정상(DOWN, OUT_OF_SERVICE) 상태가 되면 기본적으로 503 Service Unavailable 응답

## 상태 별 의미

### 정상(UP)

- Spring Boot 애플리케이션이 완전히 초기화 완료됨.
- HealthIndicator가 모든 구성 요소가 정상(UP)이라고 판단.

### 비정상(DOWN)이 되는 경우

- **데이터베이스 연결 실패**
  DataSourceHealthIndicator가 DOWN을 리턴하면 전체 Readiness도 DOWN
- **외부 API 또는 필수 서비스 연결 불가**
  예: Redis, Kafka, API Gateway 등
- **애플리케이션이 종료 중(OUT_OF_SERVICE) 상태**
  Kubernetes가 SIGTERM을 보내고 종료 과정 중이면 Readiness가 OUT_OF_SERVICE로 변경됨.

# Actuator - Health Liveness Probe

## 특징

- 기본 Endpoint:  /actuator/health/liveness
- 애플리케이션 프로세스가 정상 동작 가능한지 확인
- 상태
  - 정상(UP) 상태가 되면 200 OK 응답
  - 비정상(DOWN) 상태가 되면 기본적으로 500 Internal Server Error 응답

## 상태 별 의미

### 정상(UP)

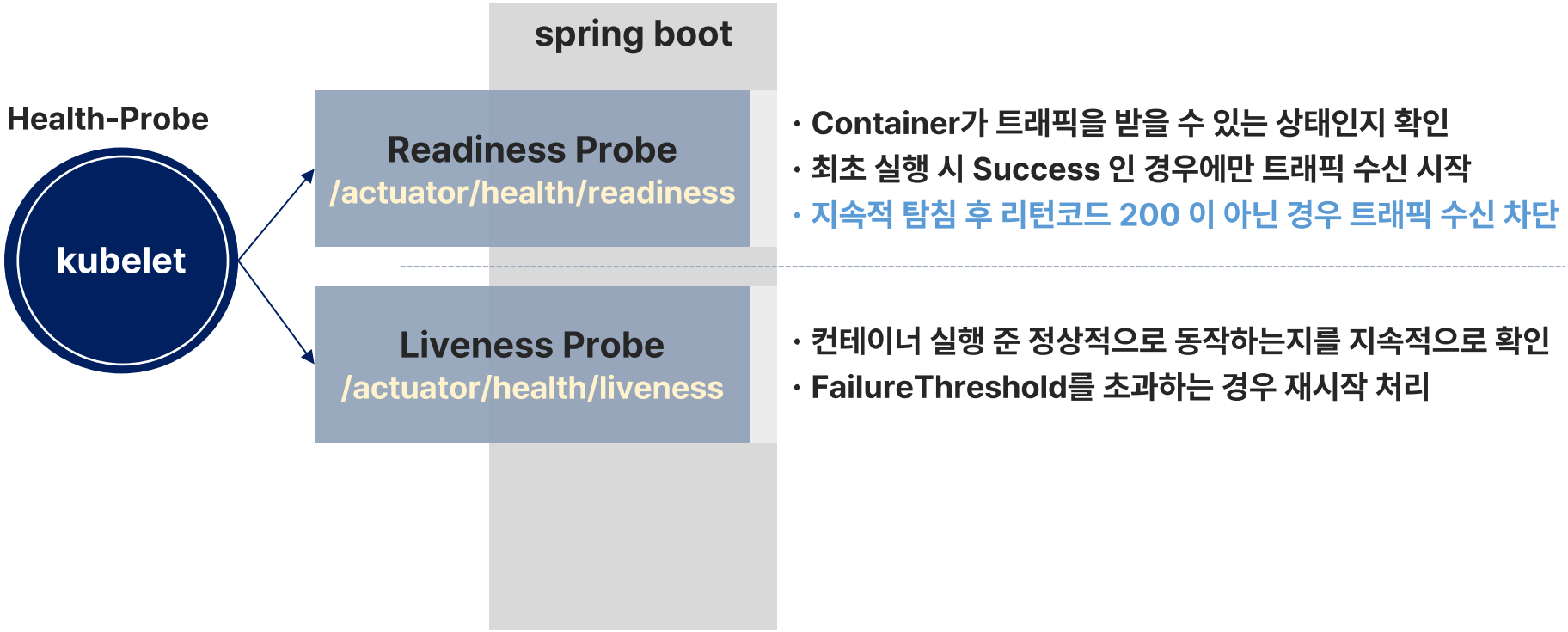
- 애플리케이션이 기본적인 동작을 수행 가능한 상태
- SpringContext가 정상적으로 유지되고 있음

### 비정상(DOWN)이 되는 경우

- JVM이 OutOfMemory(OOM) 에러 발생
  예: Heap Memory 부족 → DOWN
- 스레드가 Blocked 또는 Deadlock 발생
  예: synchronized 블록에서 대기 무한 루프
- 애플리케이션 내부적으로 치명적인 에러 발생
  예: 중요 구성 요소(예: Message Queue Consumer)가 작동 불가능한 상태

# Actuator – k8s 기반 상태 모니터링 및 PostAction

- spring boot의 애플리케이션이 컨테이너로 Kubernetes 환경에 배포될 경우 컨테이너 내 애플리케이션 (spring boot)의 준비 상태를 확인하거나 운영 상태를 확인하기 위해
- Actuator Health 의 Liveness Probe와 Readiness Probe 를 사용



# [참고] Kubernete health 설정 방법

spring boot의 readinessProbe와 livenessProbe를 통해 애플리케이션 상태에 대한 자동 복구 기능 지원

```
spec:
  containers:
    - name: my-spring-app
      image: my-registry.com/my-spring-app:latest
      ports:
        - containerPort: 8080
      livenessProbe:
        httpGet:
          path: /actuator/health/liveness
          port: 8081
        initialDelaySeconds: 5   # 컨테이너 시작 후 5초 뒤 체크 시작
        periodSeconds: 10        # 10초마다 체크
        timeoutSeconds: 3        # 응답 없으면 3초 후 실패 간주
        failureThreshold: 3      # 3번 연속 실패 시 컨테이너 재시작

      readinessProbe:
        httpGet:
          path: /actuator/health/readiness
          port: 8081
        initialDelaySeconds: 5   # 컨테이너 시작 후 5초 뒤 체크 시작
        periodSeconds: 10        # 10초마다 체크
        timeoutSeconds: 3        # 응답 없으면 3초 후 실패 간주
        failureThreshold: 3      # 3번 연속 실패 시 트래픽에서 제외
```



# 수고하셨습니다.

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C